有赞一面

说说CSS选择器以及这些选择器的优先级

- `!important`
- 内联样式（ `1000` ）
- ID选择器（ `0100` ）
- 类选择器/属性选择器/伪类选择器（ `0010` ）
- 元素选择器/伪元素选择器（ `0001` ）
- 关系选择器/通配符选择器（ `0000` ）

你知道什么是BFC么

> 小提示：这个问题重点是BFC是什么，BFC触发的条件有哪些，BFC可以干什么。这里我试着讲解了一下Boostrap的清除浮动（display：table创建匿名table-cell间接触发BFC），如果有看到别的场景使用或者自身有使用的场景可以尝试讲解一下使用技巧。这样可以让面试官觉得你不仅仅知道他问的东西是什么，你还能很好的使用它。

什么是BFC

> BFC 全称为块级格式化上下文 (Block Formatting Context) 。BFC是 W3C CSS 2.1 规范中的一个概念，它决定了元素如何对其内容进行定位以及与其他元素的关系和相互作用，当涉及到可视化布局的时候，Block Formatting Context提供了一个环境，HTML元素在这个环境中按照一定规则进行布局。一个环境中的元素不会影响到其它环境中的布局。比如浮动元素会形成BFC，浮动元素内部子元素的主要受该浮动元素影响，两个浮动元素之间是互不影响的。这里有点类似一个BFC就是一个独立的行政单位的意思。可以说BFC就是一个作用范围，把它理解成是一个独立的容器，并且这个容器里box的布局与这个容器外的box毫不相干。

触发BFC的条件

- 根元素或其它包含它的元素
- 浮动元素 (元素的 float 不是 none)
- 绝对定位元素 (元素具有 position 为 absolute 或 fixed)
- 内联块 (元素具有 display: inline-block)

夸克扫描王
极速扫描，就是高效

- 表格单元格 (元素具有 display: table-cell，HTML表格单元格默认属性)
- 表格标题 (元素具有 display: table-caption, HTML表格标题默认属性)
- 具有overflow 且值不是 visible 的块元素
- 弹性盒（flex或inline-flex）
- `display: flow-root`
- `column-span: all`

BFC的约束规则

- 内部的盒会在垂直方向一个接一个排列（可以看作BFC中有一个的常规流）
- 处于同一个BFC中的元素相互影响，可能会发生外边距重叠
- 每个元素的margin box的左边，与容器块border box的左边相接触(对于从左往右的格式化，否则相反)，即使存在浮动也是如此
- BFC就是页面上的一个隔离的独立容器，容器里面的子元素不会影响到外面的元素，反之亦然
- 计算BFC的高度时，考虑BFC所包含的所有元素，连浮动元素也参与计算
- 浮动盒区域不叠加到BFC上

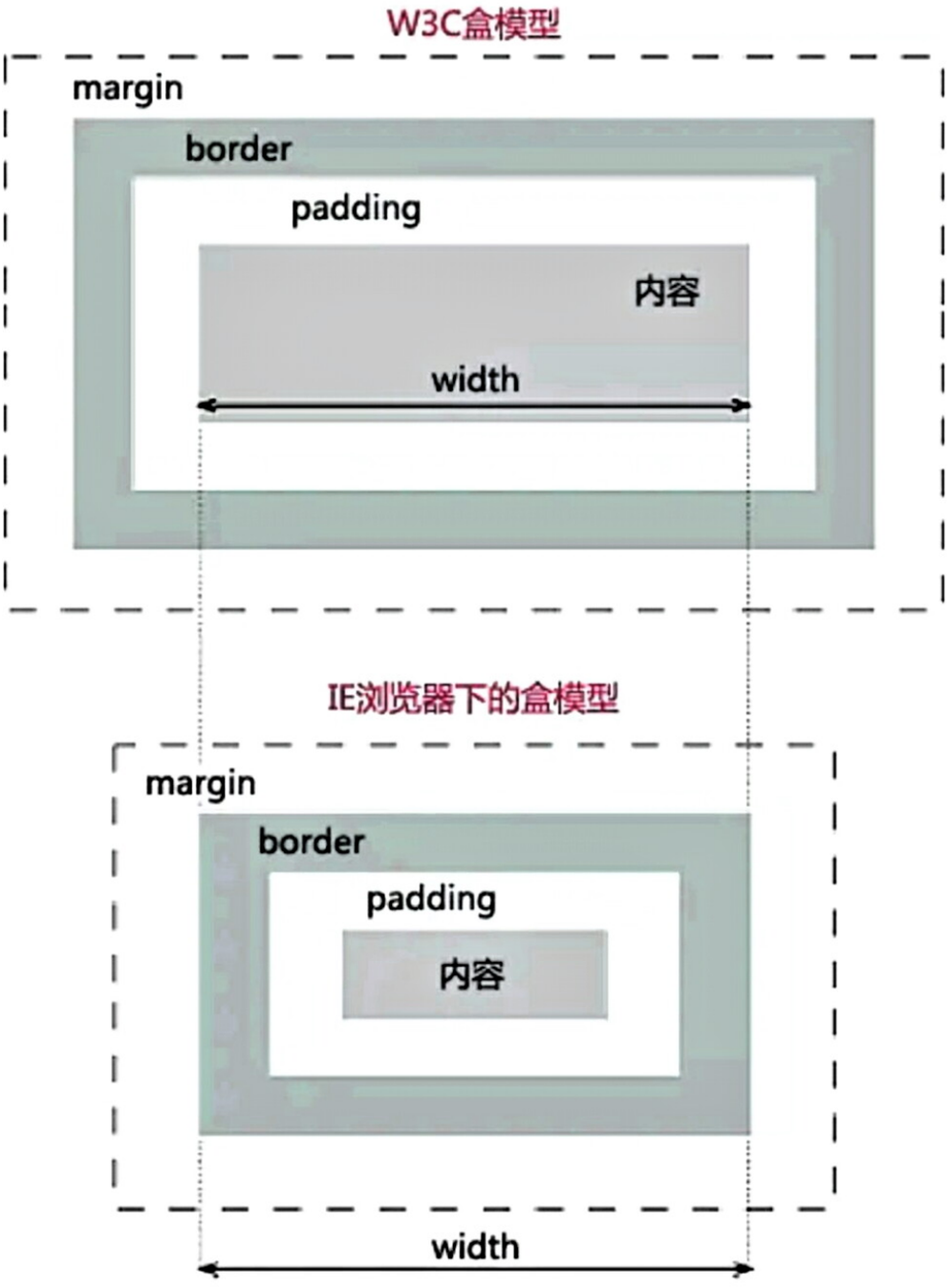
BFC可以解决的问题

- 垂直外边距重叠问题
- 去除浮动
- 自适用两列布局（ `float + overflow` ）

了解盒模型么

> 包括内容区域、内边距区域、边框区域和外边距区域。

夸克扫描王 极速扫描，就是高效



> box-sizing: content-box （W3C盒子模型）：元素的宽高大小表现为内容的大小。 box-sizing: border-box （IE盒子模型）：元素的宽高表现为内容 + 内边距 + 边框的大小。背景会延伸到边框的外沿。

IE5.x和IE6在怪异模式中使用非标准的盒子模型，这些浏览器的width属性不是内容的宽度，而是内容、内边距和边框的宽度的总和。

如何实现左侧宽度固定，右侧宽度自适应的布局

> 小提示：这个问题面试官会要求说出几种解决方法。

DOM结构

夸克扫描王 极速扫描，就是高效

```
<div class="box">
  <div class="box-left"></div>
  <div class="box-right"></div>
</div>
```

利用float + margin实现

```
.box {
 height: 200px;
}

.box > div {
  height: 100%;
}

.box-left {
  width: 200px;
  float: left;
  background-color: blue;
}

.box-right {
  margin-left: 200px;
  background-color: red;
}
```

利用calc计算宽度

```
.box {
 height: 200px;
}

.box > div {
  height: 100%;
}

.box-left {
  width: 200px;
  float: left;
  background-color: blue;
}

.box-right {
```


夸克扫描王 极速扫描，就是高效

```
  width: calc(100% - 200px);
  float: right;
  background-color: red;
}
```

利用float + overflow实现

```
.box {
 height: 200px;
}

.box > div {
  height: 100%;
}

.box-left {
  width: 200px;
  float: left;
  background-color: blue;
}

.box-right {
  overflow: hidden;
  background-color: red;
}
```

利用flex实现

> 这里不是最佳答案，应该是使用flex-basis实现更合理

```
.box {
  height: 200px;
  display: flex;
}

.box > div {
  height: 100%;
}

.box-left {
  width: 200px;
  background-color: blue;
}

.box-right {
```


夸克扫描王 极速扫描，就是高效

```
    flex: 1; // 设置flex-grow属性为1，默认为0
    background-color: red;
  }
```

了解跨域吗，一般什么情况下会导致跨域

> 小提示： 如果平常自身有使用场景可结合使用场景进行讲解，比如我在这里使用过的场景是 `CORS` 和 `Nginx` 反向代理

跨域行为

- 同源策略限制、安全性考虑
- 协议、IP和端口不一致都是跨域行为

JSONP

> 小提示：如果你提到JSONP，面试官肯定会问你整个详细的实现过程，所以一定要搞懂JSONP的实现原理，如果不是很理解可以自己起一个Express服务实践一下

- Web前端事先定义一个用于获取跨域响应数据的回调函数，并通过没有同源策略限制的script标签发起一个请求（将回调函数的名称放到这个请求的query参数里），然后服务端返回这个回调函数的执行，并将需要响应的数据放到回调函数的参数里，前端的script标签请求到这个执行的回调函数后会立马执行，于是就拿到了执行的响应数据。
- 缺点： JSONP只能发起GET请求

如何实现一个JSONP

- https://segmentfault.com/a/1190000015597029
- https://zhangguixu.github.io/2016/12/02/JSONP/
- https://www.cnblogs.com/iovec/p/5312464.html

JSONP安全性问题

1. CSRF攻击

- 前端构造一个恶意页面，请求JSONP接口，收集服务端的敏感信息。如果JSONP接口还涉及一些敏感操作或信息（比如登录、删除等操作），那就更不安全了。
- 解决方法：验证JSONP的调用来源（Referer），服务端判断Referer是否是白名单，或者部署随机Token来防御。

2. XSS漏洞

夸克扫描王 极速扫描，就是高效

> 不严谨的 content-type导致的 XSS 漏洞，想象一下 JSONP 就是你请求 http://youdomain.com?callback=douniwan, 然后返回 `douniwan({ data })，那假如请求 http://youdomain.com?callback=<script>alert(1)</script>` 不就返回 `<script>alert(1)</script>`（`{ data }`）了吗，如果没有严格定义好 `Content-Type（ Content-Type: application/json ）`，再加上没有过滤 callback 参数，直接当 html 解析了，就是一个赤裸裸的 XSS 了。

- 解决方法：严格定义 Content-Type: application/json，然后严格过滤 callback 后的参数并且限制长度（进行字符转义，例如<换成&lt，>换成&gt）等，这样返回的脚本内容会变成文本格式，脚本将不会执行。

3. 服务器被黑，返回一串恶意执行的代码

可以将执行的代码转发到服务端进行校验JSONP内容校验，再返回校验结果

CORS（跨域资源共享）

> 小提示：如果你回答跨域解决方案CORS，那么面试官一定会问你实现CORS的响应头信息 `Access-Control-Allow-Origin`

1. 什么是CORS

- 浏览器端会自动向请求头添加origin字段，表明当前请求来源。
- 服务器端需要设置响应头的Access-Control-Allow-Methods，Access-Control-Allow-Headers，Access-Control-Allow-Origin等字段，指定允许的方法，头部，源等信息。
- 请求分为简单请求和非简单请求，非简单请求会先进行一次OPTION方法进行预检，看是否允许当前跨域请求。

简单请求

> 请求方法是以下三种方法之一：

- `HEAD`
- `GET`
- `POST`

> HTTP的请求头信息不超出以下几种字段：

- `Accept`
- `Accept-Language`
- `Content-Language`
- `Last-Event-ID`
- `Content-Type` ：只限于三个值 `application/x-www-form-urlencoded` 、 `multipart/form-data` 、 `text/plain`

后端的响应头信息：

夸克扫描王 极速扫描，就是高效

- `Access-Control-Allow-Origin` ：该字段是必须的。它的值要么是请求时 `Origin` 字段的值，要么是一个*，表示接受任意域名的请求。
- `Access-Control-Allow-Credentials` ：该字段可选。它的值是一个布尔值，表示是否允许发送 `Cookie` 。
- `Access-Control-Expose-Headers` ：该字段可选。CORS请求时，XMLHttpRequest对象的getResponseHeader()方法只能拿到6个基本字段： `Cache-Control` 、 `Content-Language` 、 `Content-Type` 、 `Expires` 、 `Last-Modified` 、 `Pragma` 。如果想拿到其他字段，就必须在 `Access-Control-Expose-Headers` 里面指定。

非简单请求

> 非简单请求是那种对服务器有特殊要求的请求，比如请求方法是PUT或DELETE，或者Content-Type字段的类型是application/json。非简单请求的CORS请求，会在正式通信之前，增加一次HTTP查询请求，称为"预检"请求（preflight）。

- `Access-Control-Request-Method` ：该字段是必须的，用来列出浏览器的CORS请求会用到哪些HTTP方法，上例是PUT
- `Access-Control-Request-Headers` ：该字段是一个逗号分隔的字符串，指定浏览器CORS请求会额外发送的头信息字段，上例是X-Custom-Header

> 如果浏览器否定了"预检"请求，会返回一个正常的HTTP回应，但是没有任何CORS相关的头信息字段。这时，浏览器就会认定，服务器不同意预检请求，因此触发一个错误，被 `XMLHttpRequest` 对象的 `onerror` 回调函数捕获

JSONP和CORS的对比

- `JSONP` 只支持GET请求，CORS支持所有类型的HTTP请求
- `JSONP` 的优势在于支持老式浏览器，以及可以向不支持CORS的网站请求数据

其他跨域解决方案

- `Nginx` 反向代理
- `postMessage`
- `document.domain`
  - 自 `Chrome 101` 版本开始， `document.domain` 将变为可读属性，也就是意味着上述这种跨域的方式被禁用了

HTTP2和HTTP1有什么区别

> 相对于HTTP1.0，HTTP1.1的优化：

夸克扫描王 极速扫描，就是高效

- 缓存处理：多了Entity tag，If-Unmodified-Since, If-Match, If-None-Match等缓存信息（HTTTP1.0 If-Modified-Since,Expires）
- 带宽优化及网络连接的使用
- 错误通知的管理
- Host头处理
- 长连接： HTTP1.1中默认开启Connection： keep-alive，一定程度上弥补了HTTP1.0每次请求都要创建连接的缺点。

相对于HTTP1.1，HTTP2的优化：

- HTTP2支持二进制传送（实现方便且健壮），HTTP1.x是字符串传送
- HTTP2支持多路复用
- HTTP2采用HPACK压缩算法压缩头部，减小了传输的体积
- HTTP2支持服务端推送

你能说说缓存么

> 小提示：如果平常有遇到过缓存的坑或者很好的利用缓存，可以讲解一下自己的使用场景。如果没有使用注意过缓存问题你也可以尝试讲解一下和我们息息相关的 `Webpack` 构建（每一次构建静态资源名称的 `hash` 值都会变化），它其实就跟缓存相关

缓存分为强缓存和协商缓存。强缓存不过服务器，协商缓存需要过服务器，协商缓存返回的状态码是304。两类缓存机制可以同时存在，强缓存的优先级高于协商缓存。当执行强缓存时，如若缓存命中，则直接使用缓存数据库中的数据，不再进行缓存协商

1. 强缓存

> Expires(HTTP1.0)：Exprires的值为服务端返回的数据到期时间。当再次请求时的请求时间小于返回的此时间，则直接使用缓存数据。但由于服务端时间和客户端时间可能有误差，这也将导致缓存命中的误差。另一方面，Expires是HTTP1.0的产物，故现在大多数使用Cache-Control替代

缺点：使用的是绝对时间，如果服务端和客户端的时间产生偏差，那么会导致命中缓存产生偏差

Pragma(HTTP1.0)：HTTP1.0时的遗留字段，当值为"no-cache"时强制验证缓存，Pragma禁用缓存，如果又给Expires定义一个还未到期的时间，那么Pragma字段的优先级会更高。服务端响应添加'Pragma': 'no-cache'，浏览器表现行为和刷新(F5)类似。

Cache-Control(HTTP1.1)：有很多属性，不同的属性代表的意义也不同

- `private` ： 客户端可以缓存
- `public` ： 客户端和代理服务器都可以缓存
- `max-age=t` ： 缓存内容将在t秒后失效
- `no-cache` ： 需要使用协商缓存来验证缓存数据
- `no-store` ： 所有内容都不会缓存

夸克扫描王 极速扫描，就是高效

> 请注意 `no-cache` 指令很多人误以为是不缓存，这是不准确的， `no-cache` 的意思是可以缓存，但每次用应该去想服务器验证缓存是否可用。 `no-store` 才是不缓存内容。当在首部字段 `Cache-Control` 有指定 `max-age` 指令时，比起首部字段 `Expires` ，会优先处理 `max-age` 指令。命中强缓存的表现形式： `Firefox` 浏览器表现为一个灰色的 `200` 状态码。Chrome浏览器状态码表现为200 (from disk cache)或是200 OK (from memory cache)

2. 协商缓存

> 协商缓存需要进行对比判断是否可以使用缓存。浏览器第一次请求数据时，服务器会将缓存标识与数据一起响应给客户端，客户端将它们备份至缓存中。再次请求时，客户端会将缓存中的标识发送给服务器，服务器根据此标识判断。若未失效，返回304状态码，浏览器拿到此状态码就可以直接使用缓存数据了

- Last-Modified：服务器在响应请求时，会告诉浏览器资源的最后修改时间。
- if-Modified-Since：浏览器再次请求服务器的时候，请求头会包含此字段，后面跟着在缓存中获得的最后修改时间。服务端收到此请求头发现有if-Modified-Since，则与被请求资源的最后修改时间进行对比，如果一致则返回304和响应报文头，浏览器只需要从缓存中获取信息即可
  - 如果真的被修改：那么开始传输响应一个整体，服务器返回：200 OK
  - 如果没有被修改：那么只需传输响应header，服务器返回：304 Not Modified
- if-Unmodified-Since: 从某个时间点算起, 是否文件没有被修改，使用的是相对时间，不需要关心客户端和服务端的时间偏差
  - 如果没有被修改：则开始`继续'传送文件，服务器返回: 200 OK
  - 如果文件被修改：则不传输，服务器返回: 412 Precondition failed (预处理错误)

> 这两个的区别是一个是修改了才下载一个是没修改才下载。如果在服务器上，一个资源被修改了，但其实际内容根本没发生改变，会因为 `Last-Modified` 时间匹配不上而返回了整个实体给客户端（即使客户端缓存里有个一模一样的资源）。为了解决这个问题， `HTTP1.1` 推出了 `Etag`

- `Etag` ：服务器响应请求时，通过此字段告诉浏览器当前资源在服务器生成的唯一标识（生成规则由服务器决定）
- `If-Match` ：条件请求，携带上一次请求中资源的ETag，服务器根据这个字段判断文件是否有新的修改
- `If-None-Match` ： 再次请求服务器时，浏览器的请求报文头部会包含此字段，后面的值为在缓存中获取的标识。服务器接收到次报文后发现If-None-Match则与被请求资源的唯一标识进行对比
  - 不同，说明资源被改动过，则响应整个资源内容，返回状态码200。
  - 相同，说明资源无心修改，则响应 `header` ，浏览器直接从缓存中获取数据信息。返回状态码304.

> 但是实际应用中由于Etag的计算是使用算法来得出的，而算法会占用服务端计算的资源，所有服务端的资源都是宝贵的，所以就很少使用Etag了。

- 浏览器地址栏中写入URL，回车浏览器发现缓存中有这个文件了，不用继续请求了，直接去缓存拿（最快）
- F5就是告诉浏览器，别偷懒，好歹去服务器看看这个文件是否有过期了。于是浏览器就胆胆襟襟的发送一个请求带上 `If-Modify-since`

夸克扫描王 极速扫描，就是高效

- Ctrl+F5告诉浏览器，你先把你缓存中的这个文件给我删了，然后再去服务器请求个完整的资源文件下来。于是客户端就完成了强行更新的操作

3. 缓存场景

> 对于大部分的场景都可以使用强缓存配合协商缓存解决，但是在一些特殊的地方可能需要选择特殊的缓存策略

- 对于某些不需要缓存的资源，可以使用 `Cache-control: no-store` ，表示该资源不需要缓存
- 对于频繁变动的资源，可以使用 `Cache-Control: no-cache` 并配合 ETag 使用，表示该资源已被缓存，但是每次都会发送请求询问资源是否更新
- 对于代码文件来说，通常使用 `Cache-Control: max-age=31536000` 并配合策略缓存使用，然后对文件进行指纹处理，一旦文件名变动就会立刻下载新的文件

能说说首屏加载优化有哪些方案么

> 小提示：如果做过类似优化的同学，可能就比较好回答，没有做过类似优化的同学可以重点讲解一下懒加载（当然我这里被面试官追问过懒加载的Webpack配置问题）。同时不知道使用Vue技术栈的同学们有没有仔细观察过Vue CLI 3构建的html文件中的link标签的rel属性。

- Vue-Router路由懒加载（利用Webpack的代码切割）
- 使用CDN加速，将通用的库从vendor进行抽离
- Nginx的gzip压缩
- Vue异步组件
- 服务端渲染SSR
- 如果使用了一些UI库，采用按需加载
- Webpack开启gzip压缩
- 如果首屏为登录页，可以做成多入口
- Service Worker缓存文件处理
- 使用link标签的rel属性设置 prefetch（这段资源将会在未来某个导航或者功能要用到，但是本资源的下载顺序权重比较低，prefetch通常用于加速下一次导航）、preload（preload将会把资源得下载顺序权重提高，使得关键数据提前下载好，优化页面打开速度）

如何在Node端配置路径别名（类似于Webpack中的alias配置）

- 全局变量
- 环境变量
- 自己HACK一个@符号，指向特定的路径
- HACK require方法

夸克扫描王 极速扫描，就是高效

谈谈你对作用域链的理解

> 小提示：同类型的问题还可以是原型链、继承、闭包等，这种概念性的问题你肯定不是一句两句能说清楚的，建议在理解之后自己尝试总结一下，如何把重要的知识点用简短的话语说明白

了解作用域链之前我们要知道一下几个概念：

- 函数的生命周期
- 变量和函数的声明
- `Activetion Object（AO）`、`Variable Object（VO）`

函数的生命周期：

- 创建：`JS` 解析引擎进行预解析，会将函数声明提前，同时将该函数放到全局作用域中或当前函数的上一级函数的局部作用域中。
- 执行：`JS` 引擎会将当前函数的局部变量和内部函数进行声明提前，然后再执行业务代码，当函数执行完退出时，释放该函数的执行上下文，并注销该函数的局部变量

> 变量和函数的声明：如果变量名和函数名声明时相同，函数优先声明

Activetion Object（AO）、Variable Object（VO）：

- `AO：Activetion Object` （活动对象）
- `VO：Variable Object` （变量对象）

> `VO` 对应的是函数创建阶段，JS解析引擎进行预解析时，所有的变量和函数的声明，统称为 `Variable Object` 。该变量与执行上下文相关，知道自己的数据存储在哪里，并且知道如何访问。 `VO` 是一个与执行上下文相关的特殊对象，它存储着在上下文中声明的以下内容

- 变量（ `var` ，变量声明);
- 函数声明（ `FunctionDeclaration` ，缩写为FD);
- 函数的形参

> AO对应的是函数执行阶段，当函数被调用执行时，会建立一个执行上下文，该执行上下文包含了函数所需的所有变量，该变量共同组成了一个新的对象就是Activetion Object。该对象包含了

- 函数的所有局部变量
- 函数的所有命名参数
- 函数的参数集合
- 函数的 `this` 指向

作用域链：

夸克扫描王 极速扫描，就是高效

- 当代码在一个环境中创建时，会创建变量对象的一个作用域链（scope chain）来保证对执行环境有权访问的变量和函数。作用域第一个对象始终是当前执行代码所在环境的变量对象（VO）。如果是函数执行阶段，那么将其activation object（AO）作为作用域链第一个对象，第二个对象是上级函数的执行上下文AO，下一个对象依次类推。
- 当查找变量的时候，会先从当前上下文的变量对象中查找，如果没有找到，就会从父级(词法层面上的父级)执行上下文的变量对象中查找，一直找到全局上下文的变量对象，也就是全局对象。这样由多个执行上下文的变量对象构成的链表就叫做作用域链。

Vue响应式原理

> 小提示：如果面试者使用的是Vue技术栈，那么响应式原理是一个必问的问题，同时面试官经常也会问Vue 3.0在响应式原理上的优化方案。

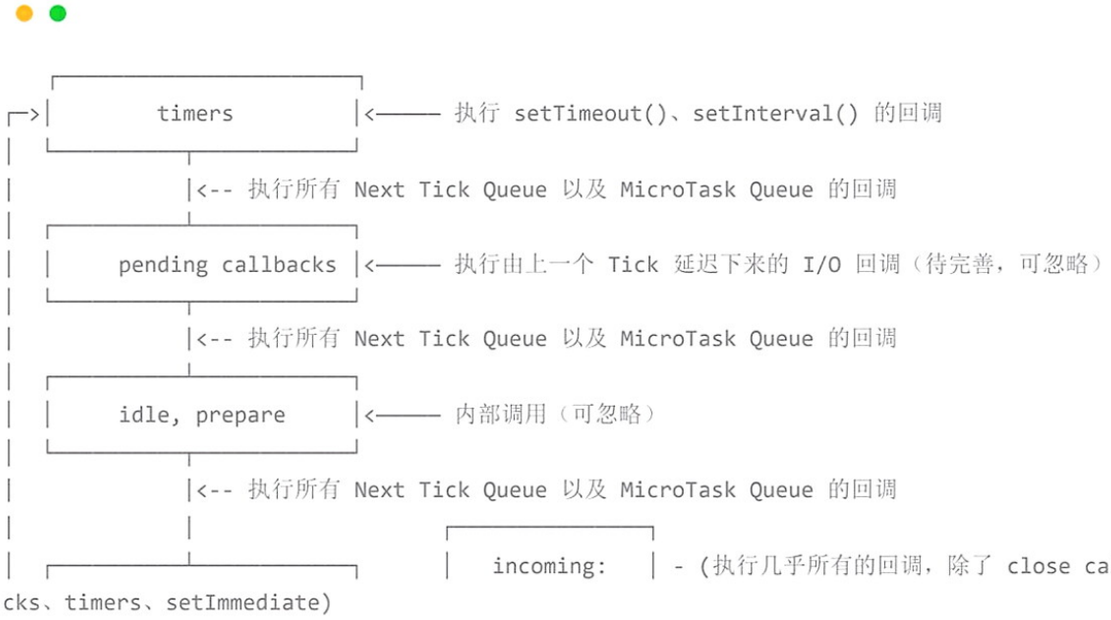
了解Event Loop么

> 小提示：这个题目问到的概率还是蛮大的，这里面试官询问了我浏览器端和Node端的Event Loop有什么不同点

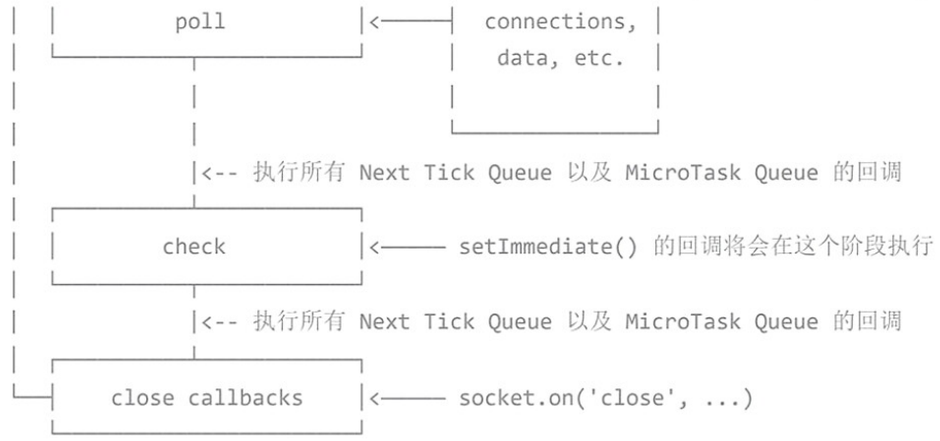
> 事件触发线程管理的任务队列是如何产生的呢？事实上这些任务就是从JS引擎线程本身产生的，主线程在运行时会产生执行栈，栈中的代码调用某些异步API时会在任务队列中添加事件，栈中的代码执行完毕后，就会读取任务队列中的事件，去执行事件对应的回调函数，如此循环往复，形成事件循环机制。JS中有两种任务类型：微任务（microtask）和宏任务（macrotask），在ES6中，microtask称为 jobs，macrotask称为 task

- 宏任务： script （主代码块）、setTimeout 、setInterval 、setImmediate 、I/O 、UI rendering
- 微任务：process.nextTick（Nodejs） 、Promise 、Object.observe 、MutationObserver

Node.js中Event Loop和浏览器中Event Loop有什么区别



夸克扫描王 极速扫描，就是高效



> Node.js中宏任务分成了几种类型，并且放在了不同的task queue里。不同的task queue在执行顺序上也有区别，微任务放在了每个task queue的末尾：

- `setTimeout/setInterval` 属于 `timers` 类型；
- `setImmediate` 属于 `check` 类型；
- `socket` 的 close 事件属于 `close callbacks` 类型；
- 其他 `MacroTask` 都属于 poll 类型。
- `process.nextTick` 本质上属于 `MicroTask` ，但是它先于所有其他 `MicroTask` 执行；
- 所有 `MicroTask` 的执行时机在不同类型的 MacroTask 切换后。
- `idle/prepare` 仅供内部调用，我们可以忽略。
- `pending callbacks` 不太常见，我们也可以忽略。

如何避免回流和重绘

- 浏览器使用流式布局模型 (Flow Based Layout)
- 浏览器会把HTML解析成DOM，把CSS解析成CSSOM，DOM和CSSOM合并就产生了Render Tree
- 有了RenderTree就能知道所有节点的样式，计算节点在页面上的大小和位置，把节点绘制到页面上
- 由于浏览器使用流式布局，对Render Tree的计算通常只需要遍历一次就可以完成，但table及其内部元素除外，通常需要多次计算且要花费3倍于同等元素的时间，这也是为什么要避免使用table布局的原因之一

浏览器渲染过程如下：

- 解析HTML，生成DOM树
- 解析CSS，生成CSSOM树
- 将DOM树和CSSOM树结合，生成渲染树(Render Tree)
- Layout(回流)：根据生成的渲染树，进行回流(Layout)，得到节点的几何信息（位置，大小）
- Painting(重绘)：根据渲染树以及回流得到的几何信息，得到节点的绝对像素

夸克扫描王 极速扫描，就是高效

- Display：将像素发送给GPU，展示在页面上。（这一步其实还有很多内容，比如会在GPU将多个合成层合并为同一个层，并展示在页面中。而css3硬件加速的原理则是新建合成层，这里我们不展开，之后有机会会写一篇博客）

何时触发回流和重绘

> 何时发生回流：

- 添加或删除可见的DOM元素
- 元素的位置发生变化
- 元素的尺寸发生变化（包括外边距、内边框、边框大小、高度和宽度等）
- 内容发生变化，比如文本变化或图片被另一个不同尺寸的图片所替代。
- 页面一开始渲染的时候（这肯定避免不了）
- 浏览器的窗口尺寸变化（因为回流是根据视口的大小来计算元素的位置和大小的）

何时发生重绘（回流一定会触发重绘）：

> 当页面中元素样式的改变并不影响它在文档流中的位置时（例如：color、background-color、visibility等），浏览器会将新样式赋予给元素并重新绘制它，这个过程称为重绘。

> 有时即使仅仅回流一个单一的元素，它的父元素以及任何跟随它的元素也会产生回流。现代浏览器会对频繁的回流或重绘操作进行优化，浏览器会维护一个队列，把所有引起回流和重绘的操作放入队列中，如果队列中的任务数量或者时间间隔达到一个阈值的，浏览器就会将队列清空，进行一次批处理，这样可以把多次回流和重绘变成一次。你访问以下属性或方法时，浏览器会立刻清空队列：

- clientWidth、clientHeight、clientTop、clientLeft
- offsetWidth、offsetHeight、offsetTop、offsetLeft
- scrollWidth、scrollHeight、scrollTop、scrollLeft
- width、height
- getComputedStyle()
- getBoundingClientRect()

> 以上属性和方法都需要返回最新的布局信息，因此浏览器不得不清空队列，触发回流重绘来返回正确的值。因此，我们在修改样式的时候，**最好避免使用上面列出的属性，他们都会刷新渲染队列** 如果要使用它们，最好将值缓存起来。

如何避免触发回流和重绘

CSS：

- 避免使用 `table` 布局。
- 尽可能在 `DOM` 树的最末端改变 `class` 。
- 避免设置多层内联样式。
- 将动画效果应用到 `p` osition `属性为` absolute `或` fixed`的元素上

夸克扫描王 极速扫描，就是高效

- 避免使用 `CSS` 表达式（例如： `calc()` ）
- `CSS3` 硬件加速（ `GPU` 加速）

JavaScript：

- 避免频繁操作样式，最好一次性重写style属性，或者将样式列表定义为class并一次性更改class属性
- 避免频繁操作DOM，创建一个 `documentFragment` ，在它上面应用所有DOM操作，最后再把它添加到文档中
- 也可以先为元素设置 `display: none` ，操作结束后再把它显示出来。因为在display属性为none的元素上进行的DOM操作不会引发回流和重绘
- 避免频繁读取会引发回流/重绘的属性，如果确实需要多次使用，就用一个变量缓存起来
- 对具有复杂动画的元素使用绝对定位，使它脱离文档流，否则会引起父元素及后续元素频繁回流

有赞二面

二面

> 小提示：进入现场面试需要注意好好准备自己的简历，面试官一般会根据项目进行问答。

笔试题环节

一开始面试官就发了两张笔试题试卷，总共四道题目，大致考了以下知识点：

- 作用域
- 原型链（例如实例属性和原型属性一样，删除实例属性后可以继续访问原型属性问题）
- 宏任务和微任务的打印顺序
- `Array.prototype.map` 的第二个参数

项目问答环节

答完试卷面试官就开始问简历上的一些项目，我记得其中几个问题如下（事实上他问的一些问题和简历不是很相关）：

- 你们产品的服务器部署在哪里
- 你是如何实现一个Tooltip组件的，能写一下怎么使用这个组件么（这算什么问题...）
- 我认识你们海康的一些开发，我知道你们的产品按套数卖的...

我当场就感受到了面试官问的问题很敷衍，可能他觉得我的简历不够好，又或者觉得我能力不行，接下来面试官又让我做了一道算法题...

算法题环节

夸克扫描王 极速扫描，就是高效

> 1块、4块、5块，求总数n块的最小硬币数

当时没做出来，非科班出身可能做这些确实有些困难，也没有系统的学习，面试官看我很困难的样子，就换了一道题。

> 1、1、2、3、5、8...计算第n个数的值（斐波那契数列）

这道题还是做出来了，毕竟比较简单，然后面试官说今天先到这里，面试结果会在一星期内通知，然后回来的那天晚上就收到了面试没过的通知。

有赞（有赞美业）电话面

浏览器事件循环的方式

宏任务和微任务如何划分

如何判断 js 变量类型

说一下vdom 和 diff

diff 算法的时间复杂度

hooks 相对于之前的开发方式有什么特点

react 开发中有哪些性能优化的方式

setState 是异步吗

如何实现批量更新的策略

夸克扫描王
极速扫描，就是高效

HTTP2 相对于 HTTP1 的优化

滴滴面经1

你知道哪些安全问题，如何避免

> 小提示：这里我简单讲解了一下Vue中的v-html防范XSS攻击。

1. XSS（跨站脚本攻击）

> XSS，即 Cross Site Script，中译是跨站脚本攻击；其原本缩写是 CSS，但为了和层叠样式表(Cascading Style Sheet)有所区分，因而在安全领域叫做 XSS。

- XSS 攻击是指攻击者在网站上注入恶意的客户端代码，通过恶意脚本对客户端网页进行篡改，从而在用户浏览网页时，对用户浏览器进行控制或者获取用户隐私数据的一种攻击方式。
- 攻击者对客户端网页注入的恶意脚本一般包括 JavaScript，有时也会包含 HTML 和 Flash。有很多种方式进行 XSS 攻击，但它们的共同点为：将一些隐私数据像 cookie、session 发送给攻击者，将受害者重定向到一个由攻击者控制的网站，在受害者的机器上进行一些恶意操作。
- XSS攻击可以分为3类：反射型（非持久型）、存储型（持久型）、基于DOM。

2. 反射型

> 反射型 XSS 只是简单地把用户输入的数据 “反射” 给浏览器，这种攻击方式往往需要攻击者诱使用户点击一个恶意链接（攻击者可以将恶意链接直接发送给受信任用户，发送的方式有很多种，比如 email, 网站的私信、评论等，攻击者可以购买存在漏洞网站的广告，将恶意链接插入在广告的链接中），或者提交一个表单，或者进入一个恶意网站时，注入脚本进入被攻击者的网站。最简单的示例是访问一个链接，服务端返回一个可执行脚本：

```js
const http = require('http');
function handleReequest(req, res) {
    res.setHeader('Access-Control-Allow-Origin', '*');
    res.writeHead(200, {'Content-Type': 'text/html; charset=UTF-8'});
    res.write('<script>alert("反射型 XSS 攻击")</script>');
    res.end();
}

const server = new http.Server();
server.listen(8001, '127.0.0.1');
server.on('request', handleReequest);
```


夸克扫描王 极速扫描，就是高效

3. 存储型

> 存储型 XSS 会把用户输入的数据 "存储" 在服务器端，当浏览器请求数据时，脚本从服务器上传回并执行。这种 XSS 攻击具有很强的稳定性。比较常见的一个场景是攻击者在社区或论坛上写下一篇包含恶意 JavaScript 代码的文章或评论，文章或评论发表后，所有访问该文章或评论的用户，都会在他们的浏览器中执行这段恶意的 JavaScript 代码

```
// 例如在评论中输入以下留言
// 如果请求这段留言的时候服务端不做转义处理，请求之后页面会执行这段恶意代码
<script>alert('xss 攻击')</script>
```

4. 基于DOM

> 基于 DOM 的 XSS 攻击是指通过恶意脚本修改页面的 DOM 结构，是纯粹发生在客户端的攻击：

```
<h2>XSS: </h2>
<input type="text" id="input">
<button id="btn">Submit</button>
<div id="div"></div>
<script>
    const input = document.getElementById('input');
    const btn = document.getElementById('btn');
    const div = document.getElementById('div');

    let val;

    input.addEventListener('change', (e) => {
        val = e.target.value;
    }, false);

    btn.addEventListener('click', () => {
        div.innerHTML = `<a href=${val}>testLink</a>`
    }, false);
</script>
```

> 点击 Submit 按钮后，会在当前页面插入一个链接，其地址为用户的输入内容。如果用户在输入时构造了如下内容：

```
'' onclick=alert(/xss/)
```

用户提交之后，页面代码就变成了：

夸克扫描王 极速扫描，就是高效

```
<a href onlick="alert(/xss/)">testLink</a>
```

此时，用户点击生成的链接，就会执行对应的脚本。

5. XSS攻击防范

- HttpOnly 防止劫取 Cookie：HttpOnly 最早由微软提出，至今已经成为一个标准。浏览器将禁止页面的 Javascript 访问带有 HttpOnly 属性的Cookie。上文有说到，攻击者可以通过注入恶意脚本获取用户的 Cookie 信息。通常 Cookie 中都包含了用户的登录凭证信息，攻击者在获取到 Cookie 之后，则可以发起 Cookie 劫持攻击。所以，严格来说，HttpOnly 并非阻止 XSS 攻击，而是能阻止 XSS 攻击后的 Cookie 劫持攻击。
- 输入检查：不要相信用户的任何输入。 对于用户的任何输入要进行检查、过滤和转义。建立可信任的字符和 HTML 标签白名单，对于不在白名单之列的字符或者标签进行过滤或编码。在 XSS 防御中，输入检查一般是检查用户输入的数据中是否包含 <，> 等特殊字符，如果存在，则对特殊字符进行过滤或编码，这种方式也称为 XSS Filter。而在一些前端框架中，都会有一份 decodingMap， 用于对用户输入所包含的特殊字符或标签进行编码或过滤，如 <，>，script，防止 XSS 攻击

```js
// vuejs 中的 decodingMap
// 在 vuejs 中，如果输入带 script 标签的内容，会直接过滤掉
const decodingMap = {
  '&lt;': '<',
  '&gt;': '>',
  '&quot;': '"',
  '&amp;': '&',
  '&#10;': '\n'
}
```

> 输出检查：用户的输入会存在问题，服务端的输出也会存在问题。一般来说，除富文本的输出外，在变量输出到 HTML 页面时，可以使用编码或转义的方式来防御 XSS 攻击。例如利用 sanitize-html 对输出内容进行有规则的过滤之后再输出到页面中。

CSRF/XSRF（跨站请求伪造）

> CSRF，即 Cross Site Request Forgery，中译是跨站请求伪造，是一种劫持受信任用户向服务器发送非预期请求的攻击方式。通常情况下，CSRF 攻击是攻击者借助受害者的 Cookie 骗取服务器的信任，可以在受害者毫不知情的情况下以受害者名义伪造请求发送给受攻击服务器，从而在并未授权的情况下执行在权限保护之下的操作。

1. Cookie

夸克扫描王 极速扫描，就是高效

> Cookie 是服务器发送到用户浏览器并保存在本地的一小块数据，它会在浏览器下次向同一服务器再发起请求时被携带并发送到服务器上。Cookie 主要用于以下三个方面：

- 会话状态管理（如用户登录状态、购物车、游戏分数或其它需要记录的信息）
- 个性化设置（如用户自定义设置、主题等）
- 浏览器行为跟踪（如跟踪分析用户行为等）

而浏览器所持有的 Cookie 分为两种：

- Session Cookie(会话期 Cookie)：会话期 Cookie 是最简单的Cookie，它不需要指定过期时间（Expires）或者有效期（Max-Age），它仅在会话期内有效，浏览器关闭之后它会被自动删除。
- Permanent Cookie(持久性 Cookie)：与会话期 Cookie 不同的是，持久性 Cookie 可以指定一个特定的过期时间（Expires）或有效期（Max-Age）

```
res.setHeader('Set-Cookie', ['mycookie=222', 'test=3333; expires=Sat, 21 Jul 2018 00:00:0
0 GMT;']);
```

> 上述代码创建了两个 Cookie：mycookie 和 test，前者属于会话期 Cookie，后者则属于持久性 Cookie

2. CSRF攻击

- 使登录用户访问攻击者的网站，发起一个请求，由于 Cookie 中包含了用户的认证信息，当用户访问攻击者准备的攻击环境时，攻击者就可以对服务器发起 CSRF 攻击。
- 在这个攻击过程中，攻击者借助受害者的 Cookie 骗取服务器的信任，但并不能拿到 Cookie，也看不到 Cookie 的内容。而对于服务器返回的结果，由于浏览器同源策略的限制，攻击者也无法进行解析。（攻击者的网站虽然是跨域的，但是他构造的链接是源网站的，跟源网站是同源的，所以能够携带cookie发起访问）- 但是攻击者无法从返回的结果中得到任何东西，他所能做的就是给服务器发送请求，以执行请求中所描述的命令，在服务器端直接改变数据的值，而非窃取服务器中的数据。例如删除数据、修改数据，新增数据等，无法获取数据。

3. CSRF攻击防范

- 验证码：验证码被认为是对抗 CSRF 攻击最简洁而有效的防御方法。从上述示例中可以看出，CSRF 攻击往往是在用户不知情的情况下构造了网络请求。而验证码会强制用户必须与应用进行交互，才能完成最终请求。因为通常情况下，验证码能够很好地遏制 CSRF 攻击。但验证码并不是万能的，因为出于用户考虑，不能给网站所有的操作都加上验证码。因此，验证码只能作为防御 CSRF 的一种辅助手段，而不能作为最主要的解决方案。
- Referer Check：根据 HTTP 协议，在 HTTP 头中有一个字段叫 Referer，它记录了该 HTTP 请求的来源地址。通过 Referer Check，可以检查请求是否来自合法的"源"。
- 添加token验证：要抵御 CSRF，关键在于在请求中放入攻击者所不能伪造的信息，并且该信息不存在于 Cookie 之中。可以在 HTTP 请求中以参数的形式加入一个随机产生的 token，并在服务器端建立一个拦

夸克扫描王 极速扫描，就是高效

截器来验证这个 token，如果请求中没有 token 或者 token 内容不正确，则认为可能是 CSRF 攻击而拒绝该请求。

介绍一下Graphql

> 小提示：这道题是给自己挖了一个坑，抱着学习的心态尝试使用Graphql技术，却没有好好理解是在什么场景下为了解决什么问题才应该使用，也没有好好准备如何描述新技术，往往这种不熟悉的技术自己在简历中应该留存一些心眼，尽量不要提，否则答不上来会很尴尬，让面试官怀疑你的项目成分。

什么是Graphql

> GraphQL是一种API查询语言。API接口的返回值可以从静态变为动态，即调用者来声明接口返回什么数据，可以进一步解耦前后端。在Graphal中，预先定义Schema和声明Type来达到动态获取接口数据的目的：

- 对于数据模型的抽象是通过Type来描述的
- 对于接口获取数据的逻辑是通过Schema来描述的

为什么要使用Graphql：

- 接口数量众多维护成本高
- 接口扩展成本高
- 接口响应的数据格式无法预知
- 减少无用数据的请求， 按需获取
- 强类型约束（API的数据格式让前端来定义，而不是后端定义）

Type（数据模型的抽象）

> Type简单可以分为两种，一种叫做Scalar Type(标量类型)，另一种叫做Object Type(对象类型)：

- Scalar Type（标量类型）：内建的标量包含，String、Int、Float、Boolean、Enum
- Object Type（对象类型）：感觉类似于TypeScript的接口类型
- Type Modifier（类型修饰符）：用于表明是否必填等

Schema（模式）

- 定义了字段的类型、数据的结构，描述了接口数据请求的规则

Query（查询、操作类型）

> 查询类型： query（查询）、mutation（更改）和subscription（订阅）

- query（查询）：当获取数据时，应当选取Query类型
- mutation（更改）：当尝试修改数据时，应当使用mutation类型

夸克扫描王 极速扫描，就是高效

- subscription（订阅）：当希望数据更改时，可以进行消息推送，使用subscription类型

Resolver（解析函数）

> 提供相关Query所返回数据的逻辑。Query和与之对应的Resolver是同名的，这样在GraphQL才能把它们对应起来。解析的过程可能是递归的，只要遇到非标量类型，会尝试继续解析，如果遇到标量类型，那么解析完成，这个过程叫做解析链。

说说Vue中$nextTick的实现原理

> 小提示：如果面试者使用的是Vue技术栈，那么$nextTick的原理是一个高频问题，面试者借此可以追问的东西较多，例如浏览器的Event Loop、微任务和宏任务、Node.js的Event Loop、异步更新DOM（响应式的数据for循环改变了1000次为什么视图只更新了一次）、$nextTick历史版本问题等等。

如何实现居中

水平居中

- 若是行内元素，给其父元素设置text-align:center即可实现行内元素水平居中
- 若是块级元素，该元素设置margin:0 auto即可（元素需要定宽）
- 若是块级元素，设置父元素为flex布局，子元素设置margin:0 auto即可（子元素不需要定宽）
- 使用flex 2012年版本布局，可以轻松的实现水平居中，子元素设置如下:

```
// flex容器
<div class="box">
 // flex项目
 <div class="box-center">
 </div>
</div>


.box {
  width: 200px;
  height: 200px;
  display: flex;
  // 使内部的flex项目水平居中
  justify-content: center;
  background-color: pink;
}

/* .box-center {
  width: 50%;
  background-color: greenyellow;
} */
```


夸克扫描王 极速扫描，就是高效

> 使用绝对定位和CSS3新增的属性transform（这个属性还和GPU硬件加速、固定定位相关）

```
.box {
  width: 200px;
  height: 200px;
  position: relative;
  background-color: pink;
}

.box-center {
  position: absolute;
  left:50%;
  // width: 50%;
  height: 100%;
  // 通过 translate() 方法，元素从其当前位置移动，根据给定的 left（x 坐标） 和 top（y 坐标） 位置参数：
  // translate(x,y) 定义 2D 转换。
  // translateX(x)  定义转换，只是用 X 轴的值。
  // translateY(y)  定义转换，只是用 Y 轴的值。
  // left: 50% 先整体向父容器的左侧偏移50%，此时是不能居中的，因为元素本身有大小
  // 接着使用transform使用百分比向左偏移本身的宽度的一半实现水平居中（这里的百分比以元素本身的宽高为基准）
  transform:translate(-50%,0);
  background-color: greenyellow;
}
```

使用绝对定位和margin-left（元素定宽）

```
.box {
  width: 200px;
  height: 200px;
  position: relative;
  background-color: pink;
}

.box-center {
  position: absolute;
  left:50%;
  height: 100%;
  // 类似于transform
  // width: 50%;
  // margin-left: -25%;
  width: 100px;
  margin-left: -50px;
  background-color: greenyellow;
}
```


夸克扫描王 极速扫描，就是高效

垂直居中

- 若元素是单行文本, 则可设置line-height等于父元素高度
- 若是块级元素，设置父元素为flex布局，子元素设置margin: auto 0即可（子元素不需要定宽）
- 若元素是行内块级元素，基本思想是使用display: inline-block, vertical-align: middle和一个伪元素让内容块处于容器中央：

```
.box {
  height: 100px;
}

.box::after, .box-center{
  display:inline-block;
  vertical-align:middle;
}
.box::after{
  content:'';
  height:100%;
}
```

居中元素高度不定

> 可用 vertical-align 属性（vertical-align只有在父层为 td 或者 th 时才会生效，对于其他块级元素，例如 div、p 等，默认情况是不支持的），为了使用vertical-align，我们需要设置父元素display:table,子元素 display:table-cell;vertical-align:middle：

```
.box {
  height: 100px;
  display: table;
}
 .box-center{
    display: table-cell;
    vertical-align:middle;
}
```

> 可用 Flex 2012版, 这是CSS布局未来的趋势。Flexbox是CSS3新增属性，设计初衷是为了解决像垂直居中这样的常见布局问题：

```
.box {
  height: 100px;
```


夸克扫描王 极速扫描，就是高效

```
  display: flex;
  align-items: center;
}
```

> 优点：内容块的宽高任意, 优雅的溢出. 可用于更复杂高级的布局技术中. 缺点：IE8/IE9不支持、需要浏览器厂商前缀、渲染上可能会有一些问题

> 可用 transform ， 设置父元素相对定位：

```
.box {
  height: 100px;
  position: relative;
  background-color: pink;
}

.box-center {
  position: absolute;
  top: 50%;
  transform: translate(0, -50%);
  background-color: greenyellow;
}
```

> 缺点：IE8不支持, 属性需要追加浏览器厂商前缀，可能干扰其他 transform 效果，某些情形下会出现文本或元素边界渲染模糊的现象。

居中元素高度固定

> 设置父元素相对定位，子元素如下css样式:

```
.box {
  position:relative;
  height: 100px;
  background-color: pink;
}

.box-center{
  position:absolute;
  top:50%;
  // 注意不能使用百分比
  // margin的百分比计算是相对于父容器的width来计算的，甚至包括margin-top和margin-bottom
  height: 50px;
  margin-top: -25px;
}
```


夸克扫描王 极速扫描，就是高效

设置父元素相对定位，子元素如下css样式:

```
.box {
  position:relative;
  width: 200px;
  height: 200px;
  background-color: pink;
}

.box-center{
  position:absolute;
  top: 0;
  bottom: 0;
  margin: auto 0;
  height: 100px;
  background-color: greenyellow;
}
```

水平垂直居中

Flex布局（子元素是块级元素）

```
.box {
  display: flex;
  width: 100px;
  height: 100px;
  background-color: pink;
}

.box-center{
  margin: auto;
  background-color: greenyellow;
}
```

Flex布局

```
.box {
  display: flex;
  width: 100px;
  height: 100px;
  background-color: pink;
  justify-content: center;
  align-items: center;
}
```


夸克扫描王 极速扫描，就是高效

```
.box-center{
  background-color: greenyellow;
}
```

> 绝对定位实现(定位元素定宽定高)

```
.box {
  position: relative;
  height: 100px;
  width: 100px;
  background-color: pink;
}

.box-center{
  position: absolute;
  left: 0;
  right: 0;
  bottom: 0;
  top: 0;
  margin: auto;
  width: 50px;
  height: 50px;
  background-color: greenyellow;
}
```

用过Flex么，能简单介绍一下么

> 小提示：如果在项目中使用过，可简单介绍一下自己使用Flex解决过什么问题，这里我在项目中印象比较深刻的是使用Flex解决上面内容高度不固定，下面内容高度自动撑满父容器剩余高度的问题

bind的源码实现

> 小提示：这里我回答使用函数柯里化加上apply或者call可实现bind，面试官追问了一些具体的实现细节。

后来我自己粗糙的实现了一下，仅供参考：

```
Function.prototype.myCall = function (obj) {
  obj.fn = this
  let args = [...arguments].splice(1)
```


夸克扫描王 极速扫描，就是高效

```
  let result = obj.fn(...args)
  delete obj.fn
  return result
}

Function.prototype.myApply = function (obj) {
  obj.fn = this
  let args = arguments[1]
  let result
  if (args) {
    result = obj.fn(...args)
  } else {
    result = obj.fn()
  }

  delete obj.fn

  return result
}

Function.prototype.myBind = function (obj) {
  let context = obj || window
  let _this = this
  let _args = [...arguments].splice(1)

  return function () {
    let args = arguments
    // 产生副作用
    // return obj.fn(..._args, ...args)
    return _this.apply(context, [..._args, ...args])
  }
}

function myFun (argumentA, argumentB) {
  console.log(this.value)
  console.log(argumentA)
  console.log(argumentB)
  return this.value
}

let obj = {
  value: 'ziyi2'
}
console.log(myFun.myCall(obj, 11, 22))
console.log(myFun.myApply(obj, [11, 22]))
console.log(myFun.myBind(obj, 33)(11, 22))
```

伪类和伪元素的区别

夸克扫描王 极速扫描，就是高效

> 小提示：这个问题我当时懵了一下，一下子没反应过来面试官想要问什么，就答了这两者在CSS优先级上有区别，然后由于遇到不会的问题有些紧张就多说了一些废话，但显然这不是面试官想要的答案并且消耗了面试官面试的耐心，说他问的不是这个。这里再次提示大家，如果你感觉你说不清楚，但是你又知道一点，我建议你说不知道，不要纠结，面试官不会因为你不知道一个问题就PASS你，相反你说了一些无关紧要的废话，反而在消耗面试官的耐性，增加负面印象。

- 伪类和伪元素是用来修饰不在文档树中的部分，比如，一句话中的第一个字母，或者是列表中的第一个元素。下面分别对伪类和伪元素进行解释：
- 伪类用于当已有元素处于的某个状态时，为其添加对应的样式，这个状态是根据用户行为而动态变化的。比如说，当用户悬停在指定的元素时，我们可以通过:hover来描述这个元素的状态。虽然它和普通的css类相似，可以为已有的元素添加样式，但是它只有处于dom树无法描述的状态下才能为元素添加样式，所以将其称为伪类。
- 伪元素用于创建一些不在文档树中的元素，并为其添加样式。比如说，我们可以通过:before来在一个元素前增加一些文本，并为这些文本添加样式。虽然用户可以看到这些文本，但是这些文本实际上不在文档树中。

区别

- 伪类的操作对象是文档树中已有的元素，而伪元素则创建了一个文档树外的元素。因此，伪类与伪元素的区别在于：有没有创建一个文档树之外的元素。
- CSS3规范中的要求使用双冒号(::)表示伪元素，以此来区分伪元素和伪类，比如::before和::after等伪元素使用双冒号(:😀，:hover和:active等伪类使用单冒号(😀。除了一些低于IE8版本的浏览器外，大部分浏览器都支持伪元素的双冒号(::)表示方法。

小结

> 对于滴滴的这次面试，我感觉到自己准备的不是很充分，尤其是自己简历上的项目技术Graphql。同时对于自己不会的题目强行做了一些解释说明，其实应该简洁明了的告诉面试官不会。

51信用卡（一面）

在ES5中如何实现继承

> 小提示：这里我说了很多，从借用构造函数到组合继承到寄生组合继承，但面试官其实最想听到的是寄生组合继承。面试官还追问我具体要如何实现寄生组合继承。当然这里其实问的问题还可以很多，比如ES6的类继承和ES5中的继承有什么区别。

绝对定位

夸克扫描王 极速扫描，就是高效

> 小提示：这个建议大家好好回忆一下，例如子元素是相对父元素的padding、border还是content进行定位之类的，当时面试官问的就这么细。

消抖和节流

> 小提示：面试官只是问了一下具体的使用场景，没有问实现原理。

简单消抖

```js
function debounce (fn, wait = 1000) {
  let timeOutId

  return function () {
    let context = this

    if (timeOutId) {
      clearTimeout(timeOutId)
    }

    timeOutId = setTimeout(() => {
      fn.apply(context, arguments)
    }, wait)
  }
}
```

带立即执行参数的消抖

```
function debounceImmediate (fn, wait = 1000, immediate) {
  let timeOutId, context, args

  const later = (immediate) => setTimeout(() => {
    if (!immediate) {
      fn.apply(context, args)
      timeOutId = context = args = null
    }
  }, wait)

  return function () {
    if (!timeOutId) {
      timeOutId = later(true)

      if (immediate) {
        fn.apply(this, arguments)
      }

      context = this
```


夸克扫描王 极速扫描，就是高效

```
        args = arguments
      } else {
        clearTimeout(timeOutId)
        timeOutId = later(false)
      }
    }
  }
```

节流

```
function throttle (fn, wait) {
  let timeoutId = null
  return function () {
    let context = this
    if (!timeoutId) {
      timeoutId = setTimeout(() => {
        fn.apply(context, arguments)
        timeoutId = null
      }, wait)
    }
  }
}
```

Vue中的computed实现原理

> 小提示：这个问题面试官问的很细，绝对是想问你是否阅读过源码。他首先问 `computed` 的实现原理，其次问了这样一个问题：现在有两个 `computed` 计算值，其中一个 `computed` 计算值为什么可以依赖另外一个 `computed` 计算值。这里顺便将watch的实现原理也贴上。

watch的实现原理

> watch的分类：

- `deep watch` （深层次监听）
- `user watch` （用户监听）
- `computed watcher` （计算属性）
- `sync watcher` （同步监听）

> watch实现过程：

- watch的初始化在data初始化之后（此时的data已经通过Object.defineProperty的设置成响应式）
- watch的key会在Watcher里进行值的读取，也就是立马执行get获取value（从而实现data对应的key执行getter实现对于watch的依赖收集），此时如果有- immediate属性那么立马执行watch对应的回调函数

夸克扫描王 极速扫描，就是高效

- 当data对应的key发生变化时，触发user watch实现watch回调函数的执行

computed运行原理

- computed的属性是动态挂载到vm实例上的，和普通的响应式数据在data里声明不同
- 设置computed的getter，如果执行了computed对应的函数，由于函数会读取data属性值，因此又会触发data属性值的getter函数，在这个执行过程中就可以处理computed相对于data的依赖收集关系了
- 首次计算computed的值时，会执行vm.computed属性对应的getter函数（用户指定的computed函数，如果没有设置getter，那么将当前指定的函数赋值computed属性的getter），进行上述的依赖收集
- 如果computed的属性值又依赖了其他computed计算属性值，那么会将当前target暂存到栈中，先进行其他computed计算属性值的依赖收集，等其他计算属性依赖收集完成后，在从栈中pop出来，继续进行当前computed的依赖收集

```js
var vm = new Vue({
  el: '#demo',
  data: {
    firstName: 'Foo',
    lastName: 'Bar'
  },
  computed: {
    fullName: function () {
      return this.firstName + ' ' + this.lastName
    }
  }
})
```

> 由于 this.firstName 和 this.lastName （上面是Vue官方示例）都是响应式变量，因此会触发它们的 getter，根据我们之前的分析，它们会把自身持有的 dep 添加到当前正在计算的 watcher 中，这个时候Dep.target就是这个 computed watcher，具体步骤如下：

- data 属性初始化 getter setter
- computed 计算属性初始化，提供的函数将用作属性 vm.fullName 的 getter
- 当首次获取 fullName 计算属性的值时，Dep 开始依赖收集
- 在执行 message getter 方法时，如果 Dep 处于依赖收集状态，则判定firstName和lastName为 fullName 的依赖，并建立依赖关系
- 当firstName或lastName 发生变化时，根据依赖关系，触发 fullName 的重新计算
- 如果计算值没有发生变化，不会触发视图更新

> 通过以上的分析，我们知道计算属性本质上就是一个 computed watcher，也了解了它的创建过程和被访问触发 getter 以及依赖更新的过程，其实这是最新的计算属性的实现，之所以这么设计是因为 Vue 想确保不仅仅是计算属性依赖的值发生变化，而是当计算属性最终计算的值发生变化才会触发渲染 watcher 重新渲染，本质上是一种优化。

computed计算值为什么还可以依赖另外一个computed计算值

夸克扫描王 极速扫描，就是高效

> 小提示：这个问题当时完全不知道，哎，官方源码的套路太深了.....

周期函数有哪些（beforeCreated和created中间都做了什么）

> 初始化 data、props、computed、watcher、provide。官方源码具体位置src/core/instance/init.js：

```
callHook(vm, 'beforeCreate')
initInjections(vm) // resolve injections before data/props
initState(vm)
initProvide(vm) // resolve provide after data/props
callHook(vm, 'created')
```

小结

> 51信用卡的这次面试其实面试官考察的点还是蛮深入的，问了一些Vue底层源码的实现，总体感觉自己回答的还可以，但是面试官说：你应该去阿里.

阿里部门未知（一面）

说说Webpack的实现原理

> 小提示：这个直接回答不知道，问题较大，我这里猜测一下是类似Babel和AST抽象语法树相关，有空去看下源码。

首屏优化有哪些解决方案

> 小提示：这个问题在回答懒加载的过程中，面试官追问懒加载的Webpack配置，我说了和代码切割相关

Node.js的加载机制（require和module.exports）

> 小提示：这个问题其实是非常常见的问题，建议大家阅读一下源码，有些也可能会问一下比较简单的问题，例如module.exports和exports的区别，或者也可能问CommonJS引入和ES6引入的区别。

你觉得你最擅长什么

夸克扫描王 极速扫描，就是高效

> 小提示：这个问题是个大坑阿，我这里直接回答我什么都不擅长，这样回答显然面试官是不会不满意的，建议大家在面试前好好想想自己到底擅长啥。

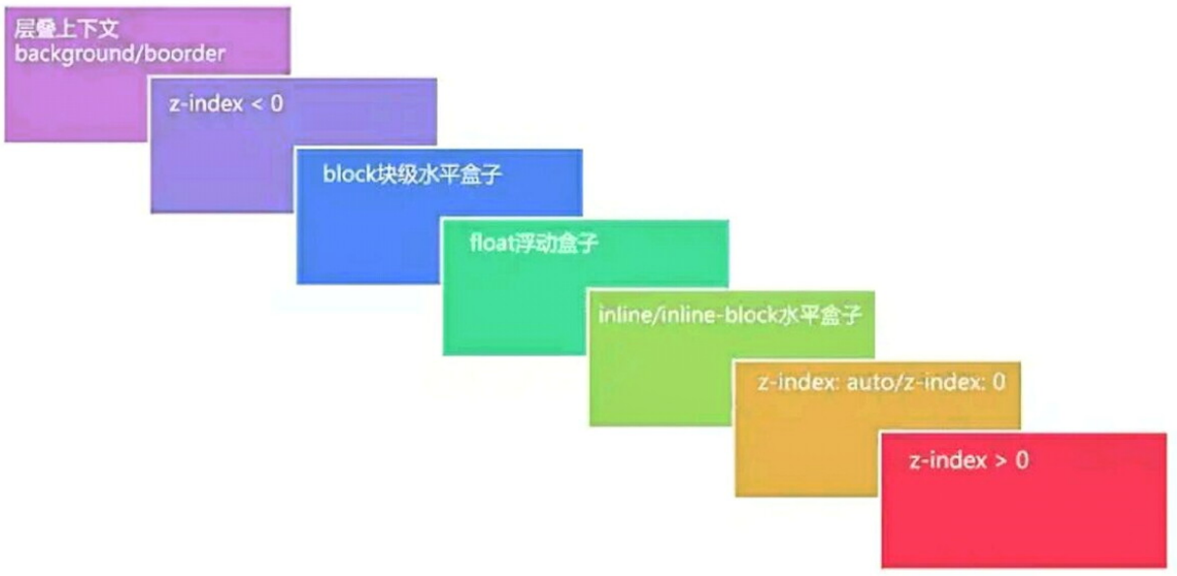
React和Vue的区别

> 小提示：这里React真的好久没用了，几乎忘记了，大致说了下单向数据流、双向数据绑定、数据监听方式、JSX以及Vue的单文件组件、函数式编程、Vue的指令之类的。

阿里CBU技术部（一面）

说说z-index有什么需要注意的地方

> 小提示：真的忘记的差不多了，就简单说了只能在同一层叠上下文中进行z-index值比较、和绝对定位的关系，z-index值不需要设置过大，只需要理清楚层级关系即可。面试官追问了z-index值和background的覆盖关系，还追问了绝对定位元素以及后来居上的准则。面试官还问了z-index默认值是什么，0和auto有没有区别？真的对于CSS可能平常就用的不多，所以这个问题答的不是很好。



> 这里由于回答了定位，面试官追问固定定位的元素是相对于什么进行定位？相对定位会脱离正常文档流么？绝对定位是相对于什么元素进行定位？

熟悉CSS3动画么

夸克扫描王
极速扫描，就是高效

> 小提示：CSS3动画硬件加速？CSS3动画的性能问题（重绘和重流，是否需要脱离正常文档流）？这个我当时答不知道，确实平常用的很少，如果熟悉Vue过渡动画的同学可以讲讲过渡动画？

有没有做过什么可视化的项目

> 小提示：我的回答：地图算么？基于OpenLayers设计过地图的Vue组件库。

你觉得你最擅长的是什么

> 小提示：这个问题简直就是给人挖坑。

Flex实现两列布局

> 这里简单实现一下（其实应该使用flex-basis属性）：

```html
<div class="box">
  <div class="box-left"></div>
  <div class="box-right"></div>
</div>
```

```css
.box {
  height: 200px;
  display: flex;
}

.box > div {
  height: 100%;
}

.box-left {
  width: 200px;
  background-color: blue;
}

.box-right {
  flex: 1; // 设置flex-grow属性为1，默认为0
  background-color: red;
}
```


夸克扫描王 极速扫描，就是高效

说说DOM事件流

> 小提示：面试官追问事件委托有什么优点（起码两个以上）、target/currentTarget/relateTarget具体指向什么目标。

你觉得你有做过推动流程或者改善流程的事件么，举例说明

> 小提示：这个如果做过什么规范或者开发工具之类的，应该比较好回答。

小结

> 总体来说这次面试面得很细，有些知识点已经忘记，建议大家面试前把一些感觉不是很熟悉的原生知识点回忆起来，尤其是在开发中都不怎么会使用一些CSS样式设计的童鞋（现在很多都是组件库的设计方案，样式早已经封装掉了）。

阿里企业智能事业部（一面）

Webpack的loader和plugins的区别

> 小提示：当时直接回答不知道，确实Webpack我只会用，还没了解过内部的实现原理和构成。这个后续无论如何都要好好理解一下原理。

HTTP状态码206是干什么的

> 小提示：工作中没有遇到过需要上传下载大型文件，所以这个问题当时老老实实回答不知道。具体应该和断点续传相关，可能也需要回答一些range的头部信息等。

React高阶组件的作用有哪些

> 小提示：好久没用过React了，大致只知道Racct是单向数据流的，利用高阶组件可以实现类似于Vue的双向数据绑定。

夸克扫描王 极速扫描，就是高效

Service Worker有哪些作用

> 小提示：当时怕说错，老老实实回答不知道。后来查了一下应该和缓存以及HTTP请求拦截相关。

跨域

文件上传的二进制具体是怎么处理的

> 小提示：只知道上传的头信息是application/x-www-form-urlencoded，也可以对上传的文件的数据进行拦截处理，例如对上传文件的信息进行加密处理。

阿里企业智能事业部（二面）

Vue的整个实现原理

> 小提示：当时面试官问的蛮好玩的，他问从开始写一个.vue文件开始到DOM渲染到页面上，Vue做了哪些工作。然后我当时没理解面试官是要问vue-loader？DOM树的渲染过程？来来回回试探性的问了面试官几次，才理解原来面试官想知道Vue源码的整个实现过程

Chrome插件如何屏蔽广告

> 小提示：这个问题当时回答不知道，其实后面想想最简单的办法是先找出广告元素的一些通用特性，然后在Chrome插件中通过注入脚本的形式将这些广告元素隐藏掉。

这里不知道有没有更好的其他方式，例如不知道Service Work对请求拦截处理是否可以有效屏蔽广告等，这个问题希望同学可以补充一下。

如何判断两个变量相等

> 小提示：这里需要分基本类型和引用类型，面试官在这里具体想问的是Object.is的实现原理。这是面试官问我的第一个问题，当时直接回答不知道，内心都觉得接下来要凉凉了。

Vue的数据为什么频繁变化但只会更新一次

夸克扫描王 极速扫描，就是高效

> 小提示：这里问的是Vue源码对于视图更新的优化。我这里的回答是乱糟糟的，希望有同学能够给出一个精准并且简短的回答。

- Vue 异步执行 DOM 更新。只要观察到数据变化，Vue 将开启一个队列，并缓冲在同一事件循环中发生的所有数据改变。如果同一个 watcher 被多次触发，只会被推入到队列中一次。这种在缓冲时去除重复数据对于避免不必要的计算和 DOM 操作上非常重要。然后，在下一个的事件循环“tick”中，Vue 刷新队列并执行实际 (已去重的) 工作。Vue 在内部尝试对异步队列使用原生的 Promise.then 和 MessageChannel，如果执行环境不支持，会采用 setTimeout(fn, 0) 代替
- 另外，关于waiting变量，这是很重要的一个标志位，它保证flushSchedulerQueue回调（$nextTick中执行）允许被置入callbacks一次。
- 因为Vue的事件机制是通过事件队列来调度执行，会等主进程执行空闲后进行调度，所以先会去等待所有的同步代码执行完成之后再去一次更新。这样的性能优势很明显，比如：

> 现在有这样的一种情况，mounted的时候test的值会被循环执行++1000次。 每次++时，都会根据响应式触发setter->Dep->Watcher->update->run。 如果这时候没有异步更新视图，那么每次++都会直接操作DOM更新视图，这是非常消耗性能的。 所以Vue实现了一个queue队列，在下一个tick（或者是当前tick的微任务阶段）统一执行queue中Watcher的run。同时，拥有相同id的Watcher不会被重复加入到该queue中去，所以不会执行1000次Watcher的run。最终更新视图只会直接将test对的DOM的0变成1000。 保证更新视图操作DOM的动作是在当前栈执行完以后下一个tick（或者是当前tick的微任务阶段）的时候调用，大大优化了性能。

> 执行顺序update -> queueWatcher -> 维护观察者队列（重复id的Watcher处理） -> waiting标志位处理（保证需要更新DOM或者Watcher视图更新的方法flushSchedulerQueue只会被推入异步执行的$nextTick回调数组一次） -> 处理$nextTick（在为微任务或者宏任务中异步更新DOM）->

- 由于VUE的数据驱动视图更新是异步的，即修改数据的当下，视图不会立刻更新，而是等同一事件循环中的所有数据变化完成之后，再统一进行视图更新。在同一事件循环中的数据变化后，DOM完成更新，立即执行nextTick(callback)内的回调。
- vue和react一样，对dom的修改都是异步的。它会在队列里记录你对dom的操作并进行diff操作，后一个操作会覆盖前一个，然后更新dom。

绝对定位、固定定位和z-index

- 一旦给元素加上absolute或float就相当于给元素加上了display:block
- absolute元素覆盖正常文档流内元素（不用设z-index，自然覆盖）
- 可以减少重绘和回流的开销（如absolute+ top:-9999em，或absolute + visibility:hidden，将动画效果放到absolute元素中）

属性介绍

- static，默认值。位置设置为static的元素，它始终会处于文档流给予的位置。
- inherit，规定应该从父元素继承 position 属性的值。但是任何的版本的 Internet Explorer （包括 IE8）都不支持属性值 “inherit”

夸克扫描王 极速扫描，就是高效

- fixed，生成绝对定位的元素。默认情况下，可定位于相对于浏览器窗口的指定坐标。元素的位置通过 “left”, “top”, “right” 以及 “bottom” 属性进行规定。不论窗口滚动与否，元素都会留在那个位置。但当祖先元素具有transform属性且不为none时，就会相对于祖先元素指定坐标，而不是浏览器窗口。
- absolute，生成绝对定位的元素，相对于距该元素最近的已定位的祖先元素进行定位。此元素的位置可通过 “left”、“top”、“right” 以及 “bottom” 属性来规定。
- relative，生成相对定位的元素，相对于该元素在文档中的初始位置进行定位。通过 “left”、“top”、“right” 以及 “bottom” 属性来设置此元素相对于自身位置的偏移。

> 浮动、绝对定位和固定定位会脱离文档流，相对定位不会脱离文档流，绝对定位相对于该元素最近的已定位的祖先元素，如果没有一个祖先元素设置定位，那么参照物是body层。

绝对定位相对于包含块的起始位置：

- 如果祖先元素是块级元素，包含块则设置为该元素的内边距边界。
- 如果祖先元素是行内元素，包含块则设置为该祖先元素的内容边界。

问答题：

- 定位的元素的起始位置为父包含块的内边距（不会在border里，除非使用负值，会在padding里）
- 定位的元素的margin还是能起作用的
- background属性是会显示在border里的
- z-index是有层叠层级的，需要考虑同一个层叠上下文的层叠优先级
- z-index是负值不会覆盖包含块的背景色（但是如果有内容，会被包含块的内容覆盖）
- z-index的值影响的元素是定位元素以及flex盒子
- 上面一个定位元素，下面一个正常流的元素，定位元素会覆盖在正常流元素之上，除非给z-index是负值
- 页面根元素html天生具有层叠上下文，称之为“根层叠上下文”

阿里企业智能事业部（HR面）

> 企业智能事业部Leader面后又收到了HR面的面试通知，这一轮面试大致问了以下问题：

- 你为什么要离开现在的公司
- 你们公司的岗位等级是怎么评定的，你现在是什么岗位等级
- 谈谈你在公司的绩效情况
- 你觉得你做的最有成就感的一件事
- 你一般解决问题的方法有哪些
- 你是因为什么契机选择做前端
- 你有对你所在的公司做过什么流程或制度规范上的改进么
- 你最近在看什么书，和工作相关么，你为什么要看这些书
- 看到你之前还面试了其他两个部门都挂在了一面，你感觉是什么原因

夸克扫描王 极速扫描，就是高效

- 你期望的薪资待遇是多少

> 小提示：这里HR会问的其实不止这些问题，例如你为什么喜欢Web前端这个岗位、你未来的职业规划、你觉得你的优点和缺点有哪些、为什么选择阿里巴巴、对之前几个面试官做下评价、你用过阿里的哪些产品顺便谈谈这些产品的优缺点、你对于互联网是怎么理解的...

小结

> 对于HR面还是要好好准备的，尤其是有些问题还是很容易挖坑的，例如你为什么离开现在的公司（你当然不应该抱怨现在的公司有哪些不好的地方，更多的应该表明自己想要寻找更好的发展机会，自己的一些现实因素，比如对于我而言是现在应聘的公司离自己的家更近，又或者是自己工作到达了迷茫期，想跳出迷茫期等等），你觉得你做的最有成就感的一件事（你要是说个简单的，HR会觉得你工作能力不强），你一般解决问题的方法有哪些（HR当然也想考察你解决问题的能力，你要是说什么百度啊之类的HR当然会觉得你解决问题的能力不强），你期望的薪资待遇是多少（你要是不喜欢这家公司，可以期望高一些，你要是很喜欢这家公司面试过程很愉快上浮个30%左右，面试过程一般上浮个20%左右）

阿里社招面经

笔试

- 阶楼梯问题(尾递归的优化[函数最后调用自身可以减少中间变量|化递归为循环去优化]/备忘录优化)
- 节流防抖的实现(分析笔试答案时问到，总结差别，说出应用场景)
- 深拷贝(类型考虑/Symbol/Date等的构造/循环优化/共用引用优化等),这里贴2篇文章给大家参考,https://juejin.im/post/5d6aa4f96fb9a06b112ad5b1,https://mp.weixin.qq.com/s/M7KBX3w2KqlWhZFHJSYP6
- 封装一个只能输入数字的React组件

问答题

- 输入URL后发生了什么（面试官：浏览器输入URL后发生了什么？）
- 事件循环说一下（面试题：说说事件循环机制(满分答案来了)）
- JS有几种基本类型，分别是什么，与对象的区别
- JS 执行过程中是如何保存上下文的（建议看看极客时间的《浏览器工作原理》）
- 你认为什么是前端
- 为什么想来阿里
- 最近在看什么书
- 微前端了解吗
- 你想成为什么样的人(个人成长规划)
- 项目中遇到的最大挑战是什么，如何解决的

夸克扫描王 极速扫描，就是高效

- node 大量日志怎么处理的（缓冲队列/采样率降低等）
- http2 的特点（「知识拾遗」 http2/http3总结）
- node 与其他语言有什么区别，其优劣势是什么
- csrf 的核心原理(https://juejin.im/post/5bc009996fb9a05d0a055192)
- node 事件循环
- FCP/FMP/FP 分别是怎样定义，如何统计
- MySQL 题目，id 唯一，name 可以重复，求 name 有重复的所有数据
- 有没有推动过什么东西
- 未来的一个发展是否是全栈，如何规划
- 链路录屏的原理（网页录屏(帧记录)调研总结）
- 链路录屏法务问题如何解决
- FMP 定义及统计，w3c的草案你知道吗
- react 项目的可持续维护性如何体现
- react hooks用过吗，为什么要用
- h5 和小程序有什么区别
- https加密原理，中间人攻击知道吗（「知识拾遗」你应该知道的 https）
- ng 负载均衡的方式有哪些
- CDN 有什么作用
- 不可变数据里面，immutable，删除数组里面的列表
- 垃圾回收中，内部函数返给上层的对象是如何管理的
- es 静态分析原理(https://zhuanlan.zhihu.com/p/33843378)
- 有一个很大很大的文件，如何统计文件里面重复最多的行是哪行(参考git的记录存储)
- 数组去重，考察 ES6 语法
- 函数柯里化
- 水平垂直居中
- Promise 实现原理
- 前端储存方式，优缺点
- 移动端页面适配解决方案
- React、Vue 区别和实现原理，React 中的 Fiber 是什么
- JS 的 Event Loop
- 三次握手四次挥手

微信社招面经

笔试

夸克扫描王
极速扫描，就是高效

- 数组交集，编写一个函数，输入两个数组，输出它们的交集。输出数组中不含重复的元素，元素排列顺序可随意。
- 二叉树的搜索，输入一个普通二叉树的根节点，实现一个调度器，调用调度器的next()方法，将返回二叉树中下一个最小的数；调用迭代器的hasNext()方法，将返回是否存在下一个数。二叉树节点是整数，无序。
- 三角形个数，输入一个非负整数的数组，如果将数组元素选作三角形的边长，编写一个函数，输出这个数组可构成的三角形数量。
- 数组切分问题，输入一个正序排列的整型数组，如果它可以被切分为1个或多个子序列，输出True，反之False。子序列需为连续的整型数组，并且长度至少为3。

```
例1：
输入： [1,2,3,3,4,5]
输出：True
解释：可以切分为2个各自连续的子序列：
1, 2, 3
3, 4, 5
例2：
输入： [1,2,3,3,4,4,5,5]
输出：True
解释：可以切分为2个各自连续的子序列：
1, 2, 3, 4, 5
3, 4, 5
例3：
输入： [1,2,3,4,4,5]
输出：False
解释：无法切分出长度至少为3的子序列。
```

- 一只青蛙一次可以跳上1级台阶，也可以跳上2级。求该青蛙跳上一个n级的台阶总共有多少种跳法.

求下面代码的输出

```
function test(a,b) {
  console.log(b)
  return {
    test:function(c){
      return test(c,a);
    }
  };
}

var retA = test(0);
retA.test(2);
retA.test(4);
retA.test(8);
var retB = test(0).test(2).test(4).test(8);
var retC = test('good').test('bad');
```


夸克扫描王 极速扫描，就是高效

```
retC.test('good');
retC.test('bad');
```

闭包问题及优化

```
for (var i = 0; i < 4; i++) {
  setTimeout(function() {
    console.log(i);
  }, 300);
}
```

与项目关系不大的问答题

- 节流防抖(实现/场景/源码，如lodash实现原理)
- 事件循环(浏览器/node/版本差异)
- setTimeout 实现原理
- react 和 vue 的区别
- Promise 原理
- 前端错误监控及容灾
- 性能优化
- 谈谈 node 的内存泄漏
- 开发过程中遇到的最大挑战是什么
- 学习的动力怎么来的，如何维持
- 浏览器的渲染机制是怎样的
- SSR 作用及优缺点
- 如何进行状态管理
- webpack 及浏览器的技术分享目的是什么，分享了什么，怎么做的分享
- 如何进行项目重构
- 进程与线程的区别
- 说说知道的设计模式

PS：由于面试全过程历时比较长，一些题目已经忘记了。大概记得这么点，然后很看项目，每一轮都详细问了项目～

网易面经

第一个场景问题

夸克扫描王 极速扫描，就是高效

> 比如直播的场景，你应该知道吧，你需要实现一个这样子的场景，比如某个老师点击某个地方，比如U盘，你这个时候需要展示U盘的动画效果，比如这个时候，老师点击这个电脑屏幕，你需要展示一个小电脑的动画效果，向上述这样子，需要在特定的时间点，完成特定的动画效果。

- 这个问题，我的想法是，动画是例外加上去的，如果说是直接后期处理的话，那应该跟我们前端的关系不大了，所以我们接下来的问题，就是如何去处理，时间同步的问题，怎么在具体的时间点，开始展示动画呢
- 第二个问题，假设我们可以获取到某个时间点的动画，那么接下来，小哥哥，给我们提出了一个新的问题，就是当你的网络拥塞时，比如有延迟的时候，这个时候，出现卡顿的效果，原本需要5秒播放完的，可能需要7秒，那么你是如何去解决动画同步的?

> 我没有做过这种类似的问题，所以回答起来感觉很吃力，有了解的小伙伴可以评论留下你们的答案，我虚心学习。

第二个场景问题

> 有一个场景，在一个输入框输入内容，怎么更加高效的去提示用户你输入的信息，举个例子，你输入天猫，那么对应的提示信息是天猫商城，天猫集团，这个信息如何最快的获取，有没有不需要发请求的方式来实现?

- 比如数据请求的时候，尽量发的请求少，那么可以做防抖和节流处理
- 接下来的小哥哥，给了新的场景，当你的服务器挂了，数据取不到，那如何设计一个小型的本地数据库
- 接下来问题就是如何设计一个本地的数据，优化的点，就是尽可能的快，每次查询数据尽可能的快
- 我的第一个思路，是key-value这样子去设计，但是随后就被小哥哥给质疑出问题了，举个例子，如果按照你的想法，如果关键字为 天，天猫，这样子设计数据的话，就会存在被数据重复，这样子显然是不合理的。
- 想了很久，这个时候，既然有前缀重合的情况，那么是不是有一种数据结构可以解决这个问题呢? 👇
- 字典树，我们可以在本地建立一个预读的字典树，这样子的话，根据用户输入的内容，查字典树，这个时间复杂度大概就是O(m+n)，所以很大程度上加快了查找效率。

第三个场景问题

> Git版本工具你使用过吧，那你能不能实现一个这样子的效果，完成Git Diff算法，比较两个文件的不同，然后说一说具体的思路，这个过程怎么去比

- 一开始我想的是diff算法，比如是vue中的虚拟dom算法，但是感觉不对啊，diff是做了优化的，这里很明显不合理，于是这个方法就不合理了。
- 那么两个文件，该如何快速的找到对应的两者文件的差别呢? 这个问题想了好久，嗯，当时自己好像是口胡了一些思路，比如去逐行逐行的比较，这样子的话，其实也不是很合理的，仔细想一想不行
- 小哥哥提示了我，我们是不是要去找最长的公共子串，这个是时候，我应该想起来这个是两个串的LCS,应该就是经典的动态规划问题,最后一个问题，确实没有想到这个，可能就是很久没有接触这类动态规

夸克扫描王 极速扫描，就是高效

划问题了。

- 核心应该是动态规划解决LCS，以及后续的处理，可以去看看有些文章，写的很不错，我这里就不张开啦。

H5新特性

> 简历上面写了这个内容，所以被问到了，害，当时脑子一蒙，都完全没有答好，这里在好好的梳理一遍?

- 本地存储特性
- 设备兼容特性 HTML5提供了前所未有的数据与应用接入开放接口
- 连接特性 WebSockets
- 网页多媒体特性 支持Audio Video SVG Canvas WebGL CSS3
- CSS3特性

> 增加拖放API、地理定位、SVG绘图、canvas绘图、Web Worker、WebSocket

然后我答了本地存储，接下来就问我这方便的问题啦?

localstroge sessionstoge 区别 应用场景

vue组件间通信

> 好几次面试都问了这个问题，这个问题我是这么看待的，取决于你平时项目中经常使用的方式是哪些，所以我每次都会答三种方式，反而网上8种组件间通信的方式，我记不住，也觉得了解一下即可，跟面试官交流一下，你在项目种是如何使用的即可。

遍历对象方法

- for in 遍历的也可以，不过对于非继承的属性名称也会获取到，通过hasOwnproperty判断
- Object.keys() 可枚举属性和方法名
- `Object.getOwnPropertyNames()` 可以获取数组内包括自身拥有的枚举或不可枚举属性名称字符串，如果是数组的话，还有可能获取到length属性

数组去重

> 常规题，讲清楚思路，最后小哥哥提示能不能使用O(n)，我给出了两者方案

- `Set`

夸克扫描王 极速扫描，就是高效

- 用对象特性，我觉得他就是想考这个，给你们贴一个代码吧?

```
let unique = arr => {
  let obj = {}
  return arr.filter((ele) => {
      return obj.hasOwnProperty(typeof ele + ele) ? false : (obj[typeof ele + ele] = tru
e)
  })
}
```

数组的扁平化

```
let flatArr = (arr) => {
  return arr.reduce((res, ele) => {
      if(Object.prototype.toString.call(ele).slice(8,-1) === 'Array') {
          return [...res, ...flatArr(ele)]
      }else{
          return [...res, ele]
      }
  },[])
}
let arr = [1,2,3,[2,3,4,5]];
console.log(flatArr(arr))
```

> 当然了，实现的方式有很多种，看你自己怎么去实现它了，最简单的就是去递归对象。

深度遍历

```
const tree = {
    name: 'root',
    children: [
        {
            name: 'c1',
            children: [
                {
                        name: 'c11',
                    children: []
                    },
                    {
                        name: 'c12',
                    children: []
```


夸克扫描王 极速扫描，就是高效

```
                }
            ]
        },
        {
            name: 'c2',
            children: [
                {
                        name: 'c21',
                    children: []
                    },
                    {
                        name: 'c22',
                    children: []
                }
            ]
        }
    ]
}

// 深度优先的方式遍历 打印 name
// ['root', 'c1','c11', 'c12', 'c2', 'c21', 'c22']
```

这题，我一开始想到的就是递归的写法，写完之后，然后小哥哥问了我递归的缺点，以及如何去优化，不用递归的方法该怎么去实现?

面试的时候，没有写出来，太紧张了，不在状态，复盘的时候，写了一下用栈的实现方式?

```
function solve(root) {
  let stack = [],
      result = [];
  if(!root) return [];
  stack.push(root)
  while(stack.length) {
      let node = stack.pop()
      if(node == null ) continue
      result.push(node.name)
      for(let i = node.children.length-1; i >= 0; i--) {
          // 这里就是面试的重点,应该从后面的节点压入栈中
          stack.push(node.children[i])
      }
  }
  return result
}
```

链表的相加问题?

夸克扫描王 极速扫描，就是高效

这个是LeetCode上面的题目，我好像还写过，面试的最后一题的时候，我以及蒙了，完全不知道自己在干嘛，其实链表题都是套路，我连套路都没有掌握

ES6语法，Promise了解吗

```js
const promise = new Promise((resolve, reject) => {
    console.log(1);
    resolve();
    reject()
    console.log(2);
})

promise.then(() => {
    console.log(3);
},() => {
    console.log("失败的状态")
})

console.log(4);
```

> 我看到以后，就直接说答案了，这点不好，因为一般而言，面试官出的题目肯定有点小坑，下次要注意了，最后面试官小姐姐还是微笑的告诉我，应该这么去做，然后怎么怎么样。

聊一聊map和set

这个我是跟她说了用法，以及它们之间的区别，也就是它们经常使用的场景是哪些。

顺便的话，就聊了一下Weakmap，然后这里的难点也不是很多，就是你的明白它们两者数据结构的区别是啥，举个例子说明情况即可。

前端性能优化

> 这个问题太大了，而且对于一个实际开发经验为0的而言，这个问题就很置命，所以呢，我就准备了从URL到页面渲染这个一块去说，里面的优化点挺多的，可以自行去了解。

- 构建请求行
- 查缓存 （重点说一说）
- dns解析（如何优化）
- tcp http （比如减少请求次数，嗯，应该还有其他优化吧，cdn？）
- 浏览器渲染过程 （这里面就有优化了，比如常见的如何避免回流和重绘）

夸克扫描王 极速扫描，就是高效

- 防抖和节流处理
- webpack打包优化也可以说一说，前提你得有自信

字节跳动三轮技术+一轮HR

抖音一面(45min)：

- 自我介绍一下
- VueRouter 的原理你能不能说一下呢？(两种路由方式说了一下)
  - 对于 History 路由而言，你觉得在服务端是如何做路由分发的呢？(愣住，面试官接下来跟我解释了一波)
- 你说你看过 Vue 源码，能不能介绍一下 Vuex 的 Mutation 和 Action 的区别吗？(mutation做同步操作，action一般用于异步)
  - 为什么要设计出 Mutation 和 Action 这两个东西？(我开始不是很清楚，扯到 Redux，和面试官交流后，一致同意 Action 作为业务逻辑的封装更合适，提供了更大的自由度)
- 进程和线程的区别(解释了一波，顺便把Chrome为什么从单进程转成多进程架构说了一下)
  - 知道哪些进程间通信(IPC)的方式？ (主从式、会话式、消息-邮箱机制、管道、共享内存、Unix Domain Socket，然后跟他讲我看过 Chromium IPC 的源码，内核里面把以前的 ChannelPosix 换成了 ChannelMojo，从而达到线程安全的目的，顺便解释了下线程安全，面试官表示很欣赏，说这个都看过，看来你学了不少)
- 能不能说说从输入 URL 到页面渲染经历了什么？(被问过很多次了，DNS 解析过程，HTML词法分析和语法分析，CSS解析， 合成图层、合成线程调用光栅化线程池，生成位图后浏览器进程间通信过程，显卡缓存与显示器的关系，面试官说可以)

抖音二面(1个小时)

> 这个面试官就比较高冷了，全程不露脸，说话语气特别凝重，可能沉默 5 秒钟然后问你一个非常严肃的问题。

- 简单的自我介绍
- ES5写一个数组去重(刚开始写了一个O(n^2)时间的)
  - 能不能优化？ (我问能不能用新空间，他说可以，然后写了一个O(n)时间的)
  - 能不能区别开数字和字符串？(想了想，最后还是用indexOf方式，最优的没想出来，面完猛然想出来了，当时脑子有点乱)
- 讲一讲 HTTPS 加密(对称加密有AES + CHACHA20, 分组模式以前有 CBC、CTR，TLS1.3 中只剩下 GCM，非对称加密 RSA、ECDHE)
  - 怎么握手的呢？(讲了三个版本：传统 RSA、TLS1.2、TLS1.3, 后面也讲了 TLS1.3 的 Session ID、Session Ticket 以及 PSK)

夸克扫描王 极速扫描，就是高效

        - HTTPS 如何保证数据是否被篡改？(说了下有签名的过程)
            - 签名是什么原理(私钥加密，公钥解密，比对哈希摘要)
                - 你知道哪些哈希摘要算法(Sha256, Sha384)
- 你能不能介绍一下你的项目(说了下项目遇到的挑战，说了这几点: 1. 怎么解决闭包陷阱。2. 通过 EventLoop 解决 transform 失效的问题)
- 能不能说说你对 EventLoop 的理解(宏任务-微任务-UI渲染)
    - 如果要在UI渲染之前做一些事情你会怎么办？(我会启动微任务执行吧)
    - requestAnimationFrame 在 EventLoop 中是一个什么位置？(给他解释显示器和浏览器的 Vsync 信号，然后rAF首先执行，他貌似不满意，我请教了他一下，给我解释实际上rAF会在UI渲染之前)
        - 分离图层做动画有什么好处呢？(给他讲了分层的原理，通过设置 CSS 的 will-change 可以转换为一个图层，调用 GPU 加速)
            - 分离图层会发生重绘吗？(会)那既然重绘，它的好处在哪里？(不会影响其他的图层)
- 你觉得你哪些技术比较厉害？(Vue 源码，浏览器，服务端渲染)
- 你说你看过 Vue 的源码，能不能说说 computed 属性为什么能够在依赖改变的时候，自己发生变化？(我说 computed 和 watch 公用一个 Watcher 类，在 computed 的情况下有一个 dep 收集依赖，从而达到更新computed属性的效果，顺便跟他讲了computed Watcher如何跟渲染Watcher关联，以及 Vue 在二次收集依赖时用 cleanupDeps 卸载一些无用的 dep)
- 你觉得你的优势是什么？(1.深度思考的能力 2.善于分享 3.社区影响力)
- 你对 webpack 了解多少？(我说了下 webpack 的一些优化手段，打包时间方面，预编译、缓存、缩小构建目标，说了大概十个插件，然后打包体积上，JS 和 CSS 的Tree-Shaking 怎么配置)
    - 你觉得 CommonJS 为什么不能做 Tree-Shaking ？
        - ESModule 既然是编译时加载，那它可以做到运行时加载吗，想过这个问题吗？(愣了一会，说 webpack 有动态 import 的方式)
            - 写过 loader 和 plugin 吗？(实话实说，没有)那你知道两者有什么差异吗？(先loader后plugin)
- 你对未来的发展是如何规划的？(谈了谈我对五级工程师的看法，我的阶段目标是到达最低的第五级)
    - 你觉得你在专业上的目标是怎样的？(成为领域前20%)
- 有什么想问我的？(问了下可能触及的技术栈，以及抖音在开源方面的打算)

抖音三面(1个小时)

- 你是如何接触到前端的？为什么要选择前端？为什么不去搞 c++ 底层系统研发、后端、人工智能？(说了下我的看法，他表示认可)
- 做这个项目的初衷是什么？里面有用到专业上学到的知识吗？
- 如果每次要生成不一样的 ID，你怎么来设计这个系统呢？(跟他聊了聊哈希冲突如何解决，他表示可以)
- 在学校应该学过 C 吧，你觉得 C 语言在设计上有什么好或者不好的地方呢？(提了下虚基类，实在想不起来这玩意啥用了，顺带说了下多继承的问题)
    - 如果让你用 JS 来实现多继承的功能，你会怎么来做呢，或者用什么其他的方式？(说了下自己对于继承的看法，以及这个设计不太好的地方，然后更喜欢组合式的方式)

夸克扫描王 极速扫描，就是高效

    - 比如说你是你妈妈的儿子，同时也是公司里的员工、学校里的学生，你怎么去设计出你这个实例呢？(将不同的角色功能进行组合，然后实例化)
- 对前端的知识体系如果要分类的话，你怎么分类？(谈了谈未来前端发展的四个分支)
    - 如果基础知识要细分的话，你怎么来分？(说了下学习基础知识的三部曲，第一级应用，第二级原理，第三级建设社区和贡献代码，我觉得我目前在第二级到第三级之间)
- 有想过去做一个开源的工具吗？(谈了谈自己对于开源的理解)
- 有什么想要问我的吗？(聊了聊面试官自己的技术经历，全程还是蛮愉快的)

HR 面:(30min)

- 做项目的初衷在什么地方?
- 公司的业务可能会压榨自己开源分享的时间，你会介意吗?
- 公司内的一些代码不能开源，和你自己的开源分享冲突了，你怎么办?
- 自己平时有什么兴趣爱好?

整体感受

面试了一下午，前面三面连续说了太久，到 HR 面的时候声音已经嘶哑了，不过字节的面试体验还是非常好的，效率相当高。自我感觉还算不错，主要是因为两点:

面试官问的问题都是引导性的，而不是死扣一个具体的知识点叫你填空，通常给一个非常宽泛的话题，让你去发挥，因此给了自己非常大的发挥空间，不过这也和自己前期充分的准备有关系。 面试官尝试去问你一些更加深入的问题，直到把你问住，这是好事情，因为他的水平一般在你之上，能问出深度来，说明他很重视你。

淘宝两轮

淘宝一面

- 首先自我介绍，怎么学习前端的
- 能不能说说浏览器的缓存(灵魂之问，一时想不起来了，尴尬，寻求提示)
    - cache-control 有哪些字段？设置 max-age: 0 跟浏览器缓不缓存有关系吗？s-max-age 的作用?
    - 强缓存和协商缓存的顺序
- 能不能说说 Cookie 有哪些字段？(我说了 domain、path、httpOnly、sameSite的三种取值)
    - 还有吗？关于 https 的一个字段(擦，不知道，过吧)
- 说说箭头函数和普通函数的区别(说了写法，原型，this，还有呢？愣了一会，寻求提示)
    - 箭头函数的 this 是声明时确定还是运行时确定？(声明时确定)

夸克扫描王 极速扫描，就是高效

    - 箭头函数可以 new 吗？ 说一说 new 的原理。
- TCP 三次握手
- 你应该对 React 原理很了解吗？(我打断了，React 原理不熟，问我 Vue 吧，后来问了一个 diff 就完事了)
- 从输入URL到页面展示发生了什么？(我说了大概 2 分钟 DNS 解析过程，被喊停，然后继续讲HTML解析，CSS解析，合成图层、合成线程调用光栅化线程池，生成位图后浏览器进程间通信过程，显卡缓存与显示器的关系，大概说10-15分钟吧)
- 重绘和回流了解吗?
- 怎么进行优化？(说了一种批量操作，别的好像忘了，他提示我 DOM 离线操作也可以)
- 有没有了解过前端一些前沿的方向(说了flutter，dart，看过你们团队的 serverless 文章)
    - 了解过 WebAssembly 吗？(没有啊)
    - 了解过 PWA ?(我个人觉得要凉，然后问我 PWA 原理是怎么样的呢？说了下大概Service Worker吧，不太熟)

淘宝二面

- 介绍一下你的项目(我说了一下技术栈，遇到的挑战和解决方案)
- 你只学了一年半，就有这么多关注量了，你是怎么做到的?
- 你觉得你基础怎么样？(JS 很扎实，Vue 翻完源码，React 略懂原理)
- 你觉得 React 和 Vue 有什么共通之处?
- 说一下浏览器渲染过程(说的很详细，面试官说可以，细节把握的很professional)
- 说一下对于前端技术的发展过程(从刀耕火种的年代说起，到后来的jq，angular，react，vue 三大框架，gulp、grunt、rollup、webpack 打包工具，然后到未来的一些方向，比如 PWA, 跨端，serverless，微前端，webassemblely，加入了我自己的理解)
- 你觉得前端除了完成页面交互稿之外，还能做其他的什么事情？(首先是性能优化，其次是处理数据，然后是工程化)
    - 你觉得工程化的理解是怎样的？(从代码的角度，编译、压缩、规范，从人的角度，团队协作、统一产出标准)
- 你觉得你选择阿里云或者淘宝，你选择的标准的什么?
- 还有什么想问我的吗？(问了一些转正和部门相关的事情)

总体而言，面试官基本没问什么基础，考察对前端的思考和个人的一些思考，跟我讲这方面的能力还是不错的。

阿里云五轮

阿里云一面

夸克扫描王 极速扫描，就是高效

- HTTPS 的握手过程讲一讲。(讲了很久很细，面试官知道我理解，喊停了)
  - HTTPS 和 HTTP 的缓存有什么区别？(懵了)
- DOM API 掌握怎么样？ (不是很熟)
  - Element 和 Node 的区别(假装想了一会，不知道)
- XSS 攻击 Cookie相关的字段(HttpOnly, 解释了一下作用)
- CSRF 攻击，解释一下 Cookie 的 SameSite 字段 (说了下，觉得可以，过)
  - chrome 最新的 xxx 特性是如何防范 CSRF 攻击 (这个真没听说过)
- fetch 和 xhr 有什么区别？(fetch 不熟)
  - 好，解释一下 xhr 的 cors 过程(简单请求，非简单请求两种情况，说了下)
- React 闭包陷阱有什么好的解决办法吗？(说实话，简历上写了，实际上理解不深，只说了一种)
  - useReducer 可以解决你知道吗？ (当时真的不清楚，主要忘了闭包陷阱的表单场景，尴尬)
- WeakMap 和 Map 的性能有什么差别?(前者对 GC 更加友好，保持弱引用)
  - 如果是在立即执行函数中，两者的性能有区别吗？(楞了一会，说强弱引用还是会有GC的区别，没影响)
    - 换个说法吧，如果这个立即执行函数中有递归函数，两者性能有区别吗？(没 GET 到面试官的点啊，过了吧这题)
- 能不能说说 AMD 和 ESModule 有什么区别？(AMD 不熟，说ESModule，介绍了import、export以及导出引用的特点)
  - 那么你能不能告诉我 ESModule 对于 Tree-Shaking 有什么优势呢？(会做一些编译阶段的优化吧)
- async await 经过编译后和 generator 有啥联系？(问了两遍，还是不知道问的啥，直接说了async await 原理，他说你讲了太深，问的不是这个，过吧过吧)

阿里云二面

- 首先自我介绍
- 为什么要做你这个开源项目？后台数据哪来的?
- 有 nodejs 相关的开发经验吗？(果断说没有，避免给后面挖坑)
- HTTP 的 GET 和 POST 请求有什么区别？(我说了 4 个区别)
- 说一说 CSRF 和 XSS 攻击？(说了一堆，他说你讲的太细了，是不是最近看过之类的文章，我说没有)
- HTTP 缓存能不能说一下？(强缓存，协商缓存，中间扯到代理了，讲了一堆，他说可以了)
- 你知道 JS 的语言标准是如何制定的吗？(确实不熟，说下去自己查查)
  - 你用过哪些 ES 最新的语法，越新越好
- 你对 babel 了解吗？能不能说说几个 stage 代表什么意思?
- Vue 的响应式对数组是如何处理的？(重写数组方法，手动派发更新)
  - Object 为什么可以自动派发更新?
- 假如我在一个for循环中改变当前组件依赖的数据，改变一万次，会有什么效果？(讲到批量更新和 nextTick 原理，他表示可以)

夸克扫描王 极速扫描，就是高效

- 假如让你设计一个适配 PC、手机和平板的项目，你有哪些布局方案？(首先是vh、vw，然后用淘宝出品的 lib-flexible 库进行 rem 适配，还有一种 flex + px 的适配方式)
- CSS 当中以 @ 开头的属性有哪些？(我说了@media, @keyframes，后来提醒我还有@import，我补充这个是串行加载 CSS)
- 了解过前端当前的发展趋势吗，比如一些新的技术方向。(说了对PWA的看法，为什么会凉，flutter、electron、微前端，serverless)
- 了解过云计算吗？能不能讲一讲云计算的发展方向和前景？(没了解过)
- 有什么兴趣爱好？
- 对自己的职业规划是怎样的？(说了一大堆，他笑了笑，说今天就到这里吧，明天笔试)

阿里云三面

- 自我介绍
- 为什么选择前端？(说了很多，他叫我用一句话总结)
- 你的职业规划是怎样的？
- 项目中遇到了哪些挑战？
- 说说你对前端架构的认识，如何设计出一个架构方案(说实话，我顶不住，没研究过)
- 在一个大型项目中，如何定位发生内存泄露的代码？(没有实践过)
- Last-Modified 和 Etag 有什么区别？
  - Cache-Control 和 Last-Modified 的区别
- 跨域有哪些方案？
- React 受控组件和非受控组件的区别

阿里云四面

- 自我介绍
- 你自己在社区做过什么具有推动性的事情？
- 你能不能说说自己比较擅长的数据结构有哪些？
- 你希望公司的环境是怎么样的？
- 为什么不投腾讯？(阿里面的已经很累了，秋招再投吧)
- 阿里云和抖音你更想去哪里？
- 你还有什么问题？

深圳某做海外加速器公司

夸克扫描王
极速扫描，就是高效

1. DIV+CSS布局的好处

- 代码精简，且结构与样式分离，易于维护
- 代码量减少了，减少了大量的带宽，页面加载的也更快，提升了用户的体验
- 对SEO搜索引擎更加友好，且H5又新增了许多语义化标签更是如此
- 允许更多炫酷的页面效果，丰富了页面
- 符合W3C标准，保证网站不会因为网络应用的升级而被淘汰

缺点:

> 不同浏览器对web标准默认值不同，所以更容易出现对浏览器的兼容性问题。

2. 如何解决a标点击后hover事件失效的问题?

- 改变a标签css属性的排列顺序
- 只需要记住LoVe HAte原则就可以了：

```
link→visited→hover→active
```

比如下面错误的代码顺序：

```
a:hover{
  color: green;
  text-decoration: none;
}
a:visited{ /* visited在hover后面，这样的话hover事件就失效了 */
  color: red;
  text-decoration: none;
}
```

正确的做法是将两个事件的位置调整一下。

注意⚠️各个阶段的含义：

> a:link：未访问时的样式，一般省略成a a:visited：已经访问后的样式 a:hover：鼠标移上去时的样式 a:active：鼠标按下时的样式

夸克扫描王 极速扫描，就是高效

3. 点击一个input依次触发的事件

```
const text = document.getElementById('text');
text.onclick = function (e) {
  console.log('onclick')
}
text.onfocus = function (e) {
  console.log('onfocus')
}
text.onmousedown = function (e) {
  console.log('onmousedown')
}
text.onmouseenter = function (e) {
  console.log('onmouseenter')
}
```

答案：

```
'onmouseenter'
'onmousedown'
'onfocus'
'onclick'
```

4. 响应式的好处

对某些数据的修改就能自动更新视图，让开发者不用再去操作DOM，有更多的时间去思考业务逻辑。

5. Vue的优点及缺点

- 首先Vue最核心的两个特点，响应式和组件化。
- 响应式：这也就是vue.js最大的优点，通过MVVM思想实现数据的双向绑定，通过虚拟DOM让我们可以用数据来操作DOM，而不必去操作真实的DOM，提升了性能。且让开发者有更多的时间去思考业务逻辑。
- 组件化：把一个单页应用中的各个模块拆分到一个个组件当中，或者把一些公共的部分抽离出来做成一个可复用的组件。所以组件化带来的好处就是，提高了开发效率，方便重复使用，使项目的可维护性更强。
- 虚拟DOM，当然，这个不是vue中独有的。

夸克扫描王 极速扫描，就是高效

- 缺点：基于对象配置文件的写法，也就是options写法，开发时不利于对一个属性的查找。另外一些缺点，在小项目中感觉不太出什么，vuex的魔法字符串，对ts的支持。兼容性上存在一些问题

6. Vue中hash模式和history模式的区别

- 最明显的是在显示上，hash模式的URL中会夹杂着#号，而history没有。
- Vue底层对它们的实现方式不同。hash模式是依靠onhashchange事件(监听location.hash的改变)，而history模式是主要是依靠的HTML5 history中新增的两个方法，pushState()可以改变url地址且不会发送请求，replaceState()可以读取历史记录栈,还可以对浏览器记录进行修改。
- 当真正需要通过URL向后端发送HTTP请求的时候，比如常见的用户手动输入URL后回车，或者是刷新(重启)浏览器，这时候history模式需要后端的支持。因为history模式下，前端的URL必须和实际向后端发送请求的URL一致，例如有一个URL是带有- 路径path的(例如www.lindaidai.wang/blogs/id)，如果后端没有对这个路径做处理的话，就会返回404错误。所以需要后端增加一个覆盖所有情况的候选资源，一般会配合前端给出的一个404页面

hash:

```js
window.onhashchange = function(event){
  // location.hash获取到的是包括#号的，如"#heading-3"
  // 所以可以截取一下
  let hash = location.hash.slice(1);
}
```

7. react的实现原理？有什么优缺点?
8. react的控制组件和非控制组件

深圳某电商公司

1. null和undefined

- null表示一个"无"的对象，也就是该处不应该有值；而undefined表示未定义。
- 在转换为数字时结果不同，Number(null)为0，而undefined为NaN。

使用场景上：

> null：

- 作为函数的参数，表示该函数的参数不是对象
- 作为对象原型链的终点

夸克扫描王 极速扫描，就是高效

> undefined:

- 变量被声明了，但没有赋值时，就等于undefined
- 调用函数时，应该提供的参数没有提供，该参数等于undefined
- 对象没有赋值属性，该属性的值为undefined
- 函数没有返回值时，默认返回undefined

2. 冒泡排序算法和数组去重

冒泡排序：

```js
function bubbleSort (arr) {
  for (let i = 0; i < arr.length; i++) {
    let flag = true;
    for (let j = 0; j < arr.length - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        flag = false;
        let temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
      }
    }
    if (flag) break;
  }
  return arr;
}
```

> 这个是优化过后的冒泡排序。用了一个flag来优化，它的意思是：如果某一次循环中没有交换过元素，那么意味着排序已经完成了。

> 冒泡排序总会执行(N-1)+(N-2)+(N-3)+..+2+1趟，但如果运行到当中某一趟时排序已经完成，或者输入的是一个有序数组，那么后边的比较就都是多余的，为了避免这种情况，我们增加一个flag，判断排序是否在中途就已经完成（也就是判断有无发生元素交换）

数组去重：

- Array.from(new Set(arr))
- [...new Set(arr)]
- for循环嵌套，利用splice去重
- 新建数组，利用indexOf或者includes去重
- 先用sort排序，然后用一个指针从第0位开始，配合while循环去重

> 当然还有很多，例如用filter、reduce、Map、Object等，具体可以看：

夸克扫描王 极速扫描，就是高效

- JavaScript数组去重(12种方法)

> Array.from(new Set(arr))或[...new Set(arr)]

```
var arr = [1,1,2,5,6,3,5,5,6,8,9,8];
console.log(Array.from(new Set(arr)))
// console.log([...new Set(arr)])
```

for循环嵌套，利用splice去重：

```
function unique (origin) {
  let arr = [].concat(origin);
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] == arr[j]) {
        arr.splice(j, 1);
        j--;
      }
    }
  }
  return arr;
}
var arr = [1,1,2,5,6,3,5,5,6,8,9,8];
console.log(unique(arr))
```

新建数组，利用includes去重:

```
function unique (arr) {
  let res = []
  for (let i = 0; i < arr.length; i++) {
    if (!res.includes(arr[i])) {
      res.push(arr[i])
    }
  }
  return res;
}
var arr = [1,1,2,5,6,3,5,5,6,8,9,8];
console.log(unique(arr))
```

先用sort排序，然后用一个指针从第0位开始，配合while循环去重：

夸克扫描王 极速扫描，就是高效

```js
function unique (arr) {
  arr = arr.sort(); // 排序之后的数组
  let pointer = 0;
  while (arr[pointer]) {
    if (arr[pointer] != arr[pointer + 1]) { // 若这一项和下一项不相等则指针往下移
      pointer++;
    } else { // 否则删除下一项
      arr.splice(pointer + 1, 1);
    }
  }
  return arr;
}
var arr = [1,1,2,5,6,3,5,5,6,8,9,8];
console.log(unique(arr))
```

深圳某云产品公司

1. 描述一下Promise

这道题我会先大概介绍一下Promise：

> Promise 是一个对象，它代表了一个异步操作的最终完成或者失败。由于它的then方法和catch、finally方法会返回一个新的Promise所以可以允许我们链式调用，解决了传统的回调地狱问题

再说一下then以及catch方法：

- Promise的状态一经改变就不能再改变
- .then和.catch都会返回一个新的Promise
- catch不管被连接到哪里，都能捕获上层未捕捉过的错误
- 在Promise中，返回任意一个非 promise 的值都会被包裹成 promise 对象，例如return 2会被包装为return Promise.resolve(2)。
- Promise 的 .then 或者 .catch 可以被调用多次, 但如果Promise内部的状态一经改变，并且有了一个值，那么后续每次调用.then或者.catch的时候都会直接拿到该值
- .then 或者 .catch 中 return 一个 error 对象并不会抛出错误，所以不会被后续的 .catch 捕获
- .then 或 .catch 返回的值不能是 promise 本身，否则会造成死循环
- .then 或者 .catch 的参数期望是函数，传入非函数则会发生值透传
- .then方法是能接收两个参数的，第一个是处理成功的函数，第二个是处理失败的函数，再某些时候你可以认为catch是.then第二个参数的简便写法
- .finally方法也是返回一个Promise，他在Promise结束的时候，无论结果为resolved还是rejected，都会执行里面的回调函数

夸克扫描王 极速扫描，就是高效

另外也可以说一下finally方法：

- .finally()方法不管Promise对象最后的状态如何都会执行
- .finally()方法的回调函数不接受任何的参数，也就是说你在.finally()函数中是没法知道Promise最终的状态是resolved还是rejected的
- 它最终返回的默认会是一个上一次的Promise对象值，不过如果抛出的是一个异常则返回异常的Promise对象。

最后可以说一下all以及race方法：

- `Promise.all()` 的作用是接收一组异步任务，然后并行执行异步任务，并且在所有异步操作执行完后才执行回调。 .race()的作用也是接收一组异步任务，然后并行执行异步任务，只保留取第一个执行完成的异步操作的结果，其他的方法仍在执行，不过执行结果会被抛弃。
- `Promise.all().then()` 结果中数组的顺序和Promise.all()接收到的数组顺序一致。
- `all` 和 `race` 传入的数组中如果有会抛出异常的异步任务，那么只有最先抛出的错误会被捕获，并且是被then的第二个参数或者后面的catch捕获；但并不会影响数组中其它的异步任务的执行。

2. Promise.all中如果有一个抛出异常了会如何处理

> all和race传入的数组中如果有会抛出异常的异步任务，那么只有最先抛出的错误会被捕获，并且是被then的第二个参数或者后面的catch捕获；但并不会影响数组中其它的异步任务的执行

3. Promise为什么能链式调用

> 由于它的 `then` 方法和 `catch` 、 `finally` 方法会返回一个新的 `Promise` 所以可以允许我们链式调用

4. 描述一下EventLoop的执行过程

- 一开始整个脚本作为一个宏任务执行
- 执行过程中同步代码直接执行，宏任务进入宏任务队列，微任务进入微任务队列
- 当前宏任务执行完出队，检查微任务列表，有则依次执行，直到全部执行完
- 执行浏览器UI线程的渲染工作
- 检查是否有 `Web Worker` 任务，有则执行
- 执行完本轮的宏任务，回到2，依此循环，直到宏任务和微任务队列都为空

5. docoment,window,html,body的层级关系

夸克扫描王 极速扫描，就是高效

```
window > document > html > body
```

- window是BOM的核心对象，它一方面用来获取或设置浏览器的属性和行为，另一方面作为一个全局对象
- document对象是一个跟文档相关的对象，拥有一些操作文档内容的功能。但是地位没有window高
- html元素对象和document元素对象是属于html文档的DOM对象，可以认为就是html源代码中那些标签所化成的对象。他们跟div、select什么对象没有根本区别。

> （我是这样记的，整个浏览器中最大的肯定就是窗口window了，那么进来的我不管你是啥，就算你是document也得给我盘着）

6. addEventListener函数的第三个参数

- 第三个参数涉及到冒泡和捕获，是 `true` 时为捕获，是false则为冒泡。
- 或者是一个对象 `{passive: true}` ，针对的是Safari浏览器，禁止/开启使用滚动的时候要用到。

7. 有写过原生的自定义事件吗

- 使用 `Event`
- 使用 `customEvent` （可以传参数）
- 使用 `document.createEvent('CustomEvent')` 和 `initEvent()`

创建自定义事件

原生自定义事件有三种写法：

1. 使用Event

```
let myEvent = new Event('event_name');
```

2. 使用 `customEvent` （可以传参数）

```
let myEvent = new CustomEvent('event_name', {
    detail: {
        // 将需要传递的参数放到这里
        // 可以在监听的回调函数中获取到：event.detail
    }
})
```


夸克扫描王 极速扫描，就是高效

3. 使用 `document.createEvent('CustomEvent')` 和 `initEvent()`

```js
let myEvent = document.createEvent('CustomEvent');// 注意这里是为'CustomEvent'
myEvent.initEvent(
    // 1. event_name: 事件名称
    // 2. canBubble: 是否冒泡
    // 3. cancelable: 是否可以取消默认行为
)
```

- `createEvent` ：创建一个事件
- `initEvent` ：初始化一个事件

可以看到， `initEvent` 可以指定3个参数。

事件的监听

自定义事件的监听其实和普通事件的一样，使用 `addEventListener` 来监听：

```
button.addEventListener('event_name', function (e) {})
```

事件的触发

触发自定义事件使用 `dispatchEvent(myEvent)` 。

> 注意⚠️，这里的参数是要自定义事件的对象(也就是 `myEvent`)，而不是自定义事件的名称('myEvent')

案例

来看个案例吧：

```js
// 1.
// let myEvent = new Event('myEvent');
// 2.
// let myEvent = new CustomEvent('myEvent', {
//   detail: {
//     name: 'lindaidai'
//   }
// })
// 3.
let myEvent = document.createEvent('CustomEvent');
myEvent.initEvent('myEvent', true, true)
```


夸克扫描王 极速扫描，就是高效

```
let btn = document.getElementsByTagName('button')[0]
btn.addEventListener('myEvent', function (e) {
  console.log(e)
  console.log(e.detail)
})
setTimeout(() => {
  btn.dispatchEvent(myEvent)
}, 2000)
```

8. 冒泡和捕获的具体过程

> 冒泡指的是：当给某个目标元素绑定了事件之后，这个事件会依次在它的父级元素中被触发(当然前提是这个父级元素也有这个同名称的事件，比如子元素和父元素都绑定了click事件就触发父元素的click)。

捕获则是从上层向下层传递，与冒泡相反。（非常好记，你就想想水底有一个泡泡从下面往上传的，所以是冒泡）

来看看这个例子：

```html
<!-- 会依次执行 button li ul -->
<ul onclick="alert('ul')">
  <li onclick="alert('li')">
    <button onclick="alert('button')">点击</button>
  </li>
</ul>
<script>
  window.addEventListener('click', function (e) {
    alert('window')
  })
  document.addEventListener('click', function (e) {
    alert('document')
  })
</script>
```

- 冒泡结果： `button > li > ul > document > window`
- 捕获结果： `window > document > ul > li > button`

9. 所有的事件都有冒泡吗?

并不是所有的事件都有冒泡的，例如以下事件就没有：

- `onblur`

夸克扫描王 极速扫描，就是高效

- onfocus
- onmouseenter
- onmouseleave

11. 手写new

```js
function myNew (fn, ...args) {
  let instance = Object.create(fn.prototype);
  let result = fn.call(instance, ...args)
  return typeof result === 'object' ? result : instance;
}
```

12. typeof和instanceof的区别

- typeof表示是对某个变量类型的检测，基本数据类型除了null都能正常的显示为对应的类型，引用类型除了函数会显示为'function'，其它都显示为object。
- 而instanceof它主要是用于检测某个构造函数的原型对象在不在某个对象的原型链上。

13. typeof为什么对null错误的显示

> 这只是 JS 存在的一个悠久 Bug。在 JS 的最初版本中使用的是 32 位系统，为了性能考虑使用低位存储变量的类型信息，000 开头代表是对象然而 null 表示为全零，所以将它错误的判断为 object 。

14. 详细说下instanceof

> instanceof它主要是用于检测某个构造函数的原型对象在不在某个对象的原型链上。

```js
function myInstanceof (left, right) {
  let proto = Object.getPrototypeOf(left);
  while (true) {
    if (proto === null) return false;
    if (proto === right.prototype) return true;
    proto = Object.getPrototypeOf(proto)
  }
}
```


夸克扫描王 极速扫描，就是高效

15. 一句话描述一下this

> 对于函数而言指向最后调用函数的那个对象，是函数运行时内部自动生成的一个内部对象，只能在函数内部使用；对于全局来说，this指向window。

16. 函数内的this是在什么时候确定的?

> 函数调用时，指向最后调用的那个对象

17. webpack中的loader和plugin有什么区别

- loader它是一个转换器，只专注于转换文件这一个领域，完成压缩、打包、语言编译，它仅仅是为了打包。并且运行在打包之前。
- 而plugin是一个扩展器，它丰富了webpack本身，为其进行一些其它功能的扩展。它不局限于打包，资源的加载，还有其它的功能。所以它是在整个编译周期都起作用。

18. HTTP和TCP的不同

- HTTP的责任是去定义数据，在两台计算机相互传递信息时，HTTP规定了每段数据以什么形式表达才是能够被另外一台计算机理解。
- 而TCP所要规定的是数据应该怎么传输才能稳定且高效的传递与计算机之间。

(还可以再扩展)

19. TCP和UDP的区别

- `TCP` 是一个面向连接的、可靠的、基于字节流的传输层协议。
- `UDP` 是一个面向无连接的传输层协议。
- `TCP` 为什么可靠，是因为它有三次握手来保证双方都有接受和发送数据的能力。

> 字节流服务：将大块数据分割为以报文段为单位的数据包进行管理

20. 介绍一下虚拟DOM

- 虚拟DOM本质就是用一个原生的JavaScript对象去描述一个DOM节点。是对真实DOM的一层抽象。

夸克扫描王 极速扫描，就是高效

- 由于在浏览器中操作DOM是很昂贵的。频繁的操作DOM，会产生一定的性能问题，因此我们需要这一层抽象，在patch过程中尽可能地一次性将差异更新到DOM中，这样保证了DOM不会出现性能很差的情况。
- 另外还有很重要的一点，也是它的设计初衷，为了更好的跨平tai，比如Node.js就没有DOM,如果想实现SSR(服务端渲染),那么一个方式就是借助Virtual DOM,因为Virtual DOM本身是JavaScript对象。
- `Vue2.x` 中的虚拟DOM主要是借鉴了snabbdom.js，Vue3中借鉴inferno.js算法进行优化

深圳某房地产公司

5道笔试题

夸克扫描王
极速扫描，就是高效

姓名：[redacted] 手机号：[redacted] 日期：[redacted]

1. 已知 ES5 中 `func.bind(context, 1, 2)(3, 4)` 等价于 `func.call(context, 1, 2, 3, 4)`，请用 ES3 实现一个 `bind` 的 polyfill

```
Function.prototype.bind = function () { ... }
```

2. 请用 ES5 实现 Animal 和 Cat，使其尽可能接近如下 TypeScript 代码的效果

```
class Animal {
  private name: string = ''

  public static getUltimateAnswer(): number {
    return 42
  }

  public getName(): string {
    return this.name
  }

  public setName(name: string): void {
    this.name = name
  }
}

class Cat extends Animal {
  public getName(): string {
    return '[Cat] ' + super.getName()
  }
}
```

3. 已知 Counter 实现如下，请写出执行 `new Counter()` 后控制台的前十条输出，以及执行 `Counter()` 后控制台的前十条输出（被执行的代码分别在非严格模式的独立 JS 环境中运行）

```
function Counter() {
  var start = Date.now()
  this.num = 0

  this.timer1 = setInterval(function () {
    this.num++
    var gap = Date.now() - start
    console.log('timer1', this.num, gap)
  }, 996)

  JSON.parse('{"desc":"..."}')
  // 解析耗时 1024 毫秒

  this.timer2 = setTimeout(() => {
    this.num++
    var gap = Date.now() - start
    console.log('timer2', this.num, gap)
  }, 0)
}
```

4. [Web] 已知一个 JSONP 请求发送函数的定义与用法如下，请实现这个函数，并满足每秒百万并发也不会出错

```
function sendJsonp(params) { ... }

sendJsonp({
  url: 'http://foo.com/api/v1/bar',
  callback: (err, data) => {
    console.log('Callback', err, data)
  },
  timeout: 100000, // 单位是毫秒
})
```

5. [Node] 已知一个 CORS 中间件的用法如下，请实现这个中间件

```
router.use(cors({
  origin: [ 'http://foo.com' ],
  methods: [ 'GET', 'HEAD' ]
  headers: [ 'Authorization' ],
}))
```

并详细说一下

- 然后第三题的关键一个考到了定时器里调用函数的this指向问题，还有就是定时器它的执行机制以及eventLoop，再还有就是new和不new时this.num的计算。

夸克扫描王 极速扫描，就是高效

- 第四题当时面试官说其实一部分是考察我对JSONP的了解，至于百万并发我写不出来可以理解。

1. JSONP的原理以及手写一个实现

> 基本原理：主要就是利用 script 标签的src属性没有跨域的限制，通过指向一个需要访问的地址，由服务端返回一个预先定义好的 Javascript 函数的调用，并且将服务器数据以该函数参数的形式传递过来，此方法需要前后端配合完成。

执行过程：

- 前端定义一个解析函数(如: `jsonpCallback = function (res) {}`)
- 通过params的形式包装script标签的请求参数，并且声明执行函数(如cb=jsonpCallback)
- 后端获取到前端声明的执行函数(jsonpCallback)，并以带上参数且调用执行函数的方式传递给前端
- 前端在script标签返回资源的时候就会去执行jsonpCallback并通过回调函数的方式拿到数据了。

缺点：

> 只能进行GET请求

优点：

> 兼容性好，在一些古老的浏览器中都可以运行

```html
<script>
    function JSONP({
        url,
        params = {},
        callbackKey = 'cb',
        callback
    }) {
        // 定义本地的唯一callbackId，若是没有的话则初始化为1
        JSONP.callbackId = JSONP.callbackId || 1;
        let callbackId = JSONP.callbackId;
        // 把要执行的回调加入到JSON对象中，避免污染window
        JSONP.callbacks = JSONP.callbacks || [];
        JSONP.callbacks[callbackId] = callback;
        // 把这个名称加入到参数中: 'cb=JSONP.callbacks[1]'
        params[callbackKey] = `JSONP.callbacks[${callbackId}]`;
        // 得到'id=1&cb=JSONP.callbacks[1]'
        const paramString = Object.keys(params).map(key => {
            return `${key}=${encodeURIComponent(params[key])}`
        }).join('&')
        // 创建 script 标签
        const script = document.createElement('script');
        script.setAttribute('src', `${url}?${paramString}`);
        document.body.appendChild(script);
        // id自增，保证唯一
```


夸克扫描王 极速扫描，就是高效

```
            JSONP.callbackId++;

        }
        JSONP({
            url: 'http://localhost:8080/api/jsonps',
            params: {
                a: '2&b=3',
                b: '4'
            },
            callbackKey: 'cb',
            callback (res) {
                console.log(res)
            }
        })
        JSONP({
            url: 'http://localhost:8080/api/jsonp',
            params: {
                id: 1
            },
            callbackKey: 'cb',
            callback (res) {
                console.log(res)
            }
        })
    </script>
```

2. 浏览器为什么要跨域？如果是因为安全的话那小程序或者其他的为什么没有跨域？

> 跨域的产生来源于现代浏览器所通用的同源策略，所谓同源策略，是指只有在地址的：

- 协议名
- 域名
- 端口名

> 均一样的情况下，才允许访问相同的cookie、localStorage，以及访问页面的DOM或是发送Ajax请求。若在不同源的情况下访问，就称为跨域。

例如以下为同源：

```
http://www.example.com:8080/index.html
http://www.example.com:8080/home.html
```

以下为跨域：

夸克扫描王 极速扫描，就是高效

```
http://www.example.com:8080/index.html
http://www3.example.com:8080/index.html
```

> 但是有两种情况：http默认的端口号为80，https默认端口号为443。

```
http://www.example.com:80 === http://www.example.com
https://www.example.com:443 === https://www.example.com
```

为什么浏览器会禁止跨域?

简答:

- 首先，跨域只存在于浏览器端，因为我们知道浏览器的形态是很开放的，所以我们需要对它有所限制。
- 其次，同源策略主要是为了保证用户信息的安全，可分为两种：Ajax同源策略和DOM同源策略。
- Ajax同源策略主要是使得不同源的页面不能获取cookie且不能发起Ajax请求，这样在一定层度上防止了CSRF攻击。
- DOM同源策略也一样，它限制了不同源页面不能获取DOM，这样可以防止一些恶意网站在自己的网站中利用iframe嵌入正gui的网站并迷惑用户，以此来达到窃取用户信息。

深答:

- 首先，跨域只存在于浏览器端。浏览器它为web提供了访问入口，并且访问的方式很简单，在地址栏输入要访问的地址或者点击某个链接就可以了，正是这种开放的形态，所以我们需要对它有所限制。

> 所以同源策略它的产生主要是为了保证用户信息的安全，防止恶意的网站窃取数据。分为两种：Ajax同源策略与DOM同源策略：

- Ajax同源策略它主要做了这两种限制：1.不同源页面不能获取cookie；2.不同源页面不能发起Ajax请求。我认为它是防止CSRF攻击的一种方式吧。因为我们知道cookie这个东西它主要是为了解决浏览器与服务器会话状态的问题，它本质上是存储在浏览器或本地文件中一个小小的文本文件，那么它里面一般都会存储了用户的一些信息，包括隐私信息。如果没有Ajax同源策略，恶意网站只需要一段脚本就可以获取你的cookie，从而冒充你的身份去给其它网站发送恶意的请求。
- DOM同源策略也一样，它限制了不同源页面不能获取DOM。例如一个假的网站利用iframe嵌套了一个银行网站mybank.com，并把宽高或者其它部分调整的和原银行网站一样，仅仅只是地址栏上的域名不同，若是用户没有注意的话就以为这个是个真的网站。如果这时候用户在里面输入了账号密码，如果没有同源策略，那么这个恶意网站就可以获取到银行网站中的DOM，也就能拿到用户的输入内容以此来达到窃取用户信息的攻击

> 同源策略它算是浏览器安全的第一层屏障吧，因为就像 `CSRF` 攻击，它只能限制不同源页面cookie的获取，但是攻击者还可能通过其它的方式来达到攻击效果

夸克扫描王 极速扫描，就是高效

（注，上面提到的iframe限制DOM查询，案例如下）

```
// HTML
<iframe name="yinhang" src="www.yinhang.com"></iframe>
// JS
// 由于没有同源策略的限制，钓鱼网站可以直接拿到别的网站的Dom
const iframe = window.frames['yinhang']
const node = iframe.document.getElementById('你输入账号密码的Input')
console.log(`拿到了这个${node}，我还拿不到你刚刚输入的账号密码吗`)
```

3. CORS跨域的原理

> 跨域资源共享(CORS)是一种机制，是W3C标准。它允许浏览器向跨源服务器，发出XMLHttpRequest或Fetch请求。并且整个CORS通信过程都是浏览器自动完成的，不需要用户参与。

> 而使用这种跨域资源共享的前提是，浏览器必须支持这个功能，并且服务器端也必须同意这种"跨域"请求。因此实现CORS的关键是服务器需要服务器。通常是有以下几个配置：

- `Access-Control-Allow-Origin`
- `Access-Control-Allow-Methods`
- `Access-Control-Allow-Headers`
- `Access-Control-Allow-Credentials`
- `Access-Control-Max-Age`

简单回答：

- 当我们发起跨域请求时，如果是非简单请求，浏览器会帮我们自动触发预检请求，也就是 OPTIONS 请求，用于确认目标资源是否支持跨域。如果是简单请求，则不会触发预检，直接发出正常请求。
- 浏览器会根据服务端响应的 header 自动处理剩余的请求，如果响应支持跨域，则继续发出正常请求，如果不支持，则在控制台显示错误。

详细回答：

- 浏览器先根据同源策略对前端页面和后台交互地址做匹配，若同源，则直接发送数据请求；若不同源，则发送跨域请求。
- 服务器收到浏览器跨域请求后，根据自身配置返回对应文件头。若未配置过任何允许跨域，则文件头里不包含 Access-Control-Allow-origin 字段，若配置过域名，则返回 Access-Control-Allow-origin + 对应配置规则里的域名的方式。
- 浏览器根据接受到的 响应头里的 Access-Control-Allow-origin 字段做匹配，若无该字段，说明不允许跨域，从而抛出一个错误；若有该字段，则对字段内容和当前域名做比对，如果同源，则说明可以跨域，浏览器接受该响应；若不同源，则说明该域名不可跨域，浏览器不接受该响应，并抛出一个错误。

夸克扫描王 极速扫描，就是高效

> - 在CORS中有简单请求和非简单请求，简单请求是不会触发CORS的预检请求的，而非简单请求会。
> - “需预检的请求”要求必须首先使用 OPTIONS 方法发起一个预检请求到服务器，以获知服务器是否允许该实际请求。"预检请求"的使用，可以避免跨域请求对服务器的用户数据产生未预期的影响。

4. 在深圳的网页上输入百度，是怎么把这个请求发到北京的

> 这个当时面试官和我说的是，中间会经过很多的站点，比如会经过湖南，或者其它城市，由各个城市的这些站点一层一层分发下去。

- 5. 输入URL到页面的呈现
- 6. Vue的响应式原理
- 7. 那在这个响应式中一个数据改变它是怎么通知要更新的，也就是如何把数据和页面关联起来?

深圳某海外直播公司

1. CommonJS和ES6模块的区别

- CommonJS模块是运行时加载，ES6 Modules是编译时输出接口
- CommonJS输出是值的拷贝；ES6 Modules输出的是值的引用，被输出模块的内部的改变会影响引用的改变
- CommonJs导入的模块路径可以是一个表达式，因为它使用的是require()方法；而ES6 Modules只能是字符串
- CommonJS this指向当前模块，ES6 Modules this指向undefined
- 且ES6 Modules中没有这些顶层变量：arguments、require、module、exports、__filename、__dirname

> 关于第一个差异，是因为CommonJS 加载的是一个对象（即module.exports属性），该对象只有在脚本运行完才会生成。而 ES6 模块不是对象，它的对外接口只是一种静态定义，在代码静态解析阶段就会生成。

2. 模块的异步加载

> 模块的异步加载可以使用AMD或者CMD规范。

夸克扫描王 极速扫描，就是高效

3. 开发一个模块要考虑哪些问题?

封闭开放式原则、安全性

4. 实现一个一组异步请求按顺序执行你有哪些方法?

- 利用reduce，初始值传入一个Promise.resolve()，之后往里面不停的叠加.then()
- 利用forEach，本质和reduce原理相同
- 还可以用ES9中的for...await...of来实现。

5. Promise.all()是并发的还是串行的?

> 并发的。不过 `Promise.all().then()` 结果中数组的顺序和 `Promise.all()` 接收到的数组顺序一致。

6. 平时写过哪些正则表达式

- 之前有用过用正则去除输入框的首尾空格，正则表达式为： `var trimReg = /(^\s+)|(\s+$)/g` ；不过后来由于Vue中有一个修饰符.trim，使用起来更方便(如 `v-model.trim="msg"`)就用这种方式多一些；再或者也可以用ES10新出的trimStart和trimEnd来去除首尾空格。
- 用于校验手机号的正则： `var phoneReg = /^1[3456789]\d{9}$/g`
- 用正则写一个根据 `name` 获取 `cookie` 中的值的方法：

```js
function getCookie(name) {
  var match = document.cookie.match(new RegExp('(^| )' + name + '=([^;]*)'));
  if (match) return unescape(match[2]);
}
```

7. 正则里的非如何实现的

> `^` 要是放在 `[]` 里的话就表示"除了^后面的内容都能匹配"，也就是非的意思。

例如：

(除了!，其它都变成了"帅")

夸克扫描王 极速扫描，就是高效

```js
var str = 'lindaidai';
console.log(str.replace(/[^l]/g, '帅'));
// l帅帅帅帅帅帅帅帅
```

反之，如果是不在 `[]` 里的话则表示开头匹配：

(只有l变成了"帅")

```js
var str = 'lindaidai';
console.log(str.replace(/^l/g, '帅'));
```

8. webpack几种hash的实现原理

- `hash` 是跟整个项目的构建相关，只要项目里有文件更改，整个项目构建的hash值都会更改，并且全部文件都共用相同的hash值。(粒度整个项目)
- `chunkhash` 是根据不同的入口进行依赖文件解析，构建对应的chunk(模块)，生成对应的hash值。只有被修改的chunk(模块)在重新构建之后才会生成新的hash值，不会影响其它的chunk。(粒度entry的每个入口文件)
- `contenthash` 是跟每个生成的文件有关，每个文件都有一个唯一的hash值。当要构建的文件内容发生改变时，就会生成新的hash值，且该文件的改变并不会影响和它同一个模块下的其它文件。(粒度每个文件的内容)

9. webpack如果使用了hash命名，那是每次都会重写生成hash吗

> 这个问题在上一个问题中已经说明了，要看webpack的配置。

有三种情况：

- 如果是hash的话，是和整个项目有关的，有一处文件发生更改则所有文件的hash值都会发生改变且它们共用一个hash值；
- 如果是chunkhash的话，只和entry的每个入口文件有关，也就是同一个chunk下的文件有所改动该chunk下的文件的hash值就会发生改变
- 如果是contenthash的话，和每个生成的文件有关，只有当要构建的文件内容发生改变时才会给该文件生成新的hash值，并不会影响其它文件。

10. webpack中如何处理图片的?

夸克扫描王 极速扫描，就是高效

> 在webpack中有两种处理图片的loader：

- `file-loader` ：解决CSS等中引入图片的路径问题；(解决通过url,import/require()等引入图片的问题)
- `url-loader` ：当图片小于设置的limit参数值时，url-loader将图片进行base64编码(当项目中有很多图片，通过url-loader进行base64编码后会减少http请求数量，提高性能)，大于limit参数值，则使用file-loader拷贝图片并输出到编译目录中；

11. 说一下回流和重绘

1. 回流：

触发条件：

> 当我们对 DOM 结构的修改引发 DOM 几何尺寸变化的时候，会发生回流的过程。

例如以下操作会触发回流：

- 一个 DOM 元素的几何属性变化，常见的几何属性有width、height、padding、margin、left、top、border 等等, 这个很好理解。
- 使 DOM 节点发生增减或者移动。
- 读写 offset族、scroll族和client族属性的时候，浏览器为了获取这些值，需要进行回流操作。
- 调用 `window.getComputedStyle` 方法。

> 回流过程：由于DOM的结构发生了改变，所以需要从生成DOM这一步开始，重新经过样式计算、生成布局树、建立图层树、再到生成绘制列表以及之后的显示器显示这整一个渲染过程走一遍，开销是非常大的

2. 重绘：

触发条件：

> 当 DOM 的修改导致了样式的变化，并且没有影响几何属性的时候，会导致重绘(repaint)。

重绘过程：由于没有导致 DOM 几何属性的变化，因此元素的位置信息不需要更新，所以当发生重绘的时候，会跳过生存布局树和建立图层树的阶段，直接到生成绘制列表，然后继续进行分块、生成位图等后面一系列操作。

如何避免触发回流和重绘：

- 避免频繁使用 style，而是采用修改class的方式。
- 将动画效果应用到position属性为absolute或fixed的元素上。
- 也可以先为元素设置display: none，操作结束后再把它显示出来。因为在display属性为none的元素上进行的DOM操作不会引发回流和重绘
- 使用createDocumentFragment进行批量的 DOM 操作。
- 对于 resize、scroll 等进行防抖/节流处理。

夸克扫描王 极速扫描，就是高效

- 避免频繁读取会引发回流/重绘的属性，如果确实需要多次使用，就用一个变量缓存起来。
- 利用 CSS3 的transform、opacity、filter这些属性可以实现合成的效果，也就是GPU加速。

12. 盒模型及如何转换

- `box-sizing: content-box` （W3C盒模型，又名标准盒模型）：元素的宽高大小表现为内容的大小。
- `box-sizing: border-box` （IE盒模型，又名怪异盒模型）：元素的宽高表现为内容 + 内边距 + 边框的大小。背景会延伸到边框的外沿。

13. 实现水平垂直居中的几种方式

Flex布局（子元素是块级元素）

```
.box {
  display: flex;
  width: 100px;
  height: 100px;
  background-color: pink;
}

.box-center{
  margin: auto;
  background-color: greenyellow;
}
```

Flex布局

```
.box {
  display: flex;
  width: 100px;
  height: 100px;
  background-color: pink;
  justify-content: center;
  align-items: center;
}

.box-center{
  background-color: greenyellow;
}
```

绝对定位实现(定位元素定宽定高)

夸克扫描王 极速扫描，就是高效

```
.box {
  position: relative;
  height: 100px;
  width: 100px;
  background-color: pink;
}

.box-center{
  position: absolute;
  left: 0;
  right: 0;
  bottom: 0;
  top: 0;
  margin: auto;
  width: 50px;
  height: 50px;
  background-color: greenyellow;
}
```

14. flex的兼容性怎样

简单回答：

> IE6~9不支持，IE10~11部分支持flex的2012版，但是需要-ms-前缀。

其它的主流浏览器包括安卓和IOS基本上都支持了。

详细回答：

- IE10部分支持2012，需要-ms-前缀
- Android4.1/4.2-4.3部分支持2009 ，需要-webkit-前缀
- Safari7/7.1/8部分支持2012， 需要-webkit-前缀
- IOS Safari7.0-7.1/8.1-8.3部分支持2012，需要-webkit-前缀

15. 你知道到哪里查看兼容性吗

> 可以到Can I use上去查看

16. 移动端中css你是使用什么单位

比较常用的：

夸克扫描王 极速扫描，就是高效

- `em` ：定义字体大小时以父级的字体大小为基准；定义长度单位时以当前字体大小为基准。例父级font-size: 14px，则子级font-size: 1em;为font-size: 14px;；若定义长度时，子级的字体大小如果为14px，则子级width: 2em;为width: 24px。
- `rem` ：以根元素的字体大小为基准。例如html的font-size: 14px，则子级1rem = 14px。 %：以父级的宽度为基准。例父级width: 200px，则子级width: 50%;height:50%;为width: 100px;height: 100px;
- `vw` 和 `vh` ：基于视口的宽度和高度(视口不包括浏览器的地址栏工具栏和状态栏)。例如视口宽度为1000px，则60vw = 600px;
- `vmin` 和 `vmax` ：vmin为当前vw 和vh中较小的一个值；vmax为较大的一个值。例如视口宽度375px，视口高度812px，则100vmin = 375px;，100vmax = 812px;

不常用的：

> ex和ch：ex以字符"x"的高度为基准；例如1ex表示和字符"x"一样长。ch以数字"0"的宽度为基准；例如2ch表示和2个数字"0"一样长。

移动端布局总结：

- 移动端布局的方式主要使用rem和flex，可以结合各自的优点，比如flex布局很灵活，但是字体的大小不好控制，我们可以使用rem和媒体查询控制字体的大小，媒体查询视口的大小，然后不同的上视口大小下设置设置html的font-size。
- 可单独制作移动端页面也可响应式pc端移动端共用一个页面。没有好坏，视情况而定，因势利导

17. rem和em的区别

em:

> 定义字体大小时以父级的字体大小为基准；定义长度单位时以当前字体大小为基准。例父级font-size: 14px，则子级font-size: 1em;为font-size: 14px;；若定义长度时，子级的字体大小如果为14px，则子级width: 2em;为width: 24px。

rem:

> 以根元素的字体大小为基准。例如html的font-size: 14px，则子级1rem = 14px。

18. 在移动端中怎样初始化根元素的字体大小

一个简易版的初始化根元素字体大小。

页面开头处引入下面这段代码，用于动态计算 `font-size`：

(假设你需要的 `1rem = 20px`)

夸克扫描王 极速扫描，就是高效

```js
(function () {
  var html = document.documentElement;
  function onWindowResize() {
    html.style.fontSize = html.getBoundingClientRect().width / 20 + 'px';
  }
  window.addEventListener('resize', onWindowResize);
  onWindowResize();
})();
```

- `document.documentElement` ：获取document的根元素
- `html.getBoundingClientRect().width` ：获取html的宽度(窗口的宽度)
- 监听window的resize事件

一般还需要配合一个meta头：

```
<meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-sacle=1.0, maximum-scale=1.0, user-scalable=no" />
```

19. 移动端中不同手机html默认的字体大小都是一样的吗

> 如果没有人为取改变根元素字体大小的话，默认是1rem = 16px；根元素默认的字体大小是16px。

20. 你做过哪些动画效果

实话实说没太做过。

21. 如果让你实现一个一直旋转的动画你会如何做

css代码：

```html
<style>
  .box {
    width: 100px;
    height: 100px;
    background-color: red;
    animation: spin 2s linear infinite;
  }
  @keyframes spin {
```


夸克扫描王 极速扫描，就是高效

```
    from { transform: rotate(0deg) }
    to { transform: rotate(360deg) }
  }
</style>
```

html代码：

```html
<div class="box"></div>
```

22. animation介绍一下

```
animation: name duration timing-function delay iteration-count direction;
```

- animation-name 规定需要绑定到选择器的 keyframe 名称。(mymove)
- animation-duration 规定完成动画所花费的时间，以秒或毫秒计。(2s)
- animation-timing-function 规定动画的速度曲线。(ease|linear|ease-in|cubic-bezier(n,n,n,n))
- animation-delay 规定在动画开始之前的延迟。(2s)
- animation-iteration-count 规定动画应该播放的次数。(n | infinite) n次/无限
- animation-direction 规定是否应该轮流反向播放动画。(normal | alternate) 正常/反向

23. animation有一个steps()功能符知道吗

> 一句话介绍：steps()功能符可以让动画不连续。

- 地位和作用：和贝塞尔曲线(cubic-bezier()修饰符)一样，都可以作为animation的第三个属性值。
- 和贝塞尔曲线的区别：贝塞尔曲线像是滑梯且有4个关键字(参数)，而steps像是楼梯坡道且只有number和position两个关键字。

```
steps(number, position)
```

- `number`：数值，表示把动画分成了多少段
- `position`：表示动画是从时间段的开头连续还是末尾连续。支持start和end两个关键字，含义分别如下：

夸克扫描王 极速扫描，就是高效

- start`：表示直接开始。
- end`：表示戛然而止。是默认值。

24. 用过哪些移动端的调试工具

- Chrome浏览器 -> more tools -> Remote devices -> chrome://inspect/#devices
- Mac + IOS + Safari

25. 说一下原型链

26. 详细说一下instanceof

27. V8的垃圾回收是发生在什么时候?

> V8引擎帮助我们实现了自动的垃圾回收管理，利用浏览器渲染页面的空闲时间进行垃圾回收。

28. 具体说一下垃圾回收机制

栈内存的回收：

> 栈内存调用栈上下文切换后就被回收，比较简单。

堆内存的回收：

> V8的堆内存分为新生代内存和老生代内存，新生代内存是临时分配的内存，存在时间短，老生代内存存在时间长。

新生代内存回收机制：

> 新生代内存容量小，64位系统下仅有32M。新生代内存分为From、To两部分，进行垃圾回收时，先扫描From，将非存活对象回收，将存活对象顺序复制到To中，之后调换From/To，等待下一次回收

老生代内存回收机制

- 晋升：如果新生代的变量经过多次回收依然存在，那么就会被放入老生代内存中
- 标记清除：老生代内存会先遍历所有对象并打上标记，然后对正在使用或被强引用的对象取消标记，回收被标记的对象
- 整理内存碎片：把对象挪到内存的一端

夸克扫描王 极速扫描，就是高效

29. 在项目中如何把http的请求换成https

> 由于我在项目中是会对axios做一层封装，所以每次请求的域名也是写在配置文件中，有一个baseURL字段专门用于存储它，所以只要改这个字段就可以达到替换所有请求http为https了。

当然后面我也有了解到：

> 利用meta标签把http请求换为https:

```
<meta http-equiv ="Content-Security-Policy" content="upgrade-insecure-requests">
```

30. 知道meta标签有把http换成https的功能吗?

参考上一题👆。

31. http请求可以怎么拦截

> 在浏览器和服务器进行传输的时候，可以被nginx代理所拦截，也可以被网关拦截。

32. https的加密方式

- HTTPS主要是采用对称密钥加密和非对称密钥加密组合而成的混合加密机制进行传输。
- 也就是发送密文的一方用"对方的公钥"进行加密处理"对称的密钥"，然后对方在收到之后使用自己的私钥进行解密得到"对称的密钥"，这在确保双发交换的密钥是安全的前提下使用对称密钥方式进行通信。

33. 混合加密的好处

- 对称密钥加密和非对称密钥加密都有它们各种的优缺点，而混合加密机制就是将两者结合利用它们各自的优点来进行加密传输。
- 比如既然对称密钥的优点是加解密效率快，那么在客户端与服务端确定了连接之后就可以用它来进行加密传输。不过前提是得解决双方都能安全的拿到这把对称密钥。这时候就可以利用非对称密钥加密来传输这把对称密钥，因为我们知道非对称密钥加密的优点就是能保证传输的内容是安全的。
- 所以它的好处是即保证了对称密钥能在双方之间安全的传输，又能使用对称加密方式进行通信，这比单纯的使用非对称加密通信快了很多。以此来解决了HTTP中内容可能被窃听的问题

夸克扫描王 极速扫描，就是高效

其它HTTP相关的问题：

如：

- HTTPS的工作流程
- 混合加密机制的好处
- 数字签名
- ECDHE握手和RSA握手
- 向前安全性

34. 浏览器如何验证服务器的身份

- 这道题主要可以从数字签名和数字证书上来答。
- 当时我答的时候就把证书的颁发流程、HTTPS数字证书的验证过程完整的说了一遍就算过了。

35. ETag首部字段说一下

> 这个无非就是配合If-None-Match来达到一个协商缓存的作用。值为服务器某个资源的唯一标识。

36. 你们的token一般是存放在哪里的

> Token其实就是访问资源的凭证。

一般是用户通过用户名和密码登录成功之后，服务器将登陆凭证做数字签名，加密之后得到的字符串作为token。 它在用户登录成功之后会返回给客户端，客户端主要有这么几种存储方式：

- 存储在 `localStorage` 中，每次调用接口的时候都把它当成一个字段传给后台
- 存储在 `cookie` 中，让它自动发送，不过缺点就是不能跨域
- 拿到之后存储在 `localStorage` 中，每次调用接口的时候放在HTTP请求头的 `Authorization` 字段里

37. token会不会被伪造?

38. redis中一般用来存什么

用户登录成功之后的一些信息

39. 前后端如何验证一个用户是否下线了

夸克扫描王 极速扫描，就是高效

40. CSP白名单知道吗?

41. nginx有配置过吗?

只会配置一些跨域方面的问题。

```
events {
    worker_connections  1024;
}
http {
    include       mime.types;
    default_type  application/octet-stream;
    sendfile        on;
    keepalive_timeout  65;
    server {
        listen       80;
        server_name  localhost;
        location / {
            proxy_pass  http://localhost:8887;
            add_header  Access-Control-Allow-Origin *;
        }
    }
}
```

> 利用ngnix跨域的关键就是在配置文件中设置server项，然后设置其中的location属性，proxy_pass：需要代理的服务器地址， `add_header` ：给响应报文中添加首部字段，例如 `Access-Control-Allow-Origin` 设置为 `*` ，即允许所有的请求源请求

42. 反向代理知道吗?

> 我们将请求发送到服务器，然后服务器对我们的请求进行转发，我们只需要和代理服务器进行通信就好。所以对于客户端来说，是感知不到服务器的。

43. 有用过抓包工具吗?

44. 你平常用的电脑是Mac吗?

夸克扫描王 极速扫描，就是高效

(...默默的点头)

45. Fiddler有用过吗?

唔...知道...

46. Vue的diff算法

47. Vue中computed和methods的区别

48. 例如要获取当前时间你会放到computed还是methods里?

> 放在methods中，因为computed会有惰性，并不能知道new Date()的改变。

49. 你们的权限功能是怎么做的?

50. 那你在判断权限的时候是用的字符串匹配还是位运算?

> 和面试官扯了一堆我数据权限判断的具体过程，其中可能有多个权限：并的情况000011110001&000011110002，或的情况000011110001|000011110002，以及如何做的权限匹配。
> 最后面试官：

"所以那还是用的字符串匹配咯？" 尬...我比较low...用的字符串匹配... （哇，真的绝了...1个小时20分钟50多道题，答的我口渴😂，不过也可以看出有很多移动端的我都没有答上来，面试官也表示理解，毕竟我主要是以PC端为主，所以竟然也算是过了，很感谢这位面试官细心的帮我分析一些问题）

深圳某国内直播公司

1. 输入URL到页面呈现

(必问...)

2. 为什么说script标签建议放在body下面?

夸克扫描王 极速扫描，就是高效

> JS代码在加载完之后是立即执行的，且JS代码执行时会阻塞页面的渲染。

3. 为什么说script标签会阻塞页面的渲染呢？渲染线程和js引擎线程不是分开的吗？

> JS属于单线程，当我们在加载script标签内容的时候，渲染线程会被暂停，因为script标签里可能会操作DOM的，所以如果你加载script标签又同时渲染页面肯定就冲突了，因此说渲染线程(GUI)和js引擎线程互斥。

4. 协商缓存说一下

> Last-Modefied配合If-Modified-Since

- `ETag` 配合 `If-None-Match`

(当时面试官还重复了一下我说的这4个头部字段，自己回顾了一下我说的对不对，好可爱～)

5. HTTP中的Keep-Alive有了解过吗？

- Keep-Alive是HTTP的一个头部字段Connection中的一个值，它是保证我们的HTTP请求能建立一个持久连接。也就是说建立一次TCP连接即可进行多次请求和响应的交互。它的特点就是只要有一方没有明确的提出断开连接，则保持TCP连接状态，减少了TCP连接和断开造成的额外开销。
- 另外，在HTTP/1.1中所有的连接默认都是持久连接的，但是HTTP/1.0并未标准化。

6. 跨域有了解吗？如何解决跨域？

- 我工作中碰到主要是利用CORS来解决跨域问题，说了一下它的原理以及后台需要如何做。
- 另外说到了JSONP的原理，以及它的优点：兼容性好；缺点：只能进行GET请求，且有安全问题。
- 还有说到了ngnix反向代理来解决跨域。

7. WebSocket有了解过吗？它也可以跨域的

这个当时答的没用过。

我知道它是能使得客户端和服务器之间存在持久的连接，而且双方都可以随时开始发送数据，这种方式本质没有使用 HTTP 的响应头，因此也没有跨域的限制。

夸克扫描王
极速扫描，就是高效

8. 前端安全方面？XSS？CSRF？

(必问...)

XSS

> XSS(Cross Site Script)跨站脚本攻击。指的是攻击者向网页注入恶意的客户端代码，通过恶意的脚本对客户端网页进行篡改，从而在用户浏览网页时，对用户浏览器进行控制或者获取用户隐私数据的一种攻击方式。

主要是分为三种：

- **存储型**：即攻击被存储在服务端，常见的是在评论区插入攻击脚本，如果脚本被储存到服务端，那么所有看见对应评论的用户都会受到攻击。
- **反射型**：攻击者将脚本混在URL里，服务端接收到URL将恶意代码当做参数取出并拼接在HTML里返回，浏览器解析此HTML后即执行恶意代码
- **DOM型**：将攻击脚本写在URL中，诱导用户点击该URL，如果URL被解析，那么攻击脚本就会被运行。和前两者的差别主要在于DOM型攻击不经过服务端

如何防御XSS攻击

- 输入检查：对输入内容中的script和 `<iframe>` 等标签进行转义或者过滤
- 设置httpOnly：很多XSS攻击目标都是窃取用户cookie伪造身份认证，设置此属性可防止JS获取cookie
- 开启CSP，即开启白名单，可阻止白名单以外的资源加载和运行

CSRF

> CSRF攻击(Cross-site request forgery)跨站请求伪造。是一种劫持受信任用户向服务器发送非预期请求的攻击方式，通常情况下，它是攻击者借助受害者的 Cookie 骗取服务器的信任，但是它并不能拿到Cookie，也看不到Cookie的内容，它能做的就是给服务器发送请求，然后执行请求中所描述的命令，以此来改变服务器中的数据，也就是并不能窃取服务器中的数据。

防御主要有三种：

- 验证Token：浏览器请求服务器时，服务器返回一个token，每个请求都需要同时带上token和cookie才会被认为是合法请求
- 验证Referer：通过验证请求头的Referer来验证来源站点，但请求头很容易伪造
- 设置SameSite：设置cookie的SameSite，可以让cookie不随跨站请求发出，但浏览器兼容不一

点击挟持

- 诱使用户点击看似无害的按钮（实则点击了透明 iframe 中的按钮）.
- 监听鼠标移动事件，让危险按钮始终在鼠标下方.
- 使用 HTML5 拖拽技术执行敏感操作（例如 deploy key）.

夸克扫描王 极速扫描，就是高效

预防策略：

- 服务端添加 `X-Frame-Options` 响应头,这个 HTTP 响应头是为了防御用 iframe 嵌套的点击劫持攻击。这样浏览器就会阻止嵌入网页的渲染。
- JS 判断顶层视口的域名是不是和本页面的域名一致，不一致则不允许操作， `top.location.hostname == self.location.hostname` ；
- 敏感操作使用更复杂的步骤（验证码、输入项目名称以删除）。

9. setTimeout的执行原理(EventLoop)

(必问...)

> setTimeout的运行机制：执行该语句时，是立即把当前定时器代码推入事件队列，当定时器在事件列表中满足设置的时间值时将传入的函数加入任务队列，之后的执行就交给任务队列负责。但是如果此时任务队列不为空，则需等待，所以执行定时器内代码的时间可能会大于设置的时间

说了一下它属于异步任务，然后说了一下还有哪些宏任务以及微任务，最后说了一下EventLoop的执行过程。

- 一开始整个脚本作为一个宏任务执行
- 执行过程中同步代码直接执行，宏任务进入宏任务队列，微任务进入微任务队列
- 当前宏任务执行完出队，检查微任务列表，有则依次执行，直到全部执行完
- 执行浏览器UI线程的渲染工作
- 检查是否有Web Worker任务，有则执行
- 执行完本轮的宏任务，回到2，依此循环，直到宏任务和微任务队列都为空

10. requestAnimationFrame有了解过吗?

(啪啪啪，不长记性，其实之前面试有被问过，但是忘了再去了解了，这就吃亏了，没答上来)

> requestAnimationFrame是浏览器用于定时循环操作的一个接口，类似于setTimeout，主要用途是按帧对网页进行重绘。对于JS动画，用requestAnimationFrame 会比 setInterval 效果更好。

11. requestAnimationFrame和setTimeout的区别?

同上...

12. 平常工作中ES6+主要用到了哪些?

(下面看着很多，但我肯定不是全答哈，挑了几个来回答)

夸克扫描王 极速扫描，就是高效

ES6:

- Class
- 模块import和export
- 箭头函数
- 函数默认参数
- ... 扩展运输符允许展开数组
- 解构
- 字符串模版
- Promise
- let const
- Proxy、Map、Set
- 对象属性同名能简写

ES7:

- includes
- ** 求幂运算符

ES8:

- async/await
- Object.values() 和 Object.entries()
- padStart() 和padEnd()`
- Object.getOwnPropertyDescriptors()
- 函数参数允许尾部 ,

ES9:

- for...await...of
- ... 展开符合允许展开对象收集剩余参数
- Promise.finally()
- 正则中的四个新功能

ES10:

- flat()
- flatMap()
- fromEntries()
- trimStart 和 trimEnd
- matchAll
- BigInt

夸克扫描王 极速扫描，就是高效

- `try/catch` 中报错允许没有 `err` 异常参数
- `Symbol.prototype.description`
- `Function.toString()` 调用时呈现原本源码的样子

13. 如何在前端实现一个图片压缩

> 实话实话没做过，但是后来面试官告诉我：可以使用canvas来实现。具体做法等我写篇文章哈。

14. 你上家公司主要是做什么的?

15. 团队多少人呢?

1个前端(我)，1个小程序老哥(IOS转行的)，6个后台。

16. 项目中有碰到什么难的问题吗？如何解决的?

例举了我最经典的bpmn.js，以此来引出我写了很多关于这方面的教材，以及建立了微信群，为国内的bpmn.js社区贡献了一份力量...怎么高大上怎么来...

当然也有提到我GitHub上的bpmn-chinse-document项目只有100多的Star，他说理解，毕竟这东西用的人不是很多。

17. 期望薪资多少?

喊了个挺高的数，老哥笑了笑，你这个工作年限我们可能给不到，然后扯了点别的。

18. 还有什么想要问我的吗?

- 团队人员分布情况
- 技术栈
- 我进去主要是负责哪块内容
- 年终奖/季度奖
- 调薪的频率以及幅度
- 加班多不多

夸克扫描王
极速扫描，就是高效

明源云（一面）

css 选择器的排序

css 相同等级的样式匹配的顺序

比如

```html
<style>
  .red {
    color: red;
  }

  .black {
    color: black;
  }
</style>
<div class="red black">

</div>
```

webpack 打包出来的 html 为什么 style 放在头部，script 放在底部

某些脚本资源无法放在底部，如何解决

defer 和 async 的区别

webpack 配置如何实现开发环境不使用 cdn、生产环境使用 cdn

你在项目中做过的首屏加载的优化

夸克扫描王
极速扫描，就是高效

懒加载的实现

防抖节流的概念

你们对于 webpack 如何配置

react 的一些优化

迁移 ts 的理由

hooks 的收益

graphql 对比 restful

算法优化

```js
// n 位数的数组
// 取 i, j
// 求n[i] 到 n[j] 的和
// n 足够大
// 且会频繁调用算法
```

mobx 对比 redux

迁移 mobx 的原因

夸克扫描王
极速扫描，就是高效

- 针对自己简历上项目的讲解

明源云（二面）

- 介绍项目
- 介绍一下为什么用 graphql
- 为什么要使用语雀来做富文本编辑器
- 如何处理语雀的防盗链
- 介绍一下公司的组件库
- 介绍一下迁移 ts 的过程
- 介绍一下 ts 的 eslint
- 为什么需要从 redux 迁移到 mobx
- 什么情况下需要用到状态库
- 业务开发中如何进行组件的拆分
- 说一下和后端接口交互那一块，在开发中如何进行规划

夸克扫描王
极速扫描，就是高效

写组件的时候使用类组件还是函数组件更多，为什么

聊一下笔试题中 LazyMan 的实现思路

离职之后，你在做什么，在思考什么，有没有关注前端新闻，职业规划

cvte（一面）

自我介绍

在维护组件库的过程中，有什么难点

说一下原型链

继承的优缺点

说一下输入 url 到页面渲染的全过程

首屏优化

ts 的编译原理

实现一个 ts 工具函数，获取 Promise 中的泛型

```ts
type A = Promise<string>
type UnGenericPromise<T extends Promise<any>> = T extends Promise<infer U> ? U : never
```


夸克扫描王 极速扫描，就是高效

实现一个工具函数，用来提取 promise 的泛型类型

React 中性能优化的做法

PureComponent 中如何对比两个对象

node 的事件循环

中信银行信用卡中心（一面）

自我介绍

介绍一下你们组件库

介绍一下redux迁移到mobx的目的

官网重构中使用了graphql，这是怎么回事?

hashMap 和 Object 的区别

解释一下 React 中元素和组件的区别

在 React 中如何销毁页面里面的定时器

介绍一下 React 中的 Fiber 架构

夸克扫描王
极速扫描，就是高效

React 如何知道下一次执行该从哪个节点开始重新执行

它使用了那种遍历方式遍历树

介绍一下 webpack4 中的 tree-shaking

介绍一下它的工作流程

CSS 可以做摇树吗

实践题

一个信息流的网站，网站中有很多图片，如果加载图片的时候，用户在进入一些网络状态比较差的情况下，如何用一张默认图片进行替换。

后续发现是 CDN 有问题，图片根本加载不出来，有没有其他办法绘制一个图形替换

实践题

脚手架中希望有一个功能，能够捕捉到整个项目中的异常（代码异常、资源异常）而不需要业务写 try catch，如何实现？

异步代码该如何进行捕获？

实践题

如果在进入页面的时候需要一次渲染10万条数据，有什么首屏优化的方案？

如果需求的场景不符合每一条数据固定高度？

算法

二叉树有哪种遍历方式

夸克扫描王 极速扫描，就是高效

跳跃游戏（贪心算法）

> 给定一个非负整数数组，你最初位于数组的第一个位置。
>
> 数组中每个元素代表你在该位置可以跳跃的最大长度。
>
> 判断你是否能够达到最后一个位置。

思路

1. 使用一个变量保存当前可到达的最大位置
2. 时刻更新最大位置
3. 可达位置小于数组长度返回 false，否则可以到达

答案

```ts
function canJump(nums: number[]): boolean {
  let k = 0
  for(let i = 0; i < nums.length; i++) {
    if(i > k) return false
    k = Math.max(k, nums[i] + i)
  }
  return true
}
```

招创礼记（一面）

自我介绍

水平垂直居中

defer 、async 区别，作用

输入url的全过程

夸克扫描王 极速扫描，就是高效

深拷贝和浅拷贝的区别

数据类型（基本类型、引用类型）

如何判断类型

js 装箱加载

没了解过，不知道

js 运行机制（事件循环）

笔试题

```js
const square = v => v * v
const double = v => v * 2
const addOne = v => v + 1

const res = pipe(square, double, addOne)
res(3) // 19;

// 实现 pipe

function pipe() {
    const args = Array.from(arguments)
  const n = args.length

  return function(num) {
    if(!n) return null
    return args.reduce((prev, cur) => cur(prev), num)
  }
}
```

webpack loader 介绍一下

夸克扫描王
极速扫描，就是高效

- hooks 作用
- hooks 和 hoc 的区别
- ts interface 和 type 的区别
- mobx 使用过程介绍一下
- git 说一下
- 首屏优化说一下
- 项目中比较难的困难

铱云科技（一面）

- 自我介绍
- 在团队中的角色
- 技术选型的考虑（redux 迁移到 mobx 的原因）
- rxjs 和 mobx 思想上的区别
- mobx 使用上有没有遇到问题，是不是mutable

夸克扫描王 极速扫描，就是高效

js 设计模式

发布订阅和观察者模式的联系与区别

弹窗嵌套弹窗的设计思路

如果需要关闭指定的弹窗，如何设计

多个弹窗需要注意什么

层级上下文

多种层级如何进行管理

OOP 的特性

ts 中泛型的作用

软件开发原则性的理解

在开发组件、编写函数这方面会注意什么问题

如果需要对已有模块进行扩展，如何进行开发保证调用方不受影响

有接触过服务端的知识吗

夸克扫描王
极速扫描，就是高效

性能优化

原生和 webview 之间如何通信

jsBridge

jsBridge

技术和产品之间的关系

平行空间（一面）

自我介绍

css 水平垂直居中

保持盒子的宽高按屏幕大小等比例缩放

浏览器的渲染机制

跨域

原型链的原理

文本超出显示 ...

夸克扫描王
极速扫描，就是高效

- 合并数组、对象的方式
- `export` 和 `export default` 的区别
- 事件队列（事件循环）
- react 生命周期
- redux、mobx 的区别
- hooks 和 hoc 的对比

平行空间（二面）

- 自我介绍
- react、angular 的比较
- 对 vue 的看法
- 工作两年中解决的难题，能体现自己能力的项目
- 迁移 ts 的原因
- 简单介绍一下 react 的生态

夸克扫描王
极速扫描，就是高效

手写代码，数字加逗号

科创先锋（一面）

自我介绍

组件库介绍

lerna 说一下

yarn workspace 了解吗

代码评审的原则

ts 迁移的原因

Mobx 原理

angular 和 react 对比

盒模型说一下

给一个 div 设置一个100px，解释一下

position 说一下

夸克扫描王
极速扫描，就是高效

设置 left top 会对兄弟节点有影响吗

需求题

页面有一个按钮，需要点击按钮拉去地图js，加载地图组件

兔展（一面）

自我介绍

缓存分类

跨域

前端安全和攻击

内容安全策略（CSP）

cookie、session | insert（题目没听清楚）

var let 区别

有哪几种作用域

闭包的理解

夸克扫描王
极速扫描，就是高效

- 异步的理解（事件循环）
- async/await、Promise、generator 的区别
- class 的理解
- new 的实现机制
- webpack 打包原理
- 首屏渲染优化的理解
- seo 有什么理解
- 服务端渲染的原理
- react 某些生命周期被废弃的理由
- fiber reconciler 和 stack reconciler 的区别
- useState 和 useRef 的区别
- hooks如何清除副作用
- ts 的 type 和 interface 的区别

夸克扫描王
极速扫描，就是高效

兔展（二面）

- fiber 架构是什么
- diff 算法实现思路
- key 在 diff 算法中的作用
- diff 算法时间复杂度
- diff 为什么这么高
- fiber 节点是一个什么数据结构
- 开发组件基于什么原则
- 迁移 ts 的过程
- 迁移过程中遇到的问题
- ts 的 type 和 interface 的区别
- ts 里面如何声明函数的重载
- ts 元组和数组的区别

夸克扫描王
极速扫描，就是高效

ts 元组的使用场景

esm 和 cjs 的模块化的区别

pc 端上常见的性能优化手段

HTTP 缓存机制

顺德美云智数（电话）

做过的项目

项目中遇到的问题

了解过 webpack 源码，写过 webpack 插件吗

es next 了解过吗，通过什么途径

浏览器的兼容有什么经验

刚好之前有遇到 try catch 的 finally，吹了一下。

讲一下 react 的主要思想，和 ng 的对比

ng 的性能会比 react 要好

深拷贝、浅拷贝讲一下

夸克扫描王
极速扫描，就是高效

> 实现一个深拷贝的方法

setInterval、setTimeout 时间是有误差的，为什么

计时器的时间是延迟执行，如果同步代码执行时间过长，甚至超出了计时器设定的时间，那么计时器内的回调函数会在同步代码执行完成之后立刻执行。

做一个连续的动画，实现方案

1. setInterval
2. setTimeout
3. css3 动画

水平垂直居中

两栏布局

有些页面需要嵌入第三方页面（iframe），如何进行通信

> 跨域问题

具体跨域解决实现

前端缓存

他应该不是想问我 HTTP 缓存，但是我强行把问题改成了 HTTP 缓存

代码管理工具是用什么

git

有用过子模块吗

夸克扫描王
极速扫描，就是高效

git 嵌套另一个子模块

有用过 rebase 吗

rebase 会导致 commit 的提交人发生改变

酷家乐（一面）

项目中的性能优化

项目中有什么值得说的

ts 有什么优势

type 和 interface 的区别

React 的事件机制

聊聊 React 的 diff

React 优化

如何理解闭包

节流的实现

es5 继承

夸克扫描王
极速扫描，就是高效

酷家乐（二面）

React 基本原理

React 的性能优化

Redux 重点概念

聊一下 React 的生命周期

Hooks 如何处理生命周期

笔试题

1.

```js
const a = { b: 3 }

function foo(obj) {
  obj.b = 5

  return obj
}

const aa = foo(a)

console.log(a.b)

console.log(aa.b)
```

2.

```js
function Ofo() {}
```


夸克扫描王 极速扫描，就是高效

```
function Bick() {
  this.name = 'mybick'
}

var myBick = new Ofo()

Ofo.prototype = new Bick()

var youbick = new Bick()

console.log(myBick.name)

console.log(youbick.name)
```

考察盒子模型和 box-sizing 属性，判断元素的尺寸和颜色。

实现一个 fill 函数，不能用循环

用 ES5 实现私有变量

酷家乐（三面）

性能优化

class组件如何做性能优化

实现一个 TypeScript 里面的 Pick

手写 Promise.all

手写并发限制器

夸克扫描王
极速扫描，就是高效

- 算法题，判断单链表相交
- 算法题，如何找到第一个相交的点

酷家乐（四面）

- 如何实现水平垂直居中
- 看过开源库的源码吗
- 聊聊 React 的源码
- FiberNode 有哪些属性
- stateNode 有什么作用

拼多多（一面）

- React 16 生命周期的变化
- 详细介绍一下 getDerivedStateFromProps
- `flex: 0 1 auto` 具体表示什么含义
- less 的 & 代表什么

夸克扫描王
极速扫描，就是高效

算法题：求字符串最大公共前缀

interface 和 type 的区别

状态管理工具

有用过 ssr 吗

node 熟悉吗

拼多多（二面）

1. class组件和函数组件的区别
2. css 优先级
3. 避免 css 全局污染
4. css modules 原理
5. 有一个 a 标签，如何动态决定他的样式
6. import 和 require 的区别
7. require 有什么性能问题
8. 组件库如何做按需加载
9. webpack 如何实现动态加载
10. react 动态加载的 api
11. React.lazy 的原理
12. webpack 能动态加载 require 引入的模块吗
13. require 引入的模块能做摇树吗
14. 设计一个input组件需要哪些属性
15. value 的类型是什么
16. onChange 怎么规定 value 的类型
17. interface 和 type 的区别
18. 写一个 Promise 重试函数，可以设置时间间隔

夸克扫描王
极速扫描，就是高效

拼多多（三面）

1. 组件平台有哪些功能
2. 实现一个 redux
3. 用 ts 实现一个 redux

拼多多（一面）

redux 理念

react-redux 中 connect 怎么实现

mixin、hoc、继承的区别、优缺点

react diff 实现

react 旧版的diff用深度优先还是广度优先。为什么用深度优先，广度优先能实现吗

diff的时间复杂度？为什么？

O(n)

react 从 O(n3) -> O(n)

react-router 实现原理

客户端路由 hash/history 实现的区别、原理

夸克扫描王
极速扫描，就是高效

- 有哪些常见的 http 头
- websocket 轮询的好处缺点
- websocket 握手的过程
- tcp 握手过程
- tcp/udp 的区别
- 尽可能多实现pdd app 导航布局实现（4等分div并列排布）
- 如何清除浮动
- BFC 实现原理
- 开放题：pdd 假设没有首页优化，尽可能想办法优化

拼多多（二面）

- css盒子模型（content-box、border-box）
- 实现动画有哪些方法
- react mixin hoc 以及 hook 之间的区别和优缺点

夸克扫描王
极速扫描，就是高效

- 浏览器渲染出一个页面的过程
- vue/react 技术选型
- 项目相关，重构开发如何并行
- 项目中工程化进行了哪些优化
- 项目中进行优化会从哪方面去做
- nodejs 性能优化
- linux 文件权限
- 前端监控怎么实现

金山软件

- DNS 解析（浏览器输入 url 全过程）
- 前端缓存
- HTTPS 原理
- HTTP 长连接实现原理

夸克扫描王
极速扫描，就是高效

- 事件循环
- 事件调用栈
- Next.js 如何实现服务端渲染

金山（wps+）一面

- CSS选择器及优先级
- CSS有什么布局方式
- 各种布局之间有什么优缺点
- 你用过什么CSS预编译器
- less 有什么缺点
- 有遇到过1px的问题？知道如何解决吗？
- js里面有什么数据类型
- 有哪些判断方法可以判断类型？
- es6中let、const和var有什么区别

夸克扫描王
极速扫描，就是高效

- const声明的变量怎么修改
- 箭头函数和普通函数的区别
- 事件代理
- 数组有什么方法
- 如何往数组中插入一个值
- 浏览器cookie、localStorage、sessionStorage的区别
- 除此之外，前端还有什么存储方案
- 对前端的安全有什么理解
- 对闭包的理解
- 简单的实现一个闭包

金山（wps+）二面

- 项目中有redux迁移mobx，详细说一下原因。这二者有什么区别
- 介绍一下高版本React和低版本React之间的api废除的理由

夸克扫描王
极速扫描，就是高效

setState 是同步还是异步

在setTimeout中，是否会进行批量更新

说一下React的合成事件

说一下函数组件和class组件的区别

对比一下hook和hoc的优势

说一下graphql解决的问题

做过的seo优化

说一下JWT机制

实践题

假设有一个网站（HTTPS）需要对传输的数据进行加密改造，请针对这个网站的源代码进行改造，让它包括它所依赖的第三方库发出的请求都能进行加密，以及它们的响应都能进行解密。

要求：

1. 当前加密解密方案涉及第三方，但不能修改第三方代码；
2. 尽可能减少对业务代码的侵入

实践题

结合上一题，加解密所需要的密钥需要从服务端获取，请你设计一套方案，保证每一个请求之前，都能通过密钥对数据进行加密。

夸克扫描王
极速扫描，就是高效

如何预防网站劫持

实践题

假设一个页面中，当前模块需要分别请求多个接口（接口可能关联也可能不关联），你需要对请求回来的数据进行组装，请问如何进行优化。

首屏优化

说一下你对前端开发的理解

做过的项目的难点以及如何解决

离开的原因

职业规划

富途证券（架构组）一面

离职原因

想加入什么样的团队

你开发过什么组件?

优化过 webpack 的性能?

夸克扫描王
极速扫描，就是高效

做过什么?

讲一下 ts 的重载? 以及为什么需要用重载?

组件库在迁移的过程中有什么难搞的问题?

做过移动端的开发吗?

了解移动端的设备适配问题吗?

finally 在 IOS 8 用不了的原因是什么?

下面这段代码有什么问题

```js
function Father() {
  this.property = true
}
Father.prototype.getFatherValue = function() {
  return this.property
}

function Son() {
  this.sonProperty = false
}

// 继承
Son.prototype = new Father()
Son.prototype.getSonValue = function() {
  return this.soneProperty
}
var instance = new Son()
```

描述一下原型、构造函数、实例之间的关系

夸克扫描王 极速扫描，就是高效

说一下 instanceof 的原理

怎样判断类型

下面代码输出什么

```js
function F() {}
F.prototype.arr = [1]
F.prototype.b = 1

let obj1 = new F()
obj1.arr.push(2)
obj1.b = 2

let obj2 = new F()
console.log(obj2.arr, obj2.b)
```

下面代码输出什么

```js
setTimeout(function() {
  console.log(1)
}, 0)

new Promise(function (resolve) {
  console.log(2)
  for(var i = 0; i < 10000; i++) {
    i == 9999 && resolve()
  }
  console.log(3)
}).then(function () {
  console.log(4)
})
console.log(5)
```

地址栏输入URL到页面渲染发生了什么

夸克扫描王 极速扫描，就是高效

介绍一下强缓存和协商缓存

HTTPS 能不能做一个中间层，在中间层之间做劫持？有办法避免吗

在 a.com 中能发送一个 ajax 请求到 b.com 吗?

如何避免页面被别人的 iframe 页面嵌套

XSS防御

除了 XSS 以外，还有什么安全问题

假设你的页面加载非常慢，你会怎么判断问题出现在哪？如果是普遍的网络问题，怎样解决？如果不是网络问题，怎么解决？

算法题

```js
// a,b 2个有序数组，a的长度小于等于b
// 元素都是整数
// 存在重复元素
// 判断a是否为b的子集
// 例如：
// a: [1, 2, 3] b: [1, 2, 3, 4] 属于
// a: [1, 1, 2, 2] b: [1, 2, 3, 4] 不属于
```

欢聚时代（总部）一面

离职原因

夸克扫描王 极速扫描，就是高效

- 说一下 Angular 的脏检查
- 有什么办法优化变更检查
- 平时工作中如何优化 Angular 变更检查的性能
- 说一下 React 的 diff 算法
- 说一下 Angular 和 React 的事件机制
- 大概说一下 setState 时，React 中 render 时如何工作的
- 有没有遇到过重新绑定实例的情况（不知道他问的是啥）
- 为什么要用 graphql
- 项目中 pc 端兼容的浏览器版本
- 如何用 css grid 做九宫格
- 说一下 vertical-aligen 的属性
- 如何用 css 写一个三角形
- 有没有用过 es next 的最新语法

夸克扫描王
极速扫描，就是高效

- Promise 的 finally 如何实现
- 有用过迭代器吗
- 有用过异步循环吗
- css 中继承的属性，什么时候会用到
- 你们的项目如何从构建发布到上线
- node 项目中如何守护进程
- 项目中有用过 cdn 进程吗
- 说一下 cdn 的原理

广州 shein 跨境电商（远程面）

- 实现一个 Modal 框的思路
- 实现水平垂直居中
- 实现上下固定，中间自适应
- 有没有做过移动端，有没有了解过自适应和响应式

夸克扫描王
极速扫描，就是高效

哪些样式会触发重排

Webpack 性能优化

有没有写过 Webpack loader

有没有参与组件开发

你们的全键盘是如何实现的

如何让 React 减少重新渲染次数

useMemo，shouldComponentUpdate，memo，组件细粒度

Mobx 如何能让组件触发重新渲染

项目中如何储存用户信息

虾皮（toC group）一面

说出打印结果

手写原型链继承

实例属性和原型属性的区别

夸克扫描王
极速扫描，就是高效

instancef 的原理

浏览器缓存

Service Worker 大小大概是30Gb，如果用户硬盘没有那么大怎么办

Service Worker 的大小和强缓存、协商缓存的大小是一起计算的还是分开计算

Push Cache 的具体处理方式

HTTP2 的优缺点

HTTP2 有没有可能比 HTTP1 还要更慢

var、let、const 的区别

说出打印结果

```js
// 任务
// 面试官提出的问题将出现在这里。

// 写出下面这段代码打印的结果
async function async1() {
  console.log('async1 start');
  await async2();
  console.log('async1 end');
}

async function async2() {
  console.log('async2 start');
}
```


夸克扫描王 极速扫描，就是高效

```
console.log('script start');

setTimeout(function() {
  console.log('setTimeout');
}, 0);

async1();

new Promise(function(resolve) {
  console.log('promise1');
  resolve();
}).then(function() {
  console.log('promise2');
}).then(function() {
  console.log('promise3');
});

console.log('script end');
```

webpack

常用插件

如果有一个工程打包特别大，如何进行优化

cjs 和 esm 模块化的区别

es6+ 有哪些新的语法

跨域解决方案

说一下 CORS 中的预请求

xss 和 csrf

夸克扫描王
极速扫描，就是高效

用户信息存储的方式

React 性能优化的方式

实现一个节流函数

要求初次执行的时候立刻执行

虾皮（toC group）二面

有哪个组件最让你印象深刻

这个组件的原理介绍一下

这个组件有做兼容性处理吗

说出以下代码的输出

```html
<body>
  <div id="box">
    <a href="javascript:console.log(1)" id="anchor">Click</a>
  </div>
</body>
<script>
    var box = document.getElementById('box')
  var anchor = document.getElementById('anchor')

  anchor.addEventListener('click', function() {
    console.log(2)
  })

  box.addEventListener('click', function() {
    console.log(3)
```


夸克扫描王 极速扫描，就是高效

```
  }, true)

    box.addEventListener('click', function() {
    console.log(4)
  })
</script>
```

`let a = "abc"`，解释器在解释在这句话的过程中，内存发生的变化，比如内存放在哪里，申请了多大的内存

介绍一下 esm 和 cjs 的差异

介绍一下前端安全问题

假设有一个页面需要实现下拉无限滚动加载，如何实现和优化

笔试题

```js
// 实现如下这样的函数`f()`，要求调用深度不限。(考察点：对 JS 对象化的理解)

// f(1).val === 1
// f(1)(2).val === 3
// f(1)(2)(3).val === 6
// f(10)(100)(1000)(10000).val === 11110
// f(a0)(a1)(a2)...(an).val === a0 + a1 + a2 +...+ an
```

虾皮（一面）

http 302 301 307 的区别

301 和 302 哪个对 seo 更友好（301）

夸克扫描王 极速扫描，就是高效

- 跨域是什么、如何解决
- jsonp 有什么缺点
- base64和外链的应用场景，各自的优缺点
- http缓存机制
- https 的握手过程
- set/map 的区别
- hook 的局限性
- setState 和 hook 的区别
- decorator 的作用，编译后是什么样子的
- symbol 是什么，一般用来做什么
- csrf 是什么，如何防范
- sql注入是什么，如何防范
- react 调用 setState 之后发生什么

夸克扫描王
极速扫描，就是高效

- nodejs 事件循环机制
- pm2原理，有哪些模式（cluster fork）
- docker 和 k8s 有了解多少
- 移动端一个元素的拖动，如何实现和优化（节流，改变位置）
- for in / for of 看代码输出
- 描述链表的反转如何实现，复杂度多少
- 实现 `instanceof`
- 实现一个对象被 `for of` 遍历
- 实现链表的添加、删除。复杂度是多少

虾皮（二面）

- 给两段效果上都可以实现继承的代码，说出差异

```js
function child() {}
function parent() {}

child.prototype.__proto__ = parent.prototype
child.prototype = new parent()
```


夸克扫描王 极速扫描，就是高效

- this 输出问题
- 如何监听 html 外链资源加载失败
- `Mutation Observer` 、 `Intersection Observer` 使用场景
- `127.0.0.1` 和 `0.0.0.0` 差别（一个只能通过 `localhost` ，另一个可以通过本机 ip 或者 localhost 都可以）
- 利用 Promise js sleep 函数实现
- jsx 转换后是什么样子的
- redux compose 函数是做什么的，中间件呢
- redux-sage 是什么，和 redux-thunk 有什么区别
- dva 了解吗
- umi.js 有用过吗
- req.pipe(res)
- stream 如何处理数据消费和数据生产的速率不一致问题

夸克扫描王
极速扫描，就是高效

writeable stream drain 事件是做什么的（这是和一个控制读写速率有关的事件）

虾皮（一面）

1. react this 绑定
2. bind 和箭头函数的区别
3. react 生命周期
4. setState 同步异步
5. vdom 渲染原理
6. 实现一个方法，将对象的键由下划线修改为小驼峰

虾皮（二面）

1. es6模块管理和cjs 对比
2. es6 装饰器
3. es6+ 新特性
4. 吹逼
5. base64原理，编码后体积大了还是小了
6. 非递归实现后序遍历

百度（安全平台）一面

请实现一个通用add方法，add参数为任意数量，如add(1, 2, 3) 输出6。

请实现Promise.all()。

请实现一个双向数据绑定。

算法题

夸克扫描王
极速扫描，就是高效

```
/*
假如有三个词库，第一个词库的内容为发布、推出，第二个词库的内容为小度音箱、小度在家， 第三个词库的内容为 2020 新款、2019 再版。现需要制定一个规则，组合任意数量词库的内容，并输出结果。如规则为第二个词库+第三个词库+第一个词库，输出内容为
小度音箱 2020 新版发布  小度在家 2020 新版发布
小度音箱 2019 再版发布  小度在家 2019 再版发布
小度音箱 2020 新版推出  小度在家 2020 新版推出
小度音箱 2019 再版推出  小度在家 2019 再版推出

请编写函数gengertorWord，首先输入固定的第一个词库、第二个词库、第三个词库的内容，然后，输入规则(规则中的词库个数可以是任意个数的)，输出对应内容。
*/
```

ts 和 js 的区别

es 和 js 的区别

amd 和 esm 的区别

浏览器是否支持 esm

webpack怎么进行首屏加载的优化

了解过 tree-shaking 吗

闭包为什么会有内存泄漏问题

了解过 nodejs 吗

express 和 koa2、koa3 的区别

夸克扫描王 极速扫描，就是高效

- 了解过 flex 布局吗
- 有8个子盒子，每个盒子宽100px，父盒子宽700px，那么它是一行显示还是会换行?
- 我想换行怎么实现?
- HTTP 和 HTTPS 的区别
- HTTPS 的加密如何实现
- 数字证书如何验证合法性

百度（安全平台）二面

- 输入URL到页面展示中的全过程
- 在这过程中的话涉及那些网络协议
- HTTP协议是否包括端口号
- 端口号是TCP协议的一部分?
- 为什么需要三次握手?

夸克扫描王
极速扫描，就是高效

- 三次握手的全过程
- TCP连接如何确认连接方
- HTTPS的握手过程
- 证书的作用以及判断过程
- 建立SSL连接之后，是对称还是非对称加密
- 为什么会有跨域的问题
- 跨域的网络情况大概是怎样
- 除了常规的GET、POST以外，DELETE等请求也是这个流程
- 解决方案
- jsonp为什么可以实现跨域呢
- 除了script标签，还有哪些标签支持跨域请求
- 直接使用表单提交请求，会有跨域问题吗
- websocket 建立过程是怎样的

夸克扫描王
极速扫描，就是高效

代码题

```ts
// 实现一个函数，输入工资，返回应该缴纳的税费
// 其中税率规则如下：
// 0～5000元不需要交税
// 5000～8000元 3%
// 8000～20000元 5%
// 20000以上 10%

function cal(price: number): number {
  const rules = getTaxRules(price)
  const last = rules.pop()

  return (price - last.begin) * last.tax +
    rules.reduce((pre, cur) => pre + (cur.end - cur.begin) * cur.tax, 0)
}

const TAX_RULE = [
  { begin: 0, end: 5000, tax: 0 },
  { begin: 5000, end: 8000, tax: 0.03},
  { begin: 8000, end: 20000, tax: 0.05},
  { begin: 20000, tax: 0.1}
]

function getTaxRules(price: number): typeof TAX_RULE {
  return TAX_RULE.filter(function (item) {
    return price >= item.begin
  })
}
```

面向对象编程中的三个特征

面向对象编程中的五个规则

开发过程中的设计模式

微信（一面）

1. leetcode-cn.com/problems/in...

夸克扫描王 极速扫描，就是高效

2. leetcode-cn.com/problems/va...
3. 这道题最难。。我使用了回溯法来做，不过不是正确答案，虽然刚好把测试用例都过了。leetcode-cn.com/problems/sp...
4. setTimeout 的原理
5. 聊项目

微信（二面）

1. 吹逼
2. react 和 vue 的区别
3. spa 渲染的优缺点
4. 对前端框架的看法

微信（三面）

1. 微信小程序底层实现
2. 吹逼
3. 最近比较关心的技术

腾讯（部门未知）一面（电话面）

什么是事件委托

了解 Promise 吗

window 的 onload事件和 domcontentloaded 谁先谁后

有遇到过跨域问题吗？如何解决

腾讯（部门未知）二面（QQ 远程面）

夸克扫描王
极速扫描，就是高效

原型链考察

有一个类如下：

```js
function Person(name) {
  this.name = name
}

let p = new Person('Tom')
```

1. p.__proto__ 是什么

   Person.prototype

2. Person.__proto__ 是什么

   Function.prototype

new 考察

有一个类如下：

```js
// 类1
function Person(name) {
  this.name = name
  return name
}

// 类2
function Person(name) {
  this.name = name
  return {}
}

let person = new Person('Tom')
```

1. 类1 实例化后返回的是什么

   { name: 'Tom' }

2. 类2 实例化后返回的是什么

   {}

夸克扫描王 极速扫描，就是高效

typeof 和 instanceof 的区别

下面代码输出什么

```js
function Person(name) {
  this.name = name
}

function Student() {
}

Student.prototype = Person.prototype
Student.prototype.constructor = Student

var s = new Student('Tom')
console.log(s instanceof Person) // true
```

new 和 instanceof 源码实现

手写 new

手写 instanceof

下面代码输出什么

```js
for(var i = 0; i < 10; i++) {
  setTimeout(() => {
    console.log(i)
  }, 0)
}

// 10 个 10
```

如果要修改成输出 0-9

```js
// 使用 let
for(let i = 0; i < 10; i++) {
```


夸克扫描王 极速扫描，就是高效

```
  setTimeout(() => {
    console.log(i)
  }, 0)
}

// 使用闭包
for(var i = 0; i < 10; i++) {
  (function(i) {
    setTimeout(() => {
    console.log(i)
  }, 0)
  })(i)
}

// 使用 setTimeout 的第三个参数
for (var i = 0; i < 10; i++) {
  setTimeout((i) => {
      console.log(i)
    }, 0, i)
}
```

箭头函数 this 的指向问题

沿着它的调用链往外找，第一个非箭头函数的 this

for..in 和 for...of 的区别

说说 generator 的了解

flex 布局中的 flex-grow 和 flex-shrink 属性的作用

说一下宏任务和微任务，并说出下面代码的运行结果

```
console.log('a')
setTimeout(() => {
  console.log('b')
}, 0)
console.log('c')
Promise.resolve().then(() => {
  console.log('d')
}).then(() => {
```


夸克扫描王 极速扫描，就是高效

```
  console.log('e')
})
console.log('f')

// a -> c -> f -> d -> e -> b
```

数组扁平化处理

```js
function flatten(list) {
  return list.reduce(function(prev, curr) {
    return prev.concat(Array.isArray(curr) ? flatten(curr) : curr)
  }, [])
}

function flatten(list) {
  while(list.some(v => Array.isArray(v))) {
    list = [].concat(...list)
  }
  return list
}
```

腾讯（部门未知）三面（电话远程面）

输入 URL 到页面加载发生了什么

缓存知识

描述一下 DNS 解析过程

TCP 是如何发起连接和断开连接

三次握手 四次挥手

你知道有什么状态码

夸克扫描王 极速扫描，就是高效

有哪些手段可以优化网页响应速度

5点15分，时针和分针的夹角

67.5 度

算法

> 8 个外表一样的小球，其中 7 个球重量相同，1 个球为异常球，可能比较重也可能比较轻，利用天平至少需要称重多少次才能确保找出异常球，并且需要知道到底是轻了还是重了。

答案：2次

步骤：

1. 将八个球分成 A、B、C 三组，分别 3个，3个，2个；
2. 将 A、B 组称重，若重量相同，则异常球在 C 组；
3. 取 C 组任意一球和 A、B 组任意一球称重，如果不一样，则当前球为异常球，且知道是重了还是轻了，反之则另一个球为异常球。

腾讯（部门未知）三面（笔试）

原生 js 实现

1. 实现一个 div 滑动的动画，由快至慢（不准使用 css3）

思路：

- 定时器
- 一个变量用来存滑动速度，并在定时器内递减

2. 页面内有一个 input 输入框，实现数组 arr 查询命中次并要求 autocomplete 效果

算法

1. 大数相加

夸克扫描王 极速扫描，就是高效

腾讯（部门未知）四面

> 聊项目，笔者项目经验主要是一个基于微信环境的讲座系统（涉及功能文字、语音、图片、表情的聊天系统）

你在做这个系统如何确保消息实时推送

Node.js + WebSocket(socket.io)

消息撤回功能如何实现?

WebSocket 有时会出现掉线，如何解决

加入心跳机制

```js
var heartCheck = {
  timeout: 60000, // 60ms
  timeoutObj: null,
  serverTimeoutObj: null,
  reset: function (){
    clearTimeout(this.timeoutObj)
    clearTimeout(this.serverTimeoutObj)
    this.start()
  },
  start: function (){
    var self = this
    this.timeoutObj = setTimeout(function (){
      ws.send('HeartBeat')
      self.serverTimeoutObj = setTimeout(function() {
        ws.close()
      }, self.timeout)
    }, this.timeout)
  }
}

ws.onopen = function() {
  heartCheck.start()
}

ws.onmessage = function() {
  heartCheck.reset()
}
```


夸克扫描王 极速扫描，就是高效

```
ws.onclose = function() {
  reconnect()
}

ws.onerror = function() {
  reconnect()
}
```

了解过哪些排序算法，描述一下归并排序的实现

有没有了解过 react 的 diff 算法

讲讲前端安全问题

腾讯（IMWeb）一面

你是怎么接触前端的

js new 执行构造函数的的过程

js 说一下 this 指向

就是说 this 绑定的几种情况

说一下 bind 函数结构

说一下 js 继承

我说了寄生组合继承

中间他问了为什么需要把子函数的constructor 修改回本身：

原因在于使用 Object.create 修改子类的原型的 `__proto__` ，从而导致子类的构造函数变更为父类的构造函数，这样会导致 instanceof 判断错误。

夸克扫描王 极速扫描，就是高效

实现正则表达式

> 实现域名匹配 qq.com 以及这个域名的所有子域名。
>
> 子域名：a.qq.com ， a.b.qq.com

```js
const reg = /((\w\.)+(qq.com)$)|^(qq.com)$/
```

实现思路：以 qq.com 作为字符串的结尾进行判断，以此判断 字符串. 的这种格式

说一下 DOM事件流模型

说一下你知道的 HTTP 状态码

浏览器缓存的过程

说一下 XSS

如何防范

说一下 CSRF

危险网站 B 如何拿到受信任网站 A 的 cookie

危险网站B 无法拿到网站 A 的cookie，它是直接在网站B中向A的目标服务器发送get请求

危险网站 B 利用哪种 API 受信任网站 A 发送请求

xhr

夸克扫描王 极速扫描，就是高效

项目经历中有写 SSR，说一下你是怎么实现的

Next.js

如何保证同构的模块不会挂掉，例如在服务端访问 document

没有做过

据了解，这种写法应该禁止

ts问题

上一个面试说过，业务项目中经常存在线上会出现 null、undefined 等类型错误，所以迁移了 ts，ts解决了什么问题

你觉得 ts 很重吗，如何确保你的同事不写 any

配置禁掉了 any，如果用了 `//@ts-ignore` 会在 review 过问

ts 给你带来的价值

实现一个 ts 的工具函数

```ts
type A = {
  a: number;
  b: string;
  c: () => void;
  d: (s: string) => boolean;
}
```

实现一个工具函数 `GetOnlyFnProps<T>` ，提取泛型类型 T 中字段值是函数的类型。

```ts
type GetOnlyFnKeys<T extends object> = {
  [K in keyof T]: T[K] extends Function ? K : never
}[keyof T]
```


夸克扫描王 极速扫描，就是高效

```
type GetOnlyFnProps<T extends object> = {
  [K in GetOnlyFnKeys<T>]: T[K]
}
```

你有维护组件库，说一下如何管理组件

git + lerna

组件质量如何保存

没有测试用例的情况下，组件发布完全靠测试去人工测试？有没有遇到出错的情况。

组件发布的是不是所有依赖这个组件库的项目都需要升级?

发布问题

如果业务A需要组件进行修改，发布了版本 1.0.1，此时业务B没有升级，仍然是 1.0.0，如果此时业务 B 需要组件修改，发布版本 1.0.2，这时候就会带上 1.0.1上的修改，这上面的修改有可能会引发业务 B 的问题，如何处理

除了 lerna，还有其他的方式吗

你们组件库，别人如何能知道如何使用

storybook

假如现在是下午 5 点 35 分，时针和分针的夹角是多少

时钟每一大格是 30 度，每一小格是 6 度

时针每过一分钟移动 0.5 度

腾讯（IMWeb）二面

夸克扫描王 极速扫描，就是高效

迁移 ts 的背景和原因

聊到一个服务端渲染的项目，具体聊聊为什么需要使用服务端渲染

这几个项目中，哪几个项目的挑战难度比较大

笔试题

1. 事件循环机制考察，打印输出的

2. this 指向问题

```js
var a = 2

var obj = {
  a: 3,
  fn: function () {
    (() => {
      console.log(this.a)
    })()
  },
}
obj.fn()
```

3. 实现 input 的 autocomplete 功能

大数相加

最近有没有了解前端最新技术

react 17 有了解过吗

夸克扫描王
极速扫描，就是高效

pwa 有了解吗

deno 和 node 的区别

在之前的公司，有没有主动推动一些技术的发展

腾讯（IMWeb）三面

为什么离职

上上家为什么离职

我聊了对技术不追求

你认为上一家公司对技术有追求吗

我答了出色的系统

如何理解一个出色的系统

你们有多少的客户

客户样本太少，如何解决

哪个功能客户用得最多

整个系统里面一共有多少个订单

夸克扫描王
极速扫描，就是高效

为什么要从 redux 从 mobx

你觉得这两个各自在哪个场景下更加合适

系统维护了多久

经手了很多人，维护的难度会不会很大

所以接入 ts

为什么接入 ts 会延长系统的生命周期

官网重构

怎样优化 seo

怎样进行首屏渲染测速

怎么知道服务端渲染的速度会快呢

是同构吗？是数据同构？是UI同构？

整个服务端渲染的瓶颈在哪里？单机qps是多少？

整个网站有多少用户量？有多少台 node 服务器

夸克扫描王
极速扫描，就是高效

用的是 http/1 还是 http/2，用的是 http 还是 https?

你最擅长的是什么

有用 lint 吗

一共有多少条规则

ts 除了静态类型检查以外，还有什么优秀的地方

只有 ts 有装饰器吗

React 的版本?

计算题

```js
/*
    假设说，你们部门70%的人喜欢打篮球，80%的人喜欢踢足球，90%的人喜欢打排球，问篮球和足球都喜欢的人有多少
*/
```

给出的条件中能算出具体值吗？不能的话能算出具体范围吗?

如果团队需要加班到晚上 2-3 点，你觉得合理吗?

腾讯微视（桌面工程师方向）一面

夸克扫描王 极速扫描，就是高效

之前做的一家跨境电商的公司，这里面有什么可以说一下的地方

主要写的是第二家，说一下

具体遇到 ts 迁移遇到的问题说一下

什么情况下会遇到这样的问题

为什么是用 babel

相比 babel 编写 es 或者 JavaScript，TypeScript 的优势是什么

mongodb 和 typescript 的 orm 是怎么做的

grapql 和 restful 的区别和优势

angular 和 react 的区别

react 的 component 的生命周期是什么样子的

实现一个双向绑定

```html
<input type='text' name='name' id='txtName'/>

<script>
    var data = {
    name: '安格斯'
  }
```


夸克扫描王
极速扫描，就是高效

```
  function binding(data) {
    var model

    return model
  }

  var model = binding(data)
  model.name = '马克'
  // 用户输入 修改 data.name
</script>
```

```html
<script>
    var data = {
    name: '安格斯'
  }

  var input = document.getElementById('txtName')

  function binding(data) {
        input.value = data.name

    return Object.defineProperty(data, 'name', {
      get() {
        return input.value
      },
      set(value) {
        input.value = value
      }
    })
  }

</script>
```

腾讯面试四问

页面通信

> 问题一：从页面 A 打开一个新的页面 B，B 页面关闭（包括意外崩溃），如何通知页面 A?

题目真实要求是：新打开的页面 B 页面关闭（包含意外崩溃）如何回传给页面 A。

拆分问题：

1. B 页面正常关闭，B 页面如何通知 A 页面（涉及参数回传、参数监听）

夸克扫描王 极速扫描，就是高效

2. B 页面意外崩溃，比如线程直接被杀死，如何通知 A 页面（涉及监听页面崩溃）

我们应该分别作答。

B 页面正常关闭

1. 页面关闭时会触发的事件?

   页面关闭时会先执行 `window.onbeforeunload` ，然后执行 `window.onunload` ，我们可以在这两个事件中设置回调。

2. 如何传参?

   > 如何跳转到一个已经打开的页面

   用 `window.open` 跳转到一个已经打开的页面，url 上面带上需要用到的参数。

3. 成功传参后如何监听 url 的变化?

   如果路由是采用 hash 模式，则使用 `onhashchange` 的方式可以监听到路由 `#` 之后的变化，

   如果是 history 模式，则监听 popstate 的变化

同时，如果两个页面如果是同源的情况下，也可以通过监听 localStorage 的方式达到两个页面之间的通信。

B 页面意外崩溃

B 页面意外崩溃，这时候 JS 线程都已经崩溃了，如何传递通知呢?

Logging Information on Browser Crashes

简单来说：在网页 onload 事件设置一个 pending 状态，beforeunload 事件下改变这个状态为 exit，如果二次访问这个页面，查看这个状态，如果这个状态是 pending，则说明上一次关闭的时候是意外崩溃。这个状态可以存在 localStorage。

如果是想实时通知 A 页面，我们可以使用 Service Worker 来实现网页崩溃的监控。

1. Service Worker 有自己独立的线程，与网页区分开，网页崩溃的情况下，一般 Service Worker 并不会崩溃。
2. Service Worker 的生命周期一般比网页要长，所以可以用来监控网页的状态。
3. 网页可以通过 `navigator.serviceWorker.controller.postMessage` API 向掌管自己的 Service Worker 发送消息。

具体是现实采用 **心跳检测**：

1. 网页 B 每 5s 给自己的 Service Worker 发送一次心跳，记录一个状态并更新时间戳，如果正常关闭的时候，通知 Service Worker 清除这个状态；

夸克扫描王 极速扫描，就是高效

2. 如果网页崩溃了，状态和时间戳都不会变更，Service Worker 定时轮询检查状态，一旦发现状态不对且时间戳已经很长没有更新了，则说明网页已经崩溃。
3. Service Worker 通过 postMessage 与页面进行通信（Service Worker）

DOM 监听

> 自己如何实现 Vue 的数据监听，能够检测到 DOM 新增加绑定的属性吗?

知道 Vue2 原理的小伙伴应该都知道，数据双向绑定主要依赖于 `Object.defineProperty` 对数据的劫持，它有 get 和 set 方法，可以监听到对象属性的读取和设置。

`Object.defineProperty` 可以监听 DOM 属性吗?

`Object.defineProperty` 监听的目标是对象，DOM 元素的属性集合 `[dom.attributes]` 也是对象，当然可以被监听。

其中要注意的是，例如：style 属性也是一个属性集合，它的子属性并不能被监听。

原因是因为 `Object.defineProperty` 的不足：

1. 无法监听数组的变化：数组这些方法是无法触发 set：push、pop、shift、unshift、splice、sort、reverse。Vue 中之所以能监听是因为对这些方法进行了重写（hack）。
2. 只能监听属性，监听的不是对象的本身，需要对对象的每个属性进行遍历。对原本不在对象中的属性难以监听。Vue 中 使用 `Vue.set(object, propertyName, value)` 方法向嵌套对象添加响应式属性。

如何监听一个新创建的属性呢?

手动对新创建的属性进行监听。

> Vue.set 的原理：
>
> 当一个数据为响应式时，vue 会给该数据添加一个 `__ob__` 属性，因此可以通过判断 target 对象是否存在 `__ob__` 属性来判断 target 是否是响应式数据。当 target 是非响应式数据的时候，我们就按照普通对象添加属性的方式来处理；当 target 对象是响应式数据时，我们将 target 的属性 key 也设置为响应式并手动触发通知其他属性值的更新。

```js
defineReactive(ob.value, key, val)
ob.dep.notify()
```


夸克扫描王 极速扫描，就是高效

除了监听滚轮，还有什么懒加载的方法?

交叉观察者

利用 IntersectionObserver 接口提供的一种异步观察目标元素与其祖先元素或顶级文档视口交叉状态的方法。

头条（一面）

1. position 有哪些属性
2. `position: sticky` 用过吗
3. typeof 原理，与 instanceof、 `Object.prototype.toString.call` 的区别
4. React的生命周期
5. React 那些生命周期可以 setState
6. 提供一个vdom对象，写一个render函数让它变成一个DOM
7. 算法题：快排

头条（二面）

1. 为什么 0.1 + 0.2 !== 0.3
2. 算法题：字符串相加
3. 问我最近对什么技术感兴趣
4. img 标签间距问题以及如何解决
5. 深拷贝

头条（三面）

1. DNS 查询过程
2. 如果你发现一个网站打不开了，有哪些可能
3. 算法题：有一个“123456789101112131415 .... n+1" 这样的序列，求第 m 个数字
4. 有一个有序递增的序列，求有多少个不同的数字
5. 吹逼

头条（四面）

1. 红黑树和哈希表的对比

夸克扫描王 极速扫描，就是高效

2. 哈希表如何解决冲突
3. 线程和进程的区别
4. 场景：有一个应用会经常创建、删除节点对象，如何优化（节电池）

字节跳动

实现一个抓包请求

> 实现一个并发限制功能

```js
function asyncPool(poolLimit, array, iteratorFn) {
  let i = 0
  const ret = []
  const executing = []
  const enqueue = function () {
    if(i === array.length) return Promise.resolve()
    const item = array[i++]
    const p = Promise.resolve().then(() => iteratorFn(item, array))
    ret.push(p)
    const e = p.then(() => executing.splice(executing.indexOf(e), 1))
    executing.push(e)
    let r = Promise.resolve()
    if(executing.length >= poolLimit) {
      r = Promise.race(executing)
    }
    return r.then(() => enqueue())
  }
  return enqueue().then(() => Promise.all(ret))
}
```

实现一个防抖功能

```js
function debounce(fn, delay, immediate) {
  let timer
  let result

  const debounced = function() {
    const context = this
    const args = arguments

    if(timer) clearTimeout(timer)
```


夸克扫描王 极速扫描，就是高效

```
    if(immediate) {
      const callNow = !timer
      timer = setTimeout(function() {
        timer = null
      }, delay)
      if(callNow) result = fn.apply(context, args)
    } else {
      timer = setTimeout(function () {
        fn.apply(context, args)
      }, delay)
    }
  }

  debounced.cancel = function() {
    clearTimeout(timer)
    timer = null
  }

  return debounced
}
```

拓展：现在有一个搜索框，已加上防抖按钮，但恰好后面的请求先回来怎么办？

给每个请求添加唯一的 id，对比请求回来的 id 是否是最新 id

或者重新发送请求的时候，取消上一次请求。

实现一个图片懒加载

判断页面滚动的时候，图片是否在视口内

获取一个异步求和函数

字节跳动（Lark）一面

平时怎么提升自己的前端水平

github 上面主要写什么

笔试题

夸克扫描王
极速扫描，就是高效

1.

2.

3.
```
// 任务
// 面试官提出的问题将出现在这里。

// 写出下面这段代码打印的结果
async function async1() {
  console.log('async1 start');
  await async2();
  console.log('async1 end');
}

async function async2() {
  console.log('async2 start');
  return new Promise((resolve, reject) => {
    resolve();
    console.log('async2 promise');
  })
}

console.log('script start');

setTimeout(function() {
  console.log('setTimeout');
}, 0);

async1();

new Promise(function(resolve) {
  console.log('promise1');
  resolve();
}).then(function() {
  console.log('promise2');
}).then(function() {
  console.log('promise3');
});

console.log('script end');
```

4.

5.

浏览器渲染机制的理解

夸克扫描王 极速扫描，就是高效

- 性能优化方面的理解

字节跳动（Lark）二面

- 你觉得最有挑战的系统
- 整个系统中如何解决各个模块之间关联的问题
- 测试上线流程是怎样
- 如何保证在改动过程中不影响别的模块
- 阐述一下在系统中哪个模块比较复杂
- 了解整个模块是怎么进行设计的吗
- 如果一个电商的页面，有很多的商品列表，用户在加载这个页面的时候很慢，你会从哪个方面解决问题
- 假如用户明确反馈在渲染列表的时候明显卡顿，如何解决
- 你会如何分析具体的问题出现的位置
- 阐述一下重绘与重排

夸克扫描王
极速扫描，就是高效

什么时候需要重排

有一个页面，可以在后台输入商品内容展示给用户查看，你会怎样进行设计

公共组件你会怎么进行设计

算法题

```js
/*
 给定一个整数数组，长度不定，如：[1, 2, 3, 4]，设计一个算法，要求你计算出每一项元素中，除了它以外其它元素的乘积。
 要求，算法中不能使用除法，同时时间复杂度必须控制在O(n)。
 demo：
 var list = [1, 2, 3, 4]
 return [24, 12, 8, 6]
*/

function helper(list) {
  var n = list.length
  var before = [1]
  var after = Array(list.length).fill(1)

  var i = 1

  while(i < n) {
    before[i] = before[i - 1] * list[i - 1]
    i++
  }

  i--
  while(i >= 0) {
        if(i === n - 1) {
      i--
      continue
    }
    after[i] = after[i + 1] * list[i + 1]
    i--
  }

  var result = []
  for(var j = 0; j < n; j++) {
    result.push(before[j] * after[j])
  }
```


夸克扫描王 极速扫描，就是高效

```
    return result
}
```

字节（一面）

- 常用的 http 状态码（101 200 204 301 302 304 307 400 404 500）
- 301 和 302 的区别（永久/临时）
- 前端路由如何实现
- 前端路由刷新404如何处理（404时重定向到index.html）
- 从前端到后端全链路如何设计一个登录的流程（session 方案、jwt）
- 什么是跨域，怎样处理
- 简单请求、复杂请求
- 浏览器的缓存策略
- domain 属性解决跨域，几种domain设置对跨域是否生效
- ts泛型做什么的，infer关键字的作用
- 实现 typescript 的 Paramters、ReturnType

夸克扫描王
极速扫描，就是高效

- 实现 Promise.all

- 实现一个并发请求控制函数，限制并发数

字节（二面）

- jwt 原理

- xss和csrf防范

- pureComponent 和 Component 区别（前者 shouldComponentUpdate 默认比较 props）

- react hook hoc 解决了什么问题

- 有哪些常用的 hook

- 项目实现一个中间层的意义（前端全链路日志打通）

- setState 同步还是异步

- setState 传递对象和函数有什么区别

- 合成事件是怎样的，有什么好处

夸克扫描王
极速扫描，就是高效

- 事件冒泡和事件捕获的区别，react中的冒泡和捕获（react 都是合成事件，无论是冒泡还是捕获，都对应原生中的冒泡）

- 水平垂直居中

- 手写 bind

- 实现快排

字节（三面）

- graphql 相比 restful 的区别，优缺点

- 为啥选择了 `redux-saga` 作为解决方案(解决异步优雅/提供很多工具函数)。 `redux` 的异步中间件方案有哪些，对比。 `saga` 对比 `dva` ？

- graphql的鉴权在哪里做

- 协程是什么（语言层面实现的并发），Go/nodeJs怎样去实现协程（Go使用go关键字、 `node` 可以使用 `Generator` 实现、我理解 `react` 16的异步渲染也是协程的实现）

- `react` `fiber` 的机制是怎样的

- 锁机制的作用，node和Go如何处理死锁

夸克扫描王
极速扫描，就是高效

- Go/node实现并发怎么做
- node和Go的优缺点怎样理解
- 实现一个 useState

快手（一面）

- react 16 新增了那些生命周期函数，有什么作用，为什么去掉某些 15 的生命周期
- fiber 原理，如何实现异步渲染（链表/可中断）
- redux 和 redux-saga 的区别和原理
- 项目中用到的 saga-duck 是什么东西
- useEffect 的实现原理
- 异步渲染和旧版的 diff 的区别
- babel 实现转码的过程（词法、语法分析）
- 项目技术栈怎么选型

夸克扫描王
极速扫描，就是高效

小程序底层实现原理了解多少（双线程模型/预加载Webview）

项目 nodejs 应用异常退出如何处理（pm2/uncaughtException 事件等）、日志上报怎么做（输出重定向到文件，elk服务传到kibana）

graphic schema 怎么设计

有哪些技术驱动业务的案例

算法：最大乘积子序列

YY

> 5月14日

面试部门：广州某部门

一面

面试形式：牛客网视频面试

面试时长：50分钟

面试感悟：YY和CVTE都是托树酱 小哥哥帮忙找的朋友内推，另外YY算第一个面的比较大的厂了吧，也是第一次在牛客网上视频面试，所以还是有些激动的。面试官稍微晚到了2分钟，不过问题不大

说一下你们项目动态表单的实现

说一下动态表格的实现

动态表格由两部分组成：

- 一个是上面的索引内容(搜索条件)，索引内容因为和动态表单很像，又有输入框，又有下拉框，所以上面的索引我也是基于动态表单UI组件库进行开发的

夸克扫描王 极速扫描，就是高效

- 二是下面具体的表格查询的内容，这个是基于Ant Design of Vue中的Table进行封装的，它的结构其实很简单，只要传入需要的表头字段列表，也就是 `columns` ，还有具体的数据进去就能显示出表格。不过这里因为表头字段的数量不确定，所以可能需要用到Vue的计算属性来进行一个计算，例如如果表头字段的数量是小于8的话则表格的 `x` 方向不需要滚动，否则整个 `x` 方向的长度就为 `数量 X 105px` 类似这样的值。以及还有涉及到是否分页，每个字段的排序呀等操作...

你们这个JSON树是在哪里生成的，前端会生成吗?

盘石中配置的，后台生成。

有了解过微前端吗?

我们后台有用到微服务，但是前端这边没有用到。

了解浏览器的哪些兼容问题

1、不同浏览器的标签默认的外补丁(margin)和内补丁(padding)不同 解决方案： css 里增加通配符 * { margin: 0; padding: 0; }

2、IE6双边距问题；在 IE6中设置了float , 同时又设置margin , 就会出现边距问题 解决方案：设置 display:inline;

3、当标签的高度设置小于10px，在IE6、IE7中会超出自己设置的高度 解决方案：超出高度的标签设置 overflow:hidden,或者设置line-height的值小于你的设置高度

4、图片默认有间距 解决方案：使用float 为img 布局

5、IE9以下下浏览器不能使用opacity 解决方案：

```css
div {
  opacity: 0.5;
  filter: alpha(opacity = 50);
  filter: progid:DXImageTransform.Microsoft.Alpha(style = 0, opacity = 50);
}
```

6、边距重叠问题；当相邻两个元素都设置了margin 边距时，margin 将取最大值，舍弃最小值； 解决方案：为了不让边重叠，可以给子元素增加一个父级元素，并设置父级元素为overflow:hidden；

7、cursor:hand 显示手型在safari 上不支持 解决方案：统一使用 cursor:pointer

8、两个块级元素，父元素设置了overflow:auto；子元素设置了position:relative ;且高度大于父元素，在IE6、IE7会被隐藏而不是溢出 解决方案：父级元素设置position:relative

夸克扫描王 极速扫描，就是高效

IE有哪些兼容性问题

同上

说一下EventLoop

必问...

在项目里有用过node吗

之前自己工作之余有玩过一些，但是没有实际运用在工作中。

如何实现一个findIndex

针对于这种要我们口述实现的问题，大家可以从这么几个方面去回答：

- 需要实现的 `API` 的功能点
- 该 `API` 的参数及返回值
- 是否需要挂载在某个原型对象上(例如 `Array.prototype`)，或者就是一个单独的函数(例如实现 `new`)
- 具体的实现方式

根据 `MDN` 上的要求，这里贴一个简易版的实现方式。

需要满足的条件：

1. `findIndex` 方法对数组中的每个数组索引 `0..length-1` （包括）执行一次 `callback` 函数，直到找到一个 `callback` 函数返回真实值（强制为 `true` ）的值。
2. 如果找到这样的元素， `findIndex` 会立即返回该元素的索引。
3. 如果回调从不返回真值，或者数组的 `length` 为0，则 `findIndex` 返回-1。
4. 回调函数调用时有三个参数：元素的值，元素的索引，以及被遍历的数组。
5. 如果一个 `thisArg` 参数被提供给 `findIndex` ，它将会被当作 `this` 使用在每次回调函数被调用的时候。如果没有被提供，将会使用 `undefined` 。
6. `findIndex` 不会修改所调用的数组。

Coding:

```js
Array.prototype.myFindIndex = function (cb, context = undefined) {
  if (typeof cb !== 'function') {
    throw new TypeError('cb must be a function');
  }
```


夸克扫描王 极速扫描，就是高效

```
    var arr = [].slice.call(this);
    var len = arr.length >>> 0;
    let i = 0;
    while (i < len) {
      if (cb.call(context, arr[i], i, arr)) {
        return i;
      }
      i++;
    }
    return -1;
  }
  function isEven (num) {
    return num % 2 === 0;
  }
  console.log([3, 4, 5].myFindIndex(isEven)) // 1
```

具体的 Polyfill 可以看 MDN 上：Array.prototype.findIndex()

移动端布局有哪些方案?

- flex + rem
- vw + px

...

如果一个移动端的项目要显示在PC端上保证结构稳定你会如何做?

这个没有唯一的答案，还希望有大佬能够补充一下，后期更新到文章中来。

这边有的一个方案是：

首先保证在移动端上显示正常，然后设置一个最大的 max-width 来限定死。

你自己的项目里有做过哪些webpack上的优化

参考文章： https://juejin.im/post/5ea528496fb9a03c576cceac#heading-2

- 先使用 webpack-bundle-analyzer 分析打包后整个项目中的体积结构，既可以看到项目中用到的所有第三方包，又能看到各个模块在整个项目中的占比。
- Vue 路由懒加载，使用 () => import(xxx.vue) 形式，打包会根据路由自动拆分打包。
- 第三方库按需加载，例如 lodash 库、 UI 组件库
- 使用 purgecss-webpack-plugin 和 glob 插件去除无用样式(glob 插件可以可以同步查找目录下的任意文件夹下的任意文件)：

夸克扫描王 极速扫描，就是高效

```
new PurgecssWebpackPlugin({
    // paths表示指定要去解析的文件名数组路径
    // Purgecss会去解析这些文件然后把无用的样式移除
    paths: glob.sync('./src/**/*', {nodir: true})
    // glob.sync同步查找src目录下的任意文件夹下的任意文件
    // 返回一个数组，如['真实路径/src/css/style.css','真实路径/src/index.js',...]
})
```

- 文件解析优化：
  - babel-loader 编译慢，可以通过配置 exclude 来去除一些不需要编译的文件夹，还可以通过设置 cacheDirectory 开启缓存，转译的结果会被缓存到文件系统中
  - 文件解析优化：通过配置 resolve 选项中的 alias 。 alias 创建 import 或者 require 的别名，加快 webpack 查找速度。
- 使用 webpack 自带插件 IgnorePlugin 忽略 moment 目录下的 locale 文件夹使打包后体积减少 278k
  - 问题原因：使用 moment 时发现会把整个 locale 语言包都打包进去导致打包体积过大，但是我们只需要用到中文包
  - 在 webpack 配置中使用 webpack 自带的插件 IgnorePlugin 忽略 moment 目录下的 locale 文件夹
  - 之后在项目中引入：

```
// index.js
// 利用IgnorePlugin把只需要的语言包导入使用就可以了，省去了一下子打包整个语言包
import moment from 'moment';
// 单独导入中文语言包
import 'moment/locale/zh-cn';
```

  - (或者不用这种方式，直接使用更加轻量的 Day.js 也可以)
- 使用 splitChunks 进行拆包，抽离公共模块，一些常用配置项：
- chunks :表示选择哪些 chunks 进行分割，可选值有： async，initial和all
  - minSize ：表示新分离出的 chunk 必须大于等于 minSize ，默认为30000，约30kb
  - minChunks ：表示一个模块至少应被minChunks个chunk所包含才能分割，默认为1
  - name ：设置 chunk 的文件名
  - cacheGroups ：可以配置多个组，每个组根据test设置条件，符合test条件的模块，就分配到该组。模块可以被多个组引用，但最终会根据priority来决定打包到哪个组中。默认将所有来自node_modules目录的模块打包至vendors组，将两个以上的chunk所共享的模块打包至default组。
- DllPlugin 动态链接库，将第三方库的代码和业务代码抽离：

夸克扫描王 极速扫描，就是高效

- 根目录下创建一个 `webpack.dll.js` 文件用来打包出 `dll` 文件。并在 `package.json` 中配置 `dll` 指令生成 `dll` 文件夹，里面就会有 `manifest.json` 以及生成的 `xxx.dll.js` 文件
- 将打包的dll通过 `add-asset-html-webpack-plugin` 添加到html中，再通过DllReferencePlugin把dll引用到需要编译的依赖。

具体说一下splitChunksPlugin

有自己写过webpack插件吗

写过一些简单的插件。

一、File-list-plugin:

作用：在每次打包成功之后都会在最终的输出目录下生成一个 `fileList.md` 文件，记录了输出目录的所有文件信息。

实现原理：

- 定义一个名为 `FileListPlugin` 的构造函数，重写它原型对象上的 `apply` 方法。
- 在此方法中触发 `compiler` 的 `'emit'` 钩子，也就是生成资源到 `output` 目录之前
- 然后因为此钩子的回调函数中有一个 `compilation` 对象，所以可以通过 `compilation.assets` 获取到所有待生成的文件
- 在构建一下 `md` 里的内容，也就是把将每一项文件的名称写入 `markdown` 文件内
- 最后通过以下方式把 `markdown` 文件放到要输出的目录下：

```js
compilation.assets[fileListName] = {
    source: function () {
      return content;
    },
    size: function () {
      return content.length;
    }
}
```

完整代码：

File-list-plugin.js:

```js
function FileListPlugin (options) {
  this.options = options || {};
```


夸克扫描王 极速扫描，就是高效

```
  this.filename = this.options.filename || 'fileList.md'
}

FileListPlugin.prototype.apply = function (compiler) {
  // 1.
  compiler.hooks.emit.tapAsync('FileListPlugin', (compilation, cb) => {
    // 2.
    const fileListName = this.filename;
    // 3.
    let len = Object.keys(compilation.assets).length;
    // 4.
    let content = `# 一共有${len}个文件\n\n`;
    // 5.
    for (let filename in compilation.assets) {
      content += `- ${filename}\n`
    }
    // 6.
    compilation.assets[fileListName] = {
      // 7.
      source: function () {
        return content;
      },
      // 8.
      size: function () {
        return content.length;
      }
    }
    // 9.
    cb();
  })
}
module.exports = FileListPlugin;
```

二、Watch-plugin

作用：观察者模式下，当每次资源变动了，将改动资源的个数以及改动资源的列表输出到控制台中

Watch-plugin.js:

```
function WatcherPlugin (options) {
  this.options = options || {};
}

WatcherPlugin.prototype.apply = function (compiler) {
  compiler.hooks.watchRun.tapAsync('WatcherPlugin', (compiler, cb) => {
    console.log('我可是时刻监听着的 🚀🚀🚀')
    let mtimes = compiler.watchFileSystem.watcher.mtimes;
    let mtimesKeys = Object.keys(mtimes);
    if (mtimesKeys.length > 0) {
      console.log(`本次一共改动了${mtimesKeys.length}个文件,目录为:`)
      console.log(mtimesKeys)
```


夸克扫描王 极速扫描，就是高效

```
        console.log('------------分割线-------------')
      }
      cb()
    })
    compiler.hooks.watchClose.tap('WatcherPlugin', () => {
      console.log('本次监听停止了哟～👌👌👌')
    })
  }
  module.exports = WatcherPlugin;
```

三、Clean-plugin

作用：每次打包之后，生成了新的文件，则需要把以前的文件给清理掉。

Clean-plugin.js:

```
const recursiveReadSync = require("recursive-readdir-sync");
const minimatch = require("minimatch");
const path = require("path");
const fs = require("fs");
const union = require("lodash.union");
function CleanPlugin (options) {
  this.options = options;
}
// 匹配文件
function getUnmatchFiles(fromPath, exclude = []) {
  const unmatchFiles = recursiveReadSync(fromPath).filter(file =>
    exclude.every(
      excluded => {
        return !minimatch(path.relative(fromPath, file), path.join(excluded), {
          dot: true
        })
      }
    )
  );
  return unmatchFiles;
}
CleanPlugin.prototype.apply = function (compiler) {
  const outputPath = compiler.options.output.path;
  compiler.hooks.done.tap('CleanPlugin', stats => {
    if (compiler.outputFileSystem.constructor.name !== "NodeOutputFileSystem") {
      return;
    }
    const assets = stats.toJson().assets.map(asset => asset.name);
    // 多数组合并并且去重
    const newAssets = union(this.options.exclude, assets);
    // 获取未匹配文件
    const unmatchFiles = getUnmatchFiles(outputPath, newAssets);
    // 删除未匹配文件
    unmatchFiles.forEach(fs.unlinkSync);
  })
```


夸克扫描王 极速扫描，就是高效

```
}

module.exports = CleanPlugin;
```

HTTP3.0.1和HTTP3.0.2的区别

说一下Vue-Router的实现原理

Vue-Router初始化是发生在什么时候

你觉得我还有哪些问题没有问到的，也就是你可以说一下你的加分项

CVTE

> 5月14日

面试部门：希沃未来教育某部门(广州)

一面

面试形式：电话面

面试时常：1小时6分钟

面试感悟：面试官人很和善，基本全程都在 "嗯" ，也会和你开玩笑，然后我答不出来的时候他也会说： "嗯,没关系,问题不大" 。不过可能是因为自己在某些问题上回答的太繁重了，导致经常被面试官打断(大概有2，3次吧)，所以大家在面试的时候也要抓重点回答不要偏离问题的方向。

离职原因

新的一份工作你主要看中的是什么?

平台、团队、薪资、地域

先说一下你的期望薪资吧

夸克扫描王 极速扫描，就是高效

你为什么觉得你这次跳槽能有这么大的工资涨幅

我是从这几个角度来分析的：

- 自己离职前本来应该还有一次加薪机会的，但是走了，所以我认为这次跳槽的薪资幅度也不算高
- 对比于相同工作年限的同行来说在技术上可能会稍微强一些
- 自己的社区影响力，喜欢写博客

如果给不到期望薪资你会怎么办

薪资只是我找工作的其中一个比较项，回到我最原始的想法，我更在意的是平台和团队。

说一下你们项目动态表单的实现

你遍历JSON树的具体过程

webpack构建流程

(当时是按欧巴的总结答的：「吐血整理」再来一打Webpack面试题(持续更新))

初始化参数 ：从配置文件和 Shell 语句中读取与合并参数，得出最终的参数

开始编译 ：用上一步得到的参数初始化 Compiler 对象，加载所有配置的插件，执行对象的 run 方法开始执行编译

确定入口 ：根据配置中的 entry 找出所有的入口文件

编译模块 ：从入口文件出发，调用所有配置的 Loader 对模块进行翻译，再找出该模块依赖的模块，再递归本步骤直到所有入口依赖的文件都经过了本步骤的处理

完成模块编译 ：在经过第4步使用 Loader 翻译完所有模块后，得到了每个模块被翻译后的最终内容以及它们之间的依赖关系

输出资源 ：根据入口和模块之间的依赖关系，组装成一个个包含多个模块的 Chunk，再把每个 Chunk 转换成一个单独的文件加入到输出列表，这步是可以修改输出内容的最后机会

输出完成 ：在确定好输出内容后，根据配置确定输出的路径和文件名，把文件内容写入到文件系统

webpack插件原理

夸克扫描王
极速扫描，就是高效

webpack 在编译代码过程中，会触发一系列 Tapable 钩子事件，插件所做的，就是找到相应的钩子，往上面挂上自己的任务，也就是注册事件，这样，当 webpack 构建的时候，插件注册的事件就会随着钩子的触发而执行了。

webpack在配置插件的时候是一个数组那它有顺序吗

当时我回答的是：这个可能要从webpack插件设计者的角度来看吧，因为webpack插件它是可以指定在哪个生命周期来做某些处理的，所以我认为它主要是和设计时的调用时机有关，而和定义时数组里的顺序无关；但是loader我知道它是有执行顺序的，一般是从右到左，从下到上，但是也可以用一个叫做 `enforce` 的属性去改变执行顺序。

有用过哪些插件做项目的分析吗

当时答的：

`webpack-dashboard` ： 可以更友好的展示相关打包信息。

`speed-measure-webpack-plugin` ： 简称 SMP，分析出 Webpack 打包过程中 Loader 和 Plugin 的耗时，有助于找到构建过程中的性能瓶颈。

`size-plugin` ： 监控资源体积变化，尽早发现问题

`webpack-bundle-analyzer` ： 分析打包后整个项目中的体积结构，既可以看到项目中用到的所有第三方包，又能看到各个模块在整个项目中的占比。

有做过哪些webpack优化吗

当时答的：

1. 使用 `purgecss-webpack-plugin` 和 `glob` 插件去除无用样式(`glob` 插件可以可以同步查找目录下的任意文件夹下的任意文件)：

```js
new PurgecssWebpackPlugin({
    // paths表示指定要去解析的文件名数组路径
    // Purgecss会去解析这些文件然后把无用的样式移除
    paths: glob.sync('./src/**/*', {nodir: true})
    // glob.sync同步查找src目录下的任意文件夹下的任意文件
    // 返回一个数组，如['真实路径/src/css/style.css','真实路径/src/index.js',...]
})
```

2. 文件解析优化：

夸克扫描王 极速扫描，就是高效

- babel-loader 编译慢，可以通过配置 exclude 来去除一些不需要编译的文件夹，还可以通过设置 cacheDirectory 开启缓存，转译的结果会被缓存到文件系统中
- 文件解析优化：通过配置 resolve 选项中的 alias 、 extensions 、 modules 来实现。 alias 创建 import 或者 require 的别名；加快 webpack 查找速度。 extensions 自动解析确定的扩展； modules 解析模块时应该搜索的目录，通常建议使用绝对路径，避免层层查找祖先目录。

3. 还有的话，从 webpack-merge 提取一些公共的配置项

有自己写过插件吗

必问...

让你从零开始构建一个webpack项目你可以吗

实不相瞒的告诉您，我之前有写过webpack相关的这一部分教程，包括从零开始搭建，然后再到基本的 loader 的使用，比如 style-loader 、 css-loader 、 raw-loader 等，然后到一些基本的插件的使用，比如 html-webpack-plugin 、 clean-webpack-plugin ，以及像 webpack-dev-server ...

面试官： "好了好了，我大概清楚了，OK，这个问题我们过吧"

从输入URL到页面呈现的过程

必问...

为什么TCP要三次握手而不是两次

(这个大家应该都清楚)

为了保证双方都有发送和接收的能力，如果只有两次握手的话就只能保证发送方，一般就是我们的客户端有发送的能力，但是并不能保证它有接收的能力，所以必须是三次。

HTTP和TCP的区别

当时的答案：

HTTP的责任是去定义数据，在两台计算机相互传递信息时，HTTP规定了每段数据以什么形式表达才是能够被另外一台计算机理解。

而TCP所要规定的是数据应该怎么传输才能稳定且高效的传递与计算机之间。

夸克扫描王 极速扫描，就是高效

你们项目一般是如何做缓存的

当时答的很乱...就提到了一些 `Expires` 以及 `cache-control` 等等。

- 对于一些没有指纹信息的资源，例如 `index.html` 可以使用 `Cache-Control: no-cache` 开启协商缓存
- 对于带有指纹信息的资源，一般会使用splitChunksPlugin进行代码分割，来保证造成最小范围的缓存失效，再设置 `Cache-Control: max-age=3153600`

(答的时候，由于想给自己争取更多的思考时间，就佯装咳嗽，面试官问你还好吧，要不要喝口水😂)

什么情况会阻塞页面的加载

提到了 `script` 加载的时候会阻塞页面。JS属于单线程，当我们在加载 `script` 标签内容的时候，渲染(`GUI`)线程会被暂停，因为 `script` 标签里可能会操作 `DOM` 的，所以如果你加载 `script` 标签又同时渲染页面肯定就冲突了，因此说渲染线程(`GUI`)和js引擎线程互斥。

然后我还想说一下引用 `CSS` 的时候使用 `@import` 也会，然后就被面试官打断了。

script放在body头部就一定会阻塞吗

没想出来，然后反问了他是不是不操作DOM就不会?

面试官： `呵呵呵，没事，你等会去了解一下`

但后面有了解到，面试官想问的可能是 `async` 和 `defer` 吧:

正常模式

这种情况下 JS 会阻塞浏览器，浏览器必须等待 index.js 加载和执行完毕才能去做其它事情。

```html
<script src="index.js"></script>
```

async(异步) 模式

async 模式下，JS 不会阻塞浏览器做任何其它的事情。它的加载是异步的，当它加载结束，JS 脚本会立即执行。

```html
<script async src="index.js"></script>
```

defer(延缓) 模式

夸克扫描王 极速扫描，就是高效

defer 模式下，JS 的加载是异步的，执行是被推迟的。等整个文档解析完成、DOMContentLoaded 事件即将被触发时，被标记了 defer 的 JS 文件才会开始依次执行。

```html
<script defer src="index.js"></script>
```

从应用的角度来说，一般当我们的脚本与 DOM 元素和其它脚本之间的依赖关系不强时，我们会选用 async；当脚本依赖于 DOM 元素和其它脚本的执行结果时，我们会选用 defer。

添加删除了DOM节点会发生什么？(回流)

把回流的过程说一下就可以了

js中改变transform的left和right对比于css修改transform

我的回答： `css` 中使用 `transform` 会促发 `CPU` 加速，使浏览器不会进行回流和重绘。但是如果用 `js` 改变的话我不确定会不会，这样吧，我盲猜一下我感觉它会，因为我记得我看过说要避免频繁的使用 `style` ，不过不知道自己说的对不对。所以相对于用 `js` 调用的话后者的性能肯定就会更好一些。

面试官： `呵呵呵，这个你下去在了解一下吧`

什么是GPU加速

在使用 `CSS3` 中的 `transform` 、 `opacity` 、 `filter` 这些属性的时候，能跳过布局和绘制流程，直接进入非主线处理的部分，即交给合成线程。

- 合成线程中会调用线程池进行GPU加速，因为GPU是比较擅长处理位图数据的
- 且它没有占有主线程

进程和线程的区别

进程：资源分配的最小单位

线程：资源调度的最小单位

面试官； `"你是不是回答反了？算了算了，我们不绕了，这个问题就这样吧"`

(上面这个是正确的，当时应该是答反了)

做过H5页面没有？你怎样适配移动端的

夸克扫描王 极速扫描，就是高效

做的少，然后我提到了在程序初始化的时候可以使用以下方式来修改根节点的字体大小：

(假设你需要的 `1rem = 20px`)

```js
(function () {
  var html = document.documentElement;
  function onWindowResize() {
    html.style.fontSize = html.getBoundingClientRect().width / 20 + 'px';
  }
  window.addEventListener('resize', onWindowResize);
  onWindowResize();
})();
```

- `document.documentElement` ：获取 `document` 的根元素
- `html.getBoundingClientRect().width` ：获取 `html` 的宽度(窗口的宽度)
- 监听 `window` 的 `resize` 事件件

一般还需要配合一个 `meta` 头：

```html
<meta name="viewport" content="width=device-width, initial-scale=1.0, minimum-sacle=1.0, maximum-scale=1.0, user-scalable=no" />
```

HTTP/2对比HTTP1.1

- 使用 `HPACK` 算法进行头部压缩， `HTTP/2` 当中废除了起始行的概念，将起始行中的请求方法、URI、状态码转换成了头字段，不过这些字段都有一个 `":"` 前缀，用来和其它请求头区分开。
- 使用二进制分帧解决了HTTP层面的队头阻塞，即多个请求都通过一个TCP连接并发地完成，实现多路复用。
- 服务器推送。服务器不再是完全被动地接收请求，响应请求，它也能新建 stream 来给客户端发送消息
- 新的二进制格式。HTTP/2采用二进制格式传输数据，相比于HTTP/1.1的文本格式，二进制格式具有更好的解析性和拓展性。

HTTP/2是怎么解决队头阻塞的

(当时只答出了第一段话)

HTTP/2引入了一个二进制分帧层，客户端和服务端进行传输时，数据会先经过二进制分帧层处理，转化为一个个带有请求ID的帧，这些帧在传输完成后根据ID组合成对应的数据。

夸克扫描王 极速扫描，就是高效

所以原本一个个完整的HTTP请求报文变成了现在的一堆乱序的二进制帧，这些二进制帧因为不存在先后顺序所以也就不会排队等待，也就没有了HTTP队头阻塞的问题。

通信双方都可以给对方发送二进制帧，这种二进制帧的双向传输的序列，就叫做流。HTTP/2在一个TCP连接上来进行多个数据帧的通信，这就是多路复用的概念。

(所谓的乱序，指的是不同 ID 的 Stream 是乱序的，但同一个 Stream ID 的帧一定是按顺序传输的。)

HTTP/2是如何压缩头部的

(回答参考：https://juejin.im/post/5e76bd516fb9a07cce750746#heading-91)

主要是通过HPACK算法来实现压缩头部的功能，其主要有以下两个亮点：

- 首先是在服务器和客户端之间建立哈希表，将用到的字段存放在这张表中，那么在传输的时候对于之前出现过的值，只需要把**索引**(比如0，1，2，...)传给对方即可，对方拿到索引查表就行了。这种**传索引**的方式，可以说让请求头字段得到极大程度的精简和复用。
- 其次是对于整数和字符串进行**哈夫曼编码**，哈夫曼编码的原理就是先将所有出现的字符建立一张索引表，然后让出现次数多的字符对应的索引尽可能短，传输的时候也是传输这样的**索引序列**，可以达到非常高的压缩率。

为什么说HTTPS比HTTP安全呢

- 通过混合加密保证传输的数据不被窃听
- 通过数字签名的方式保证数据不会被篡改
- 通过数字证书保证服务器身份的真实性

说一下对称加密和非对称加密

对称密钥加密是最简单的一种加密方式，它的加解密用的都是相同的密钥，这样带来的好处就是加解密效率很快，但是并不安全，如果有人拿到了这把密钥那谁都可以进行解密了。

而非对称密钥会有两把密钥，一把是私钥，只有自己才有；一把是公钥，可以发布给任何人。并且加密的内容只有相匹配的密钥才能解。这样带来的一个好处就是能保证传输的内容是安全的，因为例如如果是公钥加密的数据，就算是第三方截取了这个数据但是没有对应的私钥也破解不了。不过它也有缺点，一是公钥因为是公开的，谁都可以过去，如果内容是通过私钥加密的话，那拥有对应公钥的黑客就可以用这个公钥来进行解密得到里面的信息；二来公钥里并没有包含服务器的信息，也就是并不能确保服务器身份的合法性；并且非对称加密的时候要消耗一定的时间，减低了数据的传输效率。

HTTP请求的什么时候用的对称加密什么时候非对称加密

在建立连接的时候为了保证双方都能拿到 对称密钥 ，所以需要使用非对称加密

夸克扫描王 极速扫描，就是高效

而双方都拿到了这把 对称密钥 之后，就可以用这把对称密钥进行对称加密了。

对称加密的原理

(面试官提示万变不离其宗...可我还是悟不出来)

如果让你去实现一个CSRF攻击你会怎做?

当时答的只知道CSRF的攻击原理，但是不知道怎么去做...

还知道其它的攻击方式吗?

Vue中key的作用

key 的作用是尽可能的复用 DOM 元素。

新旧 children 中的节点只有顺序是不同的时候，最佳的操作应该是通过移动元素的位置来达到更新的目的。

需要在新旧 children 的节点中保存映射关系，以便能够在旧 children 的节点中找到可复用的节点。key也就是children中节点的唯一标识。

如果我将key设置为了一个Math.random()可以吗

这道题我开始没太理解面试官的意思，然后他提示了我：回到 key 设计的初衷。之后开着玩笑的说了一句： "我在给你点时间思考一下，我挺希望你能答上来的。"

我： 对不起我可能要让你失望了，我确实没想出来

面试官： 嗯...没关系问题不大哈，这个问题我们过吧

(不过在最后面试完之后我也有问面试官这个问题，他说： key 它就是为了保证我们的DOM节点能够复用，如果你都把它设置为了一个随机数的话，那每次重写渲染都会是另一个数了根本不可能达到复用的效果)

😂...绝了，原来想问的是这个意思，是我学艺不精了。

如果让你设计一个双向绑定你会如何设计

我： "我有了解Vue的双向绑定原理，如果你想听的话我可以说给你听"

面试官： "不用，你就大致说一下就可以了，让我知道你确实是了解了"

夸克扫描王 极速扫描，就是高效

我： "巴拉巴拉"

如何实现if(a===1&&a===2&&a===3)

由于双向绑定的时候提到了 `Object.defineProperty()` 属性，面试官可能就想到了这道题。

这里其实我应该反思一下自己...

在面试官还没说完题目的时候我就急着先说自己知道甚至还想问面试官是不是这道题，这样显然是不好的，可能也是和面试官的面试氛围太好了...放飞了自我吧。但还好面试官不和我斤斤计较。

是有看过这道题吗？这么快的就回答上来了

😂😂😂，老实回答，在自己的文章中有写到这道题，所以很快就知道你想考的是什么了。

之后面试官说： "嗯呢，我昨天也有看了一下你的文章，lv4，望尘莫及..."

我： 哈哈哈哈，没有没有，您自谦了，您只是没有写而已

(不能膨胀，谦虚)

你觉得我还有哪些问题没有问到的，也就是你可以说一下你的加分项

> 5月16日

二面

面试形式：电话面

面试时常：46分钟

面试感悟：面试官相对于一面的面试官来说会严肃一些，总体面试氛围也还可以。有很多问题，例如问到ngnix或者node，我表明了自己不熟之后也没有再问了。面到后面面试官的语气也和蔼了很多，我认为和一面一样都是比较好的面试体验吧。

说一下你们权限的控制，路由权限以及按钮权限

按钮权限除了用你现在的方式你还能想到更好的吗？比如封装一个组件

没有...

你们的token一般是放在哪里

夸克扫描王
极速扫描，就是高效

- token放在Cookie和放在localStorage、sessionStorage中有什么不同吗
- Cookie存在哪些安全问题？如何预防？
- SameSite设置为了lax之后是怎样来控制Cookie的发送的
- 如果顶级域名不同会发送吗
- 还有哪些前端安全性问题
- 如果使用jsonp的话会有什么安全问题吗？
- 你们有做过单点登陆吗
- 有用过ngnix吗？用它做过哪些事？
- 有用过node吗？比如用它写一些中间层
- 项目里做过哪些优化？
- 你认为你做的项目最大的亮点在哪里
- 项目中碰到的技术难点
- 是什么让你有动力写bpmn.js教材呢

夸克扫描王
极速扫描，就是高效

有看过哪些源码呢

有给git上的比较好的项目提过pull吗

如果现在给你一个React的项目你觉得自己能多久上手呢

你的职业规划是怎样呢

说一下你的期望薪资吧?

有对比过深圳和广州这边的薪资吗?

> 5月18日

HR面

面试形式：微信视频

面试时长：50分钟

面试感悟：HR小姐姐很漂亮。然后问了我是否单身，那边也有很多单身的小姐姐。我说如果有缘的话要给我介绍一波。整个氛围还是比较轻松的。问了挺多问题的，我大概挑一些列举，其它的问题在别的面试官那里也有问到我就不重复了。另外在面试完之后，她告诉我，HR还会有二面，请等待通知...

有了解过我们公司吗?

之前不是在深圳吗？为什么打算来广州?

人生中有没有经历过什么对你触动比较大的事情?

有，失恋那次，但我总不能说这个吧。说点别的。

夸克扫描王
极速扫描，就是高效

父母是如何看待你的工作的?

你现在单身吗?

单身！单身！我单身！(你要给我介绍女朋友吗？)

工作之余的兴趣爱好

期望薪资给不到怎么办

> 5月23日

HR二面

面试形式：微信视频

面试时长：58分钟

面试感悟：二面的HR小姐姐也很漂亮😂...开始的HR小姐姐并没有骗我，哈哈哈，开个玩笑，收。其实到了这一步，面试的公司基本就挺想要你了，所以问题也都是一些关于你自己对公司的看法，以及你未来的一个人生或者职业规划。关于这些问题，你如实回答就可以了。就比如HR问你：你是否打算在广州长期发展呀，会不会干了一年之后就又跳了呢？之类的。

编程猫

> 5月20日

编程猫是从上午10点面到了12点，一共是四轮面试，5位面试官(二面两名面试官一起)，在二面之后就已经累的说不出话了...感觉身体还是不行啊，得加强锻炼。总体来说效率还是很高的，2小时搞定了。

一面

(前端精神小伙)

面试形式：现场面

面试时长：50分钟

面试感悟：由于一面的面试官年纪应该和我相仿，所以还算是聊的来，在自我介绍了之后就问了一些项目相关的问题，之后还是死磕基础知识问。期间也有提到过看了我的掘金文章，可是面试的时候竟然还是要我说一说数据转换类型(我怀疑他没有仔细看哦)。最后聊到福利待遇这一块他也还是挺尴尬的😅

夸克扫描王 极速扫描，就是高效

动态表单的实现

权限的控制

你们项目一般是如何做缓存的

- 对于一些没有指纹信息的资源，例如 `index.html` 可以使用 `Cache-Control: no-cache` 开启协商缓存
- 对于带有指纹信息的资源，一般会使用splitChunksPlugin进行代码分割，来保证造成最小范围的缓存失效，再设置 `Cache-Control: max-age=3153600`
- 不需要缓存的资源设置 `Cache-Control: no-store` ，即不强缓存也不进行协商缓存

项目中的环境变量是如何控制的?

有两种方式：

- `Shell` 语句中指定 `--mode`
- `webpack.config.js` 中指定 `mode`

假设有两个子项目，他们需要共用同一个用户体系如何保证关掉页面之后打开另一个项目用户还是登录状态?

Cookie跨站点访问就可以解决。把用户每次登录获取到的 `Token` 存储在 `Cookie` 中，因为 `Cookie` 是可以同站传输的。

(反正面试官点头了，我也不知道对不对)

项目如何优化

业务层面

打包层面

首屏加载优化

(白屏时间是指浏览器从响应用户输入网址地址，到浏览器开始显示内容的时间。首屏时间是指浏览器从响应用户输入网址地址，到首屏内容渲染完成的时间，此时整个网页不一定要全部渲染完成，但在当前视

夸克扫描王 极速扫描，就是高效

窗的内容需要。白屏时间是首屏时间的一个子集。）

- CDN分发
- HTTP缓存方案：强缓存+hash文件
- 前端做好缓存方案，例如存储在内存(即保存变量)，另一个存储在LocalStorage
- 前端的动态资源加载：路由动态加载、组件动态加载
- 利用好 `async` 和 `defer` 两个属性
- 页面使用骨架屏
- 使用SSR渲染
- 利用好HTTP压缩，比如指定 `Content-Encoding` 首部字段为 `gzip`

SSR的使用场景

由于提到了SSR，但是在项目中没有用，不过面试官还是问了我它的使用场景。

很多网站是出于效益(seo)的考虑才启用服务端渲染，性能倒是在其次。

EventLoop

必问...

requestAnimationFrame属于宏任务还是微任务

它不属于宏任务也不属于微任务，因为它是独立于主线程之外的任务，不归主线程管。

输入URL到页面的呈现

必问...

script与css还有页面的渲染顺序

首先对于 `script` 标签正常情况下它会阻塞浏览器，必须得等 `script` 标签加载并执行完里面的脚本代码才能去做其他的事情，渲染线程(`GUI`)和js引擎线程互斥。

而css是由单独的下载线程异步下载的，由于DOM树的解析和构建这一步与css并没有关系，所以它并不会影响DOM的解析。不过最终的布局树是需要DOM树和DOM样式的，因此它会阻塞布局树的建立。

script标签的async是什么时候加载的

夸克扫描王 极速扫描，就是高效

async 模式下，JS 不会阻塞浏览器做任何其它的事情。它的加载是异步的，当它加载结束，JS 脚本会立即执行。

说一下==数据类型转换吧

当使用 `==` 进行比较的时候，会有以下转换规则（判断规则）：

1. 两边类型如果相同，值相等则相等，如 `2 == 3` 肯定是为 `false` 的了
2. 比较的双方都为基本数据类型：

- 若是一方为 `null、undefined` ，则另一方必须为 `null或者undefined` 才为 `true` ，也就是 `null == undefined` 为 `true` 或者 `null == null` 为 `true` ，因为 `undefined` 派生于 `null`
- 其中一方为 `String` ，是的话则把 `String` 转为 `Number` 再来比较
- 其中一方为 `Boolean` ，是的话则将 `Boolean` 转为 `Number` 再来比较

3. 比较的一方有引用类型：

- 将引用类型遵循类似 `ToNumber` 的转换形式来进行比较(也就是 `toPrimitive(obj, 'defalut')`
- 两方都为引用类型，则判断它们是不是指向同一个对象

(具体可以看我的这篇文章呀：【精】从206个console.log()完全弄懂数据类型转换的前世今生(下))

说一下Vue的diff算法

简单来说，diff算法有以下过程

- 先同级比较再比较子节点
- 先判断一方有子节点和一方没有子节点的情况。如果新的一方有子节点，旧的一方没有，相当于新的子节点替代了原来没有的节点；同理，如果新的一方没有子节点，旧的一方有，相当于要把老的节点删除。
- 再来比较都有子节点的情况，这里是 `diff` 的核心。首先会通过判断两个节点的 `key、tag、isComment、data同时定义或不定义以及当标签类型为input的时候type相不相同` 来确定两个节点是不是相同的节点，如果不是的话就将新节点替换旧节点。
- 如果是相同节点的话才会进入到 `patchVNode` 阶段。在这个阶段核心是采用双端比较的算法，同时从新旧节点的两端进行比较，在这个过程中，会用到模版编译时的静态标记配合 `key` 来跳过对比静态节点，如果不是的话再进行其它的比较。

`Vue3.x` 借鉴了 `ivi` 算法和 `inferno` 算法。

它在创建 `VNode` 的时候就确定了其类型，以及在 `mount/patch` 的过程中采用 `位运算` 来判断一个 `VNode` 的类型，在这个基础之上再配合核心的Diff算法，使得性能上较Vue2.x有了提升。

diff算法的缺点

夸克扫描王 极速扫描，就是高效

因为采用的是同级比较，所以如果发现本级的节点不同的话就会将新节点之间替换旧节点，不会再去比较其下的子节点是否有相同。

二面

(架构师)

面试形式：现场面

面试时长：30分钟左右

面试感悟：这次二面唯一一次同时被两名面试官面的情况。两名面试官一个问完就看看另一个(内心活动：该你了，你有什么要问的)，还是挺逗的。而且架构师老哥我回答一句他就 "嗯" 一句，贼飘...收不回来了。但是可以看到在他们那个层次还是能看到很多我看不到的东西的，问的问题也都是偏项目构建方面。

说下你们项目的亮点

- 动态表单表格，它核心的价值在于解决了不同业务产生的差异化和个性化，让我们的作业效率有了很大的提升，这一点不仅仅是对于使用者，也同样提高了我们开发人员的效率。使得一份表单在不同的阶段，也就是生命周期能够呈现不同的内容。
- 另一点就是工作流，因为一个进件(也就是订单)它是如何审批，由谁审批，这些流程是如何走下去的，这点在盘石系统中都能来进行一个配置。

你们的UI组件库怎么实现按需引入的呢?

(这里的按需引入的意思是，对于一些基础的组件，暴露给父级时，如果父级没有用到它就不引入进来)

可以写一个方法，判断传入进来的数据中用到了哪些类型的基础组件，只对用到的基础组件进行一个引入。

Webpack打包原理

(回答参考：「吐血整理」再来一打Webpack面试题(持续更新))

初始化参数 ：从配置文件和 Shell 语句中读取与合并参数，得出最终的参数

开始编译 ：用上一步得到的参数初始化 Compiler 对象，加载所有配置的插件，执行对象的 run 方法开始执行编译

确定入口 ：根据配置中的 entry 找出所有的入口文件

编译模块 ：从入口文件出发，调用所有配置的 Loader 对模块进行翻译，再找出该模块依赖的模块，再递归本步骤直到所有入口依赖的文件都经过了本步骤的处理

夸克扫描王 极速扫描，就是高效

完成模块编译 ：在经过第4步使用 Loader 翻译完所有模块后，得到了每个模块被翻译后的最终内容以及它们之间的依赖关系

输出资源 ：根据入口和模块之间的依赖关系，组装成一个个包含多个模块的 Chunk，再把每个 Chunk 转换成一个单独的文件加入到输出列表，这步是可以修改输出内容的最后机会

输出完成 ：在确定好输出内容后，根据配置确定输出的路径和文件名，把文件内容写入到文件系统

Webpack打包有哪些优化

参考文章： https://juejin.im/post/5ea528496fb9a03c576cceac#heading-2

- 先使用 webpack-bundle-analyzer 分析打包后整个项目中的体积结构，既可以看到项目中用到的所有第三方包，又能看到各个模块在整个项目中的占比。
- Vue 路由懒加载，使用 () => import(xxx.vue) 形式，打包会根据路由自动拆分打包。
- 第三方库按需加载，例如 lodash 库、 UI 组件库
- 使用 purgecss-webpack-plugin 和 glob 插件去除无用样式(glob 插件可以可以同步查找目录下的任意文件夹下的任意文件)：

```
new PurgecssWebpackPlugin({
    // paths表示指定要去解析的文件名数组路径
    // Purgecss会去解析这些文件然后把无用的样式移除
    paths: glob.sync('./src/**/*', {nodir: true})
    // glob.sync同步查找src目录下的任意文件夹下的任意文件
    // 返回一个数组，如['真实路径/src/css/style.css','真实路径/src/index.js',...]
})
```

- 文件解析优化：
  - babel-loader 编译慢，可以通过配置 exclude 来去除一些不需要编译的文件夹，还可以通过设置 cacheDirectory 开启缓存，转译的结果会被缓存到文件系统中
  - 文件解析优化：通过配置 resolve 选项中的 alias 。 alias 创建 import 或者 require 的别名，加快 webpack 查找速度。
- 使用 webpack 自带插件 IgnorePlugin 忽略 moment 目录下的 locale 文件夹使打包后体积减少 278k
  - 问题原因：使用 moment 时发现会把整个 locale 语言包都打包进去导致打包体积过大，但是我们只需要用到中文包
  - 在 webpack 配置中使用 webpack 自带的插件 IgnorePlugin 忽略 moment 目录下的 locale 文件夹
  - 之后在项目中引入：

夸克扫描王 极速扫描，就是高效

```js
// index.js
// 利用IgnorePlugin把只需要的语言包导入使用就可以了，省去了一下子打包整个语言包
import moment from 'moment';
// 单独导入中文语言包
import 'moment/locale/zh-cn';
```

- 使用 `splitChunks` 进行拆包，抽离公共模块，一些常用配置项：
- `chunks` :表示选择哪些 `chunks` 进行分割，可选值有： `async`，`initial`和`all`
  - `minSize` ：表示新分离出的 `chunk` 必须大于等于 `minSize` ，默认为30000，约30kb
  - `minChunks` ：表示一个模块至少应被minChunks个chunk所包含才能分割，默认为1
  - `name` ：设置 `chunk` 的文件名
  - `cacheGroups` ：可以配置多个组，每个组根据test设置条件，符合test条件的模块，就分配到该组。模块可以被多个组引用，但最终会根据priority来决定打包到哪个组中。默认将所有来自node_modules目录的模块打包至vendors组，将两个以上的chunk所共享的模块打包至default组。
- `DllPlugin` 动态链接库，将第三方库的代码和业务代码抽离：
  - 根目录下创建一个 `webpack.dll.js` 文件用来打包出 `dll` 文件。并在 `package.json` 中配置 `dll` 指令生成 `dll` 文件夹，里面就会有 `manifest.json` 以及生成的 `xxx.dll.js` 文件
  - 将打包的dll通过 `add-asset-html-webpack-plugin` 添加到html中，再通过DllReferencePlugin把dll引用到需要编译的依赖。

HTTP/2对比HTTP/1.1

- 使用 `HPACK` 算法进行头部压缩， `HTTP/2` 当中废除了起始行的概念，将起始行中的请求方法、URI、状态码转换成了头字段，不过这些字段都有一个 `":"` 前缀，用来和其它请求头区分开。
- 使用二进制分帧解决了HTTP层面的队头阻塞，即多个请求都通过一个TCP连接并发地完成，实现多路复用。
- 服务器推送。服务器不再是完全被动地接收请求，响应请求，它也能新建 stream 来给客户端发送消息
- 新的二进制格式。HTTP/2采用二进制格式传输数据，相比于HTTP/1.1的文本格式，二进制格式具有更好的解析性和拓展性。

怎样看待你做的这几个项目呢

还有挺多问题的...当时脑子有点糊，面完后都忘了...

三面

(项目负责人)

夸克扫描王 极速扫描，就是高效

面试形式：现场面

面试时长：10分钟左右

面试感悟：项目负责人老哥文文静静的，说话还是挺幽默的，不过感觉自带气场，造成了一定的压力。

项目亮点

你在项目里负责什么

项目难点

你的优点

还有什么想要问的吗?

项目负责人主要都是问一些项目相关的问题，还有你自身的一些问题。

HR面

面试形式：现场面

面试时长：15分钟左右

面试感悟：HR小姐姐：看你面了三场了肯定特别累了吧，我们就简单的聊一下(不得不夸一下此小姐姐温柔体贴😝)。然后大概介绍了一下公司的情况，以及公司的优势。

非技术的自我介绍

住哪里?

工作之余的爱好

其它的就是上家公司薪资、期望薪资、还有什么想要了解的之类的...

夸克扫描王 极速扫描，就是高效

滴滴

> 5月20日

面试部门：杭州某部门

一面

面试形式：视频面试

面试时长：50分钟

面试感悟：面试官给人的感觉...唔...人畜无害？哈哈哈，很和蔼。但是问的问题都很有规律，比如问完项目，就说我看你会用Vue，接下来我问点Vue相关的吧，然后就是双向绑定原理、nextTick原理、虚拟DOM等等；之后又问了HTTP相关。最后他也说了他们现在也是做这种动态表单相关的项目，所以看看我们之前是怎样设计的...自我感觉滴滴面的还是挺好的，但是没有了后续...之后拖内推的老哥帮忙打听原因，那边也没有回复...仔细想了一下，可能还是项目不够硬吧，因为对比来说亮点还不够闪亮。

说一下你们的第一个项目

说一下你们的UI组件库

动态表格的实现

动态表格由两部分组成：

- 一个是上面的索引内容(搜索条件)，索引内容因为和动态表单很像，又有输入框，又有下拉框，所以上面的索引我也是基于动态表单UI组件库进行开发的
- 二是下面具体的表格查询的内容，这个是基于Ant Design of Vue中的Table进行封装的，它的结构其实很简单，只要传入需要的表头字段列表，也就是 `columns` ，还有具体的数据进去就能显示出表格。不过这里因为表头字段的数量不确定，所以可能需要用到Vue的计算属性来进行一个计算，例如如果表头字段的数量是小于8的话则表格的 `x` 方向不需要滚动，否则整个 `x` 方向的长度就为 `数量 X 105px` 类似这样的值。以及还有涉及到是否分页，每个字段的排序呀等操作。

如果有一个组件需要和前面的组件相关联你会怎么做

我主要是提供三种做法：

1. 类似于省份城市这种关联的情况我是有专门为它们定制一个组件的

夸克扫描王 极速扫描，就是高效

2. 类似于行业/职业这种不确定有几层的情况我也有专门为它们定制一个组件，当每次选择了前一个下拉框中的选项之后更加选项的内容动态生成一个新的下拉框
3. 与后台配合，如果后一个子组件和前面一个相关联的话，在选择了前一个组件之后发送请求给后台，后台返回你需要改动的子组件的信息，然后前端再进行更新
4. 每种子组件都会有一个回调事件，可以在业务代码中判断是哪个子组件发起的事件，然后根据这个来判断。

说一下你们的工作流

说一下你在项目中碰到的难点

Vue的双向绑定原理

Object.defineProperty()有什么缺点？Vue3为什么用Proxy?

nextTick实现原理

nextTick中的waiting是什么时候变为true的呢

在下次DOM更新循环结束之后

说一下虚拟DOM的作用以及实现原理

Vue3有哪些新的API或者有做哪些优化?

performance

首先在性能(performance)上有了更多的优化，一方面表现在 `virtual dom` 的生成上更快了，另外在底层还做了一些监听的缓存，也就是事件在被创建的时候会被推进一个缓存中，后续没有改变会直接取缓存。

tree-shaking

tree-shaking它表示的是在打包的时候会去除一些无用的代码。而在Vue3中对它的支持更加友好了，例如像transition、v-model、computed等功能没有用到的话，那么最后打包产生的代码就会将它们去除。也

夸克扫描王 极速扫描，就是高效

就是说，如果你的Vue项目只写了一个Hello Word的话，那么最后打包的代码中就只有一些核心的代码，如更新算法、响应式等，打包生成的文件可能就只有13.5kb。

Fragments

碎片(Fragments)，原本在Vue2.x中每个template下只能允许有一个根节点，但是在Vue3中它可以允许你有多个，用尤大大的话来说就是会将这些内容自动变为一个碎片。

TS

再者就是对TS的支持度很好。虽然Vue3本来就是用TS写的，但是不一定要用TS。另外它也支持Class Component，不过不是第一推荐。

Component API

语法上，对模版语法是零改变的。只不过更加推荐用Component API来写JS部分。Component API它并不是语法，而是新增的API。它带来的好处一个是逻辑重用，方便我们把一些功能的部分抽离出来。另一个它相对于options来说更加集中，用options来写代码想要追寻一个变量的变化比较麻烦。

关于兼容性

目前的Vue3.beta版本是不支持IE11的，因为核心的响应式原理用到了ES6的Proxy，但是以后会去兼容IE11。后面我们在创建一个Vue项目的时候，可以选择不同的版本，支持IE11和不支持IE11的。

有关HTTP缓存的首部字段说一下

必问...

HTTP中的keep-alive有了解吗?

`Connection` 的一个属性值， `HTTP/1.1` 才被标准化，之前一直为 `Close` 。

作用是建立长连接， 建立一次TCP请求即可发送多次HTTP请求。

在一次传输中它是如何保证每个数据包之间的顺序的?

TCP的报文中有序号和确认号来保证每个数据包之间的顺序。

HTTP/2有了解过吗?

- 使用 `HPACK` 算法进行头部压缩， `HTTP/2` 当中废除了起始行的概念，将起始行中的请求方法、URI、状态码转换成了头字段，不过这些字段都有一个 `":"` 前缀，用来和其它请求头区分开。
- 使用二进制分帧解决了HTTP层面的队头阻塞，即多个请求都通过一个TCP连接并发地完成，实现多路复用。

夸克扫描王 极速扫描，就是高效

- 服务器推送。服务器不再是完全被动地接收请求，响应请求，它也能新建 stream 来给客户端发送消息
- 新的二进制格式。HTTP/2采用二进制格式传输数据，相比于HTTP/1.1的文本格式，二进制格式具有更好的解析性和拓展性。

具体说一下HTTP/2中的多路复用

HTTP/2引入了一个二进制分帧层，客户端和服务端进行传输时，数据会先经过二进制分帧层处理，转化为一个个带有请求ID的帧，这些帧在传输完成后根据ID组合成对应的数据。

所以原本一个个完整的HTTP请求报文变成了现在的一堆乱序的二进制帧，这些二进制帧因为不存在先后顺序所以也就不会排队等待，也就没有了HTTP队头阻塞的问题。

通信双方都可以给对方发送二进制帧，这种二进制帧的双向传输的序列，就叫做流。HTTP/2在一个TCP连接上来进行多个数据帧的通信，这就是多路复用的概念。

(所谓的乱序，指的是不同 ID 的 Stream 是乱序的，但同一个 Stream ID 的帧一定是按顺序传输的。)

你认为你的优点是什么?

1. 从领导以及同事的评价来说，都说我是一个很靠谱的人，因为我对答应的事或者分配的任务我都做到或者很好的完成，所以第一点对于工作我是比较认真负责的。
2. 因为自己呆的团队都不大，所以很多时候自己不仅仅是前端，也会站在用户的角度去看用户在使用我们项目的时候会有哪些痛点，或者如何去做一些优化，提出一些合理的意见并最终被采纳。举几个例子：一、我们项目中在订单页面会有一个审批记录的列表，这个审批记录的列表是紧靠在页面的右侧可进行收缩折叠的。而这个东西最开始里面显示的内容会很多，比如每个阶段审批的人，时间，备注，同意还是拒绝等等信息。而我当时想的是，站在用户的角度来说，他看到这个记录的时候最关心的应该是审批的人，以及审批时间还有到了哪个阶段，而一些无关紧要的信息没必要开始就带出来。所以我当时就有一个提议：每条记录只显示出用户最关心的东西，而不重要的可以等用户鼠标移动上去的时候再进行一个显示。并且对整个审批记录做了一些精简的处理，使它不那么冗长，比如判断时间，如果是今年的话，就把 `2020-08-08` 设置为 `08-08` 。二、因为考虑到用户可能会频繁的切换某些页面，而项目中的页面菜单的层级有时候可能会比较深，不方便来回切换。我就有提议在当前页面的顶部实现一个导航栏，每当进入一个新的页面的时候，导航栏就会新增一个标签，记录当前页面，这样相当于留了一个快捷的入口给用户，方便他们进行页面的切换，同时导航栏也会有例如删除左侧的全部标签，删除右侧，只保留当前页等功能。
3. 我是一个比较喜欢总结和分享的人，所以在工作之余也会经常写博客，目前的更新频率是会保持每周有一篇比较高质量的文章。也一直保持着自己对前端的这份热情。

阿里

> 5月21日

面试部门：财务钉钉

夸克扫描王 极速扫描，就是高效

一面

面试形式：电话面试

面试时长：30分钟

面试感悟：阿里的面试是子弈小哥哥帮忙内推的，算是放在最后来了，和之前了解的一样，面试官可能会针对于你说的内容然后深挖，所以你说的每一句话都必须考虑清楚才好。一面就挂了，归根到底的原因不解释，技术还不到家，对一些知识点的理解还不够深刻。但是抛开了技术以外的原因，我认为面试时状态的不佳也是导致这次面试失败的一个原因。因为当时是晚上7点进行的电话面试，一整天都在忙于复习没有休息，并匆匆的吃了晚饭，所以到了面试那个点自身的精神状态已经非常不好了，导致面试时没有一点精神劲。举个例子，面试官问我的第一个问题，能说一下 `position` 属性有哪些值吗？我把 `position` 这个单词重复了好几遍大脑一片空白，然后说了一句： `"不好意思面试官，您能把这个单词拼一下吗"` ...

这听起来可能特别的好笑🤭，但是是真实发生，所以也是想要告诉各位小伙伴，面试前的准备确实要充分，但是自己的状态以及心态也非常的重要，不要因为一些非技术以外的原因让自己遗憾...

position属性有哪些值分别介绍一下

- 静态定位(static)：默认的定位，也就是说网页中所有元素默认都是静态定位的，它始终会处于文档流给予的位置。
- `inherit` ：规定应该从父元素继承 `position` 属性的值。但是任何的版本的 `IE` (包括 IE8)都不支持属性值 `inherit` 。
- 相对定位(relative)：相对于该元素在文档中的初始位置进行定位。通过 “left”、“top”、“right” 以及 “bottom” 属性来设置此元素相对于自身位置的偏移。它不会脱离标准的文档流。
- 绝对定位(absolute)：相对的元素是它**最近的一个祖先，该祖先满足：position的值必须是：relative、absolute、fixed，若没有这样的祖先则相对于body进行定位**。盒子的偏移位置不影响常规流中的任何元素，其margin不与其他任何margin折叠。
- 固定定位(fixed)：生成绝对定位的元素。默认情况下，可定位于相对于浏览器窗口的指定坐标。元素的位置通过 “left”, “top”, “right” 以及 “bottom” 属性进行规定。不论窗口滚动与否，元素都会留在那个位置。但当祖先元素具有 `transform` 属性且不为none时，就会相对于祖先元素指定坐标，而不是浏览器窗口。

relative的定位规则

- 相对于该元素在文档中的初始位置进行定位。通过 “left”、“top”、“right” 以及 “bottom” 属性来设置此元素相对于自身位置的偏移。
- 如果他原来在常规流的默认位置改变了，那他也会跟着变位置，永远围着整个body自己原来的那一小块老地方转。所以说相对定位没有脱离文档流。

脱离文档流是会呈现什么样的效果呢?

夸克扫描王 极速扫描，就是高效

脱离文档流，也就是**将元素从普通的布局排版中拿走**，其他盒子在定位的时候，会当做脱离文档流的元素不存在而进行定位。

而在 `CSS` 中，使用 `float` 和设置 `position:absolute` 都会使得元素脱离文档流。只不过它两的区别是：

使用 `float` 脱离文档流时，其他盒子会无视这个元素，但其他盒子内的文本依然会为这个元素让出位置，环绕在周围。而对于使用 `position:absolute` 脱离文档流的元素，其他盒子与其他盒子内的文本都会无视它。

常规流(文档流)是个怎样的排列关系

将窗体自上而下分成一行一行,并在每行中按从左至右的挨次排放元素。

inline-block的使用场景

1. 要设置某些子元素在一行或者多行内显示，尤其是排列方向一致的情况下，应尽量用 `inline-block` 。
2. 希望若干个元素平行排列，且在父元素中居中排列，此时可以用 `inline-block` ，且给父元素设 `text-align: center` 。
3. `inline-block` 可以用一排 `a {display: inline-block}` 实现横向导航栏，无论是居左的导航栏还是居右的都适用。

对于第一种和第三种情况虽然都可以使用 `float` 来实现，不过 `inline-block` 会比它好一些，原因如下：

- 浮动导致父元素高度塌陷的问题

(望补充)

GET和POST的区别

从缓存的角度上说，GET会被浏览器主动缓存下来，留下历史记录，但是POST不会。

从编码的角度上说，GET只能进行URL编码，它只能接收ASCII字符，但是POST没有限制。

从参数的角度上说，GET一般放在URL上传递参数，POST放在请求体里，更适合传递敏感信息。

从幂等的角度上说，GET是幂等的，而POST不是。

不过据我了解的，其实GET和POST本质上都是TCP连接，并无差别。但是由于HTTP的规定和浏览器/服务器的限制，导致它们在应用过程中体现出一些不同。

还有可以从TCP的角度上说，GET请求会把请求报文一次性发出去，但是POST会分为两个TCP数据包。首先发送的是header部分，若是服务器响应100(continue)，则会发送body部分，当然**火狐**浏览器除外，它的 POST 请求只发一个 TCP 包。

夸克扫描王 极速扫描，就是高效

为什么说GET会留下历史记录?

浏览器规定的

GET可以上传图片吗?

get可以通过URL传参上传图片，可以将图片转为base64格式再进行传输，不过由于get请求的参数是有大小限制的，最多不超过4kb。

GET和POST的安全性问题为什么说POST相对安全一些

首先get只能使用URL编码，post支持各种类型的编码。然后get参数在URL上，容易暴露，所以说post相对安全一些。

GET就一定是幂等的吗?

不一定，我们平常说的幂等只是一种约定，最终幂等还是不幂等取决于服务器怎么处理请求。

说一下你所知道的缓存方案

你们为什么要做这个动态表单？(业务痛点)

动态表单具体的难点在哪?

有没有了解行业内其它做动态表单这一块的

问卷网：https://www.wenjuan.com/list

功能主要是一些问卷调查的形式，例如员工培训满意度、投票评选之类的。

魔方网表：http://www.mf999.com/index.html

UI样式较旧，类似老套的系统。

你还有什么想要问我的吗?

夸克扫描王
极速扫描，就是高效

深圳开思

> 5月25日

面试部门：某业务部门

一面

面试形式：现场面

面试时长：40分钟

面试感悟：面试开始前，看到面试官在面另一个小哥，隐约听到小哥被面试官怼的好惨...所以面试前内心还有点坎坷。但是当真正去面试的时候，面试官好像没有那么凶了，甚至自我感觉眼神中透露着对我的不怀好意...原来他说他有看过我写的文章，说得想点我不知道的考考我... 😅

说一说原型链

(面试官叫我详细描述一下，然后说自己也学习一下...我： `"不不不，您太谦虚了。"`)

null为什么被typeof错误的判断为了'object'

说一说几种this指向

- 默认绑定(非严格模式下this指向全局对象, 严格模式下 `this` 会绑定到 `undefined`)
- 隐式绑定(当函数引用有**上下文对象**时, 如 `obj.foo()` 的调用方式, `foo` 内的 `this` 指向 `obj`)
- 显示绑定(通过 `call()` 或者 `apply()` 方法直接指定 `this` 的绑定对象, 如 `foo.call(obj)`)
- new绑定
- 箭头函数绑定(`this` 的指向由外层作用域决定的)

使用箭头函数时需要注意什么?

1. 不能用于构造函数

```js
let Person = (sex) => {
  this.sex = sex
```


夸克扫描王 极速扫描，就是高效

```
}
console.log(new Person('boy'))
```

结果为：

```
Uncaught TypeError: Person is not a constructor
```

2. 不要用于事件绑定

例如：

```
const btn = document.getElementById('btn');
btn.addEventListener('click', function() {
  console.log(this) // window
})

const btn = document.getElementById('btn');
btn.addEventListener('click', () => {
  console.log(this) // <button id="btn">按钮</button>
})
```

可以看到，使用箭头函数进行事件绑定，会丢失原本的 `this` 指向。

3. 原型链上添加方法也不能用箭头函数

说一下JS内置对象

构造器内置对象：

- Object
- Boolean
- String
- Number
- Function
- Array
- RegExp
- Date
- Error

其它对象：

夸克扫描王 极速扫描，就是高效

- Math
- JSON
- 全局对象
  - NaN
  - parseInt
  - encodeURIComponent

描述一下作用域链

(答案参考：《面试分享：两年工作经验成功面试阿里P6总结》)

当代码在一个环境中创建时，会创建变量对象的一个作用域链（scope chain）来保证对执行环境有权访问的变量和函数。作用域第一个对象始终是当前执行代码所在环境的变量对象（VO）。如果是函数执行阶段，那么将其activation object（AO）作为作用域链第一个对象，第二个对象是上级函数的执行上下文AO，下一个对象依次类推。

在《JavaScript深入之变量对象》中讲到，当查找变量的时候，会先从当前上下文的变量对象中查找，如果没有找到，就会从父级(词法层面上的父级)执行上下文的变量对象中查找，一直找到全局上下文的变量对象，也就是全局对象。这样由多个执行上下文的变量对象构成的链表就叫做作用域链。

闭包的使用场景

闭包产生的本质就是，当前环境中存在指向父级作用域的引用。

- 函数防抖
- 使用闭包设计实现单例模式
- 为多个组件独立属性，例如在闭包中进行 `id` 的累加
- 设置私有变量
- 解决经典问题 `for` 循环和 `setTimeout`

具体可以看：闭包实际场景应用

使用闭包需要注意什么

闭包有三个特性：

1. 函数嵌套函数；
2. 内部函数使用外部函数的参数和变量；
3. 参数和变量不会被垃圾回收机制回收。

所以可以看到它的缺点：

夸克扫描王 极速扫描，就是高效

1. 常驻内存，增加内存使用量；

2. 使用不当造成内存泄漏。

因此我们在使用时需要注意不用的变量要及时的清理掉。

EventLoop

用过哪些ES6的功能

谈谈你对模块的理解

(答案参考：ES6专题—class与模块化（10）)

模块可以理解为函数代码块的功能，是封装对象的属性和方法的javascript代码，它可以是某单个文件、变量或者函数。

模块实质上是对业务逻辑分离实现低耦合高内聚，也便于代码管理而不是所有功能代码堆叠在一起，模块真正的魔力所在是仅导出和导入你需要的绑定，而不是将所有的东西都放到一个文件。

在理想状态下我们只需要完成自己部分的核心业务逻辑代码，其他方面的依赖可以通过直接加载被人已经写好模块进行使用即可。

二面

(项目负责人)

面试官：你好，你可以脱下口罩，我们面对面交流一下。

我：...(难道颜值加分吗？那我明天就可以来上班了...)

都是项目相关的问题，然后：

- 之前的团队人员分配情况
- 职业规划
- 期望薪资
- 有什么想问的

HR面

常规问题...和上面问的差不多...

开思也算是深圳的一家比较大的汽车金融公司吧，这里面的也是3～5年的岗位，其实从一面之后就有预感对方挺中意我的了，但是在最后HR面的时候他们可能也觉得达不到我的期望薪资，所以也是在我回家之后与我电话沟通，表明如果以后有想继续留在深圳的话可以再考虑一下。应该说虽无缘但也算互相结交了一下吧。

夸克扫描王 极速扫描，就是高效

头条

格式化发布时间

> 类似于微信朋友圈里面的发布时间，其中发布时间有几种特殊情况：
>
> - 小于 1 min，则显示 **刚刚**；
> - 小于 1 h，则显示 x **分钟前**；
> - 小于 1 天，则显示 x **小时前**；
> - 小于 1 个星期，则显示 x **天前**；
> - 否则显示具体时间。

```ts
declare function i18n(s: string): string

function format(date: number): string {
  const MIN = 60 * 1000
    const HOUR = 60 * MIN
  const DAY = 24 * HOUR
  const WEEK = 7 * DAY

  if(date < MIN) return i18n('刚刚')
  if(date < HOUR) return Math.floor(date / MIN) + i18n('分钟前')
  if(date < DAY) return Math.floor(date / HOUR) + i18n('小时前')
  if(date < WEEK) return Math.floor(date / DAY) + i18n('天前')
  return new Date(Date.now() - date).toLocaleString()
}

console.log(format(60 * 1000 - 1)); // 59 秒 999
console.log(format(60 * 1000)); // 1 分

console.log(format(60 * 1000 * 59 + 59 * 1000 + 999)); // 59 分 59 秒 999
console.log(format(60 * 1000 * 60)); // 1 小时

console.log(format(60 * 1000 * 60 * 23 + 60 * 1000 * 59 + 59 * 1000 + 999)); // 23 小时 59
分 59 秒 999
console.log(format(60 * 1000 * 60 * 24)); // 1 天

console.log(format(60 * 1000 * 60 * 24 * 6)); // 6 天
console.log(format(60 * 1000 * 60 * 24 * 7)); // 7 天

console.log(format(1554111847534)); // 发布时的时间戳
```


夸克扫描王 极速扫描，就是高效

格式化数字

> 为数字添加都好，使用正则和非正则实现

```ts
// 方法1：使用内置api
// 缺点：小数部分会保留三位小数
var num = 12345
console.log(num.toLocaleString()) // '12,345'
num = 12345.6789
console.log(num.toLocaleString()) // '12,345.679'

// 方法2：使用数组
function format(num: number): string {
  if(typeof num !== 'number') return num

  const [first, last] = `${num}`.split('.')
  const len = first.length
    const temp = first.split('').reverse().reduce((pre, cur, index) => {
    if((i + 1) % 3 === 0 && i !== flen - 1) pre.push(cur, ',')
    else pre.push(cur)
  }, []).reverse().join('')

  return last ? `${temp}.${last}` : temp
}
```

一面

实现 js ES5 数据类型的深拷贝

```ts
function deepClone<D>(data: D): D {
  const isObj = (v: D) => Object.prototype.toString.call(v).slice(8, -1) === 'Object'

  function _deepClone(val: any): D {
    if(Array.isArray(val)) {
            const source = val as any[]
      return source.reduce((res, item) => {
        res.push(_deepClone(item))
        return res
      }, [])
    }

    if(isObj(val)) {
```


夸克扫描王 极速扫描，就是高效

```
    const source = val as object
    return Object.keys(source).reduce((res, key) => {
      res[key] = _deepClone(source[key])
      return res
    }, {})
  }

  return val
 }

 return _deepClone(data)
}
```

二面

实现简易 MVVM

```ts
interface Handlers {
    get(val): void
    set(newVal, oldVal): void
}

class Observer<T extends object = {}> {
    private data: T

    constructor(data: T) {
        this.data = data
        this.proxyData()
    }

    private proxyData(): void {
        const keys = Object.keys(this.data)
        for (const key of keys) {
            Object.defineProperty(this, key, {
                get: () => {
                    return this.data[key]
                },
                set: (val) => {
                    this.data[key] = val
                }
            })
        }
    }

    observe(key: keyof T, handlers: Handlers): void {
        const value = this.data[key]

        Object.defineProperty(this.data, key, {
```


夸克扫描王 极速扫描，就是高效

```
            get: () => {
                handlers.get(value)
                return value
            },
            set: (val) => {
                if (val === value) return
                handlers.set(val, value)
                this.data[key] = val
            }
        })
    }
}
```

三面

顺序发送4个请求a，b，c，d，要求按顺序输出

```ts
function getData(urls: string) {
  return new Promise<object[]>((resolve, reject) => {
    const len = urls.length
    const res: object[] = []

    urls.forEach((url, index) => {
      fetch(url)
        .then(res => res.json())
        .then(data => {
            res[index] = { data, printed: false }
            let flag = true // 用来判断所有请求是否都已经跑完
          for(let i = 0; i < len && flag; i++) {
            if(res[i]) {
              if(!res[i].printed) { // 循环内，所以去掉已经打印的
                console.log(res[i].data)
                res[i].printed = true
                i === len - 1 && resolve(res.map(v => v.data)) // 当最后一个打印完成后，结束
promise
              }
            } else {
              flag = false
            }
          }
        }, reject)
    })
  })
}
```


夸克扫描王 极速扫描，就是高效

美团

二面

用 promise 实现一个请求超时功能

实现关键在于 promise 的状态改变是单向且唯一的，即只能由 `pending -> fulfilled` 或者 `pending -> rejected` ，所以可以使用 `resolve + setTimeout` 来实现。

```ts
function promiseWithTimeout(url, delay = 3000) {
  return new Promise((resolve, reject) => {
    setTimeout(function () {
      reject(Error('time is out!'))
    }, delay)
    fetch(url).then(data => resolve(data.json()))
  })
}
```

综合面试题

[高德一面] 一个 tcp 连接能发几个 http 请求

如果是 HTTP 1.0，默认不支持长连接，所以 一个 TCP 发送 一个 HTTP 请求；

如果是 HTTP 1.1，支持了长连接，只要 TCP 连接不断开，便可以一直发送 HTTP 请求，HTTP 2.0 同理

[腾讯一面] V-DOM 的优势

DOM 引擎 、JS 引擎相互独立，又工作在同一线程。JS 代码调用 DOM API 必须挂起 JS 引擎、转换传入参数数据、激活 DOM 引擎，DOM 重绘后再转换可能有的返回值，最后激活 JS 引擎并继续执行。若频繁的 DOM API 调用，重新计算布局、重新绘制图像会引起更大的性能消耗。

优势：

1. V-DOM 不会立马进行重排和重绘
2. V-DOM 进行频繁修改，然后一次性比较并修改真实 DOM 中需要改的部分，最后在真实 DOM 中进行排版重绘，减少过多的 DOM 节点排版与重绘损耗

夸克扫描王 极速扫描，就是高效

3. V-DOM 有效降低大面积真实 DOM 的重绘与排版，因为最终与真实 DOM 比较差异，可以只渲染局部

[字节跳动] CommonJS 和 es6 模块引用的区别

目前浏览器端虽写法是以 esm 为主，但是各种前端工具转换为 cjs，差别：

1. cjs 模块输出的是一个值的拷贝，esm 模块输出的是值的引用
2. cjs 模块是运行时加载，esm 模块是编译时输出接口
3. cjs 是单个值导出，esm 允许导出多个
4. cjs 是动态语法，可以写在判断内，esm 是静态语法，只能写在顶层
5. cjs 的 this 是当前模块，esm 的 this 是 undefined

[头条] 如何选择图片格式

图片格式	压缩方式	透明度	动画	浏览器兼容	适应场景
JPEG	有损压缩	不支持	不支持	所有	复杂颜色及形状、尤其是照片
GIF	无损压缩	支持	支持	所有	简单颜色、动画
PNG	无损压缩	支持	不支持	所有	需要透明度时
APNG	无损压缩	支持	支持	Firefox、Safari、IOS Safari	需要半透明效果的动画
WebP	有损压缩	支持	支持	Chrome、Opera、Android、Chrome Android、Browser	复杂颜色以及形状，浏览器平台可预知
SVG	无损压缩	支持	支持	所有（IE8以上）	简单图形，需要良好的缩放体验，需要动态控制图片特效

首屏和白屏时间如何计算

首屏时间计算，可以由 Native WebView 提供类似 onload 的方法实现，在 ios 下对应的是 webViewDidFinishLoad，在 android 下对应的是 onPageFinished 事件。

白屏的定义有很多种，场景不同，时间计算方式不同。

夸克扫描王 极速扫描，就是高效

小程序 和 H5 的区别

1. 渲染方式不同，小程序时 native 渲染，同时也支持 web 渲染
2. 小程序特有的双线程设计

如何判断 0.1 + 0.2 与 0.3 相等?

1. 非 ES 独有
2. IEEE754 标准中 64 位的存储格式，比如 11 位存偏移值
3. 其中涉及的三次精度丢失

JavaScript 深入之浮点数精度

参考 math.js big.js

[腾讯二面] v8 引擎执行一段 js 代码

在执行一段代码时，JS 引擎首先会创建一个执行栈

然后 JS 引擎会创建一个全局执行上下文，并 push 到执行栈中，这个过程 JS 引擎会为这段代码中所有的变量分配内存并赋一个初始值(undefined)，在创建完成之后，JS 引擎会进入执行阶段，这个过程 JS 引擎会逐行执行代码，并为之前分配好内存的变量逐个赋真实值。

如果这段代码中存在 function的声明和调用，那么 JS 引擎会创建一个函数执行上下文，并 push 到执行栈中，其创建过程与全局执行上下文一样。但有特殊情况，即当函数中存在对其他函数的调用时，JS 引擎会在父函数执行的过程中，将子函数的全局执行上下文 push 到执行栈，这也是为什么子函数能够访问到父函数内所声明的变量。

还有一种特殊情况是，在子函数执行的过程中，父函数已经 return 了，这种情况下，JS 引擎会讲父函数的上下文从执行栈中移除，与此同时，JS 引擎会为还在执行的子函数上下文创建一个闭包，这个闭包里保存了父函数内生命的变量以及其赋值，子函数仍然能够在其上下文中访问并使用这些变量/常量。当子函数执行完毕，JS 引擎才会将子函数的上下文以及闭包一并从执行栈中移除。

中大厂面试总结

下面题目中标记有【高频】至少被问过两次，标记有【超高频】的基本面试的公司都有问道。

笔试题

1. 【超高频】手写深拷贝，考虑 **正则**、Date

夸克扫描王 极速扫描，就是高效

2. 【高频】实现一个 Vue 自定义指令懒加载
3. 判断 DOM 标签的合法性，标签的闭合，span 里面不能有 div，写一个匹配 DOM 标签的正则
4. 替换日期格式， `xxxx-yy-zz` 替换成 `xxx-zz-yy` ，可以使用正则捕获组来实现

```js
var reg = /(\d{2})\.(\d{2})\/(\d{4})/
var date = '10.24/2017'
date = date.replace(reg, '$3-$1-$2')
console.log(date) // 2017-10-24
```

5. 【高频】实现 Promise.all、Promise.allSettled
6. 获取一段 DOM 节点中标签个数最多的标签
7. 写一个简单的 diff
8. 【高频】手写节流
9. 手写 es6 继承
10. 实现一个自定义hook -- usePrevious

可以参考 usePrevious 这个的实现

```ts
import { useRef } from 'react'

export type compareFunction<T> = (prev: T | undefined, next: T) => boolean;

export default <T>(state: T, compare?: compareFunction<T>): T | undefined => {
  const prevRef = useRef<T>()
  const curRef = useRef<T>()

  const needUpdate = typeof compare === 'function' ? compare(curRef.current, state) : tru
e

  if(needUpdate) {
    prevRef.current = curRef.current
    curRef.current = state
  }

  return prevRef.current
}
```

11. 【高频】实现一个 vue 的双向绑定

算法题

夸克扫描王 极速扫描，就是高效

1. 二叉树的最大深度
2. 另一棵树的子树
3. 相同的树
4. 翻转二叉树
5. 【高频】斐波那契数列
6. 【高频】合并两个有序数组
7. 【高频】打乱数组

webpack、babel

1. babel 的缓存是怎么实现的
2. webpack 的 HMR 怎么配置：

- 浏览器是如何更新的
- 如何做到页面不刷新就能自动更新
- webpack-dev-server webpack-dev-middleware

3. 有没有写过 ast，webpack 通过什么把公共部分抽出来，属性配置是什么
4. webpack 怎么配置 mock 转发代理，mock的服务，怎么拦截转换的
5. webpack 的 plugin 和 loader 的编写，调用顺序
6. webpack 的打包构建优化，体积和速度
7. DLLPlugin 原理，为什么不直接使用压缩版本的 js

HTTP

1. 【超高频】缓存（强缓存），如何设置缓存
2. 【高频】HTTP2，HTTP2的性能优化方面，真的优化很多吗
3. 【高频】简单请求和复杂请求
4. 【高频】HTTPS的整个详细过程
5. 301 和 302 的区别
6. 怎么用 get 实现 post，就是使用 get 方法但是参数放到 body 里面
7. TCP 和 UDP 的区别

CSS

1. 【超高频】flex

- 说一下 flex

夸克扫描王 极速扫描，就是高效

- flex：1 具体代表什么，有什么应用场景
- flex-basic 是什么含义

2. css var 自定义变量的兼容性
3. 行内元素和块级元素的区别
4. position 有哪些值，分别是什么还以
5. 盒模型
6. CSS 实现

- 淘宝购物车添加商品到购物车的动画（CSS 实现抛物线运动效果）
- tooTip 的实现

7. 【高频】实现固定宽高比的 div
8. 【高频】伪类和伪元素

js

1. 单例应用
2. 【超高频】什么是闭包，闭包的应用场景
3. 如何判断当前浏览器是否支持 webp
4. Vue3 和 Vue2 实现双向绑定的区别，然后手写两种方式，proxy 除了拦截 getter setter 之外还能做什么
5. 若一下你对同步阻塞，异步非阻塞的理解
6. 弱引用，WeakMap 和 Map 的区别
7. 【高频】XSS 反射型是什么，如何避免
8. 【超高频】事件循环
9. 【超高频】说一下你对 promise 的理解
10. 【超高频】浏览器渲染（从输入 url 到页面渲染完成的整个过程）
11. 【超高频】首屏加载优化，通过哪些指标去衡量性能优化
12. canvas 和 svg 区别？优缺点？
13. 牛客网如何监听你调用了其他页面

```js
document.addEventListener('visibilitychange', function() {
  console.log(document.hidden)
})
```


夸克扫描王
极速扫描，就是高效

14. js 原生 3种绑定事件

```html
<!-- 第一种 直接在标签里绑定 -->
<button id="btn" onclick="handleClick()">
  自定义函数
</button>

<script>
// 利用 DOM0 进行绑定
  var btn = document.getElementById('btn')
  btn.onclick = handleClick()

  // 利用 DOM3 进行绑定
  btn.addEventListener('click', handleClick)
</script>
```

15. 说一下 websocket
16. 【超高频】实现复杂数据去重
17. 基本数据类型有哪些，为什么 symbol 是一个函数，BigInt 为什么可以用来存储大整数
18. 什么是依赖注入
19. js类型转换
20. 富文本编辑器相关的 js 知识
21. cli 工具的一些实现逻辑

Vue

1. 【高频】vue3 新特性，比较 composition 和 hooks 的区别
2. new Vue 做了什么
3. 双向绑定原理
4. vue 组件通信方法

React

1. 【高频】hooks 相关

- 为什么引入，什么原理
- 如何监听响应，内部是如何做到只有数据修改时才执行函数
- 依赖的值发生变化，需要不停的绑定和监听事件
- render props 和 hoc 相比的优缺点
- 和 mixin，hoc区别在哪

夸克扫描王 极速扫描，就是高效

2. 创建 ref 的几种方法
3. context 怎么使用，内部原理怎么做到
4. 【超高频】生命周期
5. redux

- 使用方法，为什么 action 要返回一个函数，返回一个对象可以吗
- state 为什么要设计成不可变

夸克扫描王
极速扫描，就是高效